



Fundamentos de Python I

Curso de Cisco

MARTIN ALBERTO BENITES MARIN



TABLA DE CONTENIDO

1	Python y la Programación Informática	5
1.1	Introducción a la programación.....	5
1.1.1	¿Cómo funciona un programa de computadora?	5
1.1.2	Lenguajes naturales vs lenguajes de programación	5
1.1.3	¿Qué compone a un lenguaje?	6
1.1.4	Lenguaje máquina vs. lenguaje de alto nivel.....	6
1.1.5	Compilación vs. Interpretación.....	7
1.1.6	¿Qué hace el intérprete?	7
1.1.7	Compilación vs. Interpretación – Ventajas y Desventajas	8
1.2	Introducción a Python	9
1.2.1	¿Qué es Python?	9
1.2.2	¿Quién creó Python?	9
1.2.3	Un proyecto de programación por pasatiempo	10
1.2.4	¿Qué hace que Python sea tan especial?	10
1.2.5	¿Rivales de Python?	11
1.2.6	¿Dónde podemos ver a Python en acción?.....	11
1.2.7	¿Por qué no Python?	11
1.2.8	Python 2 vs. Python 3.....	12
1.2.9	Implementaciones de Python	13
2	Tipos de datos, Variables, Operaciones Básicas de Entrada y Salida, Operadores Básicos	14
2.1	El Programa “Hola Mundo”	14
2.1.1	La función print()	14
2.1.2	Argumentos de funciones	15
2.1.3	Invocación de funciones	16
2.1.4	La función print() y su efecto, argumentos, y valores retornados	17
2.1.5	Instrucciones	17
2.1.6	Caracteres de escape y nueva línea en Python.....	19
2.1.7	Usando múltiples argumentos	20
2.1.8	Argumentos posicionales	20
2.1.9	Argumentos de palabra clave.....	20
2.1.10	Resumen de Sección	23
2.2	Literales de Python.....	23
2.2.1	Literales - los datos en sí mismos	23
2.2.2	Enteros	24

2.2.3	Flotantes	26
2.2.4	Codificando Flotantes	27
2.2.5	Cadenas	28
2.2.6	Valores Booleanos.....	28
2.2.7	Resumen de Sección.....	29
2.3	Operadores - herramientas de manipulación de datos.....	30
2.3.1	Python como una calculadora.....	30
2.3.2	Operadores Básicos	30
2.3.3	Operadores y sus prioridades	35
2.4	Variables	38
2.4.1	Variables – cajas con forma de datos	38
2.4.2	Nombres de Variables.....	38
2.4.3	Cómo crear una variable.....	39
2.4.4	Cómo asignar un nuevo valor a una variable ya existente	40
2.4.5	Operadores Abreviados.....	41
2.4.6	Resumen de Sección.....	42
2.5	Comentarios	42
2.5.1	Comentarios – ¿por qué, cuándo, y dónde?	42
2.5.2	Marcar fragmentos de código	43
2.5.3	Resumen de Sección.....	43
2.6	Interacción con el usuario.....	44
2.6.1	La función input().....	44
2.6.2	La función input() con un argumento	45
2.6.3	El resultado de la función input()	46
2.6.4	Conversión de tipos (conversiones de tipos)	46
2.6.5	Más sobre input() y conversión de tipo	47
2.6.6	Operadores cadena.....	48
2.6.7	Conversiones de tipos una vez más	49
3	Valores Booleanos, Ejecución Condicional, Bucles, Listas y su procesamiento, Operaciones Lógicas y de Bit a Bit.....	49
3.1	Como tomar decisiones en Python.....	49
3.1.1	Preguntas y Respuestas	49
3.1.2	Comparación: operador de igualdad	50
3.1.3	Operadores	50
3.1.4	Haciendo uso de las respuestas.....	51
3.1.5	Condiciones y ejecución condicional	52

3.1.6	Pseudocódigo e introducción a los bucles	56
3.2	Bucles en Python	58
3.2.1	Un bucle infinito	59
3.2.2	Bucles en tu código con for.....	59
3.2.3	Más acerca del bucle for y la función range() con tres argumentos	60
3.2.4	Las sentencias break y continue	61
3.2.5	El bucle while y el bloque else	62
3.2.6	El bucle for y el bloque else	62
3.3	Operaciones lógicas y de bits en Python	63
3.3.1	Lógica de computadoras.....	63
3.3.2	Valores lógicos vs bits individuales	64
3.3.3	Operadores bit a bit	64
3.3.4	¿Cómo tratar con bits individuales?.....	66
3.3.5	Desplazamiento binario a la izquierda y desplazamiento binario a la derecha	68
3.4	Listas	70
3.4.1	¿Por qué necesitamos listas?	70
3.4.2	Acceso al contenido de las listas.....	72
3.4.3	Eliminando elementos de una lista.....	72
3.4.4	Los índices negativos son legales	73
3.4.5	Funciones vs métodos	73
3.4.6	Agregando elementos a una lista: append() y insert()	74
3.4.7	Listas en acción	75
3.5	Ordenando listas simples: el ordenamiento de burbuja	75
3.5.1	Ordenamiento Burbuja	75
3.5.2	Ordenando una lista	77
3.6	Operaciones con listas.....	78
3.6.1	La vida al interior de las listas	79
3.6.2	Rebanadas poderosas	79
3.6.3	Rebanadas - índices negativos	81
3.6.4	Los operadores in y not in	82
3.7	Aplicaciones avanzadas de listas.....	83
3.7.1	Listas dentro de listas	83
3.7.2	Arreglos de dos dimensiones.....	84
3.7.3	Naturaleza multidimensional de las listas: aplicaciones avanzadas	86
4	Funciones, Tuplas, Diccionarios, Excepciones y Procesamiento de Datos	88
4.1	Funciones	88

4.1.1	¿Por qué necesitamos funciones?.....	88
4.1.2	Descomposición.....	89
4.1.3	¿De dónde provienen las funciones?	90
4.1.4	Tu primera función	90
4.2	Cómo las funciones se comunican con su entorno	91
4.2.1	Funciones parametrizadas.....	91
4.2.2	Paso de parámetros posicionales.....	93
4.2.3	Paso de argumentos de palabra clave	93
4.2.4	Mezclando argumentos posicionales y de palabras clave	93
4.2.5	Funciones parametrizadas - más detalles	94
4.3	Retomando el resultado de una función	94
4.3.1	Efectos y resultados: la instrucción return.....	95
4.3.2	Unas pocas palabras sobre None	97
4.3.3	Efectos y resultados: listas y funciones	97
4.4	Los alcances en Python	98
4.4.1	Funciones y alcances	98
4.4.2	Funciones y alcances: la palabra clave global.....	99
4.4.3	Cómo interactúa la función con sus argumentos.....	100
4.4.4	Recursividad	101
4.5	Tuplas y diccionarios	103
4.5.1	Tipos de secuencia y mutabilidad	103
4.5.2	Tuplas	104
4.5.3	Diccionarios	106
4.5.4	Métodos y funciones de los diccionarios	108
4.6	Excepciones	111
4.6.1	Errores - el pan diario del desarrollador.....	111
4.6.2	La rama try-except.....	112
4.6.3	Cómo lidiar con más de una excepción	113
4.6.4	La excepción predeterminada y cómo usarla	114
4.6.5	Algunas excepciones útiles	115
4.6.6	Por qué no puedes evitar probar tu código	116
4.6.7	Pruebas, pruebas y más pruebas	116
4.6.8	print debugging (depuración).....	117
4.6.9	Algunos consejos útiles.....	118
4.6.10	Pruebas unitarias - un mayor nivel de codificación.....	118

1 PYTHON Y LA PROGRAMACIÓN INFORMÁTICA

1.1 INTRODUCCIÓN A LA PROGRAMACIÓN

1.1.1 ¿Cómo funciona un programa de computadora?

Un programa hace que una computadora sea usable. Sin un programa, una computadora, incluso la más poderosa, no es más que un objeto. Del mismo modo, sin un reproductor, un piano no es más que una caja de madera.

Las computadoras pueden realizar tareas muy complejas, pero esta habilidad no es innata. La naturaleza de una computadora es bastante diferente.

Solo puede ejecutar operaciones extremadamente simples. Por ejemplo, una computadora no puede comprender el valor de una función matemática complicada por sí misma, aunque esto no está fuera del alcance de la posibilidad en un futuro cercano.

Las computadoras contemporáneas solo pueden evaluar los resultados de operaciones muy fundamentales, como sumar o dividir, pero pueden hacerlo muy rápido y pueden repetir estas acciones prácticamente cualquier cantidad de veces.

Imagina que quieres saber la velocidad media que has alcanzado durante un viaje largo. Conoces la distancia, conoces el tiempo, necesitas la velocidad.

Naturalmente, la computadora podrá calcular esto, pero la computadora no es consciente de cosas como la distancia, la velocidad o el tiempo. Por lo tanto, es necesario instruir a la computadora para:

- aceptar un número que represente la distancia;
- aceptar un número que represente el tiempo de viaje;
- divide el valor anterior por el segundo y almacenar el resultado en la memoria;
- mostrar el resultado (que representa la velocidad promedio) en un formato legible.

Estas cuatro simples acciones forman un **programa**. Por supuesto, estos ejemplos no están formalizados y están muy lejos de lo que la computadora puede entender, pero son lo suficientemente buenos para ser traducidos a un idioma que la computadora pueda aceptar.

El **lenguaje** es la palabra clave.

1.1.2 Lenguajes naturales vs lenguajes de programación

Un lenguaje es un medio (y una herramienta) para expresar y registrar pensamientos. Hay muchos lenguajes a nuestro alrededor. Algunos de ellos no requieren ni hablar ni escribir, como el lenguaje corporal; es posible expresar tus sentimientos más profundos muy precisamente sin decir una palabra.

Otro lenguaje que utilizas cada día es tu lengua materna, que utilizas para manifestar tu voluntad y reflexionar sobre la realidad. Las computadoras también tienen su propio lenguaje, llamado lenguaje **máquina**, que es muy rudimentario.

Una computadora, incluso la más sofisticada técnicamente, está desprovista de cualquier rastro de inteligencia. Se podría decir que es como un perro bien adiestrado: responde sólo a un conjunto predeterminado de comandos conocidos.

Los comandos que reconoce son muy simples. Podemos imaginar que la computadora responde a órdenes como "toma ese número, divide por otro y guarda el resultado".

Un conjunto completo de comandos conocidos se llama **lista de instrucciones**, a veces abreviada **IL** (por sus siglas en inglés). Los diferentes tipos de computadoras pueden variar según el tamaño de sus **IL** y las instrucciones pueden ser completamente diferentes en diferentes modelos.

Nota: los lenguajes máquina son desarrollados por humanos.

Ninguna computadora es actualmente capaz de crear un nuevo idioma o lenguaje. Sin embargo, eso puede cambiar pronto. Por otro lado, las personas también usan varios idiomas muy diferentes, pero estos idiomas se crearon ellos mismos. Además, todavía están evolucionando.

Cada día se crean nuevas palabras y desaparecen las viejas. Estos lenguajes se llaman **lenguajes naturales**.

1.1.3 ¿Qué compone a un lenguaje?

Podemos decir que cada lenguaje (máquina o natural, no importa) consta de los siguientes elementos:

- ✚ **Alfabeto:** Un conjunto de símbolos utilizados para formar palabras de un determinado lenguaje (por ejemplo, el alfabeto latino para el inglés, el alfabeto cirílico para el ruso, el kanji para el japonés, y así sucesivamente)
- ✚ **Léxico:** También conocido como diccionario, un conjunto de palabras que el lenguaje ofrece a sus usuarios (por ejemplo, la palabra "computadora" proviene del diccionario en inglés, mientras que "cmoptrue" no; la palabra "chat" está presente en los diccionarios de inglés y francés, pero sus significados son diferentes)
- ✚ **Sintaxis:** Un conjunto de reglas (formales o informales, escritas o interpretadas intuitivamente utilizadas para precisar si una determinada cadena de palabras forma una oración válida (por ejemplo, "Soy una serpiente" es una frase sintácticamente correcta, mientras que "Yo serpiente soy una" no lo es)
- ✚ **Semántica:** Un conjunto de reglas que determinan si una frase tiene sentido (por ejemplo, "Me comí una dona" tiene sentido, pero "Una dona me comió" no lo tiene)

1.1.4 Lenguaje máquina vs. lenguaje de alto nivel

El **IL** es, de hecho, el alfabeto de un lenguaje máquina. Este es el conjunto de símbolos más simple y primario que podemos usar para dar comandos a una computadora. Es la lengua materna de la computadora.

Desafortunadamente, esta lengua materna está muy lejos de la lengua materna humana. Ambos (computadoras y humanos) necesitamos algo más, un lenguaje común para computadoras y humanos, o un puente entre los dos mundos diferentes.

Necesitamos un lenguaje en el que los humanos puedan escribir sus programas y un lenguaje que las computadoras pueden usar para ejecutar los programas, uno que es mucho más complejo que el lenguaje de máquina y, sin embargo, mucho más simple que el lenguaje natural.

Estos lenguajes a menudo se denominan lenguajes de programación de alto nivel. Son al menos algo similares a los naturales en que usan símbolos, palabras y convenciones legibles para los

humanos. Estos lenguajes permiten a los humanos expresar comandos a las computadoras que son mucho más complejos que los que ofrecen las IL.

Un programa escrito en un lenguaje de programación de alto nivel se denomina código fuente (en contraste con el código máquina ejecutado por las computadoras). Del mismo modo, el archivo que contiene el código fuente se denomina archivo fuente.

1.1.5 Compilación vs. Interpretación

La programación informática es el acto de componer los elementos del lenguaje de programación seleccionado en el orden que provocará el efecto deseado. El efecto podría ser diferente en cada caso específico – depende de la imaginación, el conocimiento y la experiencia del programador.

Por supuesto, dicha composición tiene que ser correcta en muchos sentidos:

- **alfabéticamente** – un programa debe estar escrito en un alfabeto reconocible, como romano, cirílico, etc.
- **léxicamente** – cada lenguaje de programación tiene su diccionario y hay que dominarlo; afortunadamente, es mucho más simple y pequeño que el diccionario de cualquier idioma natural;
- **sintácticamente** – cada idioma tiene sus reglas y hay que obedecerlas;
- **semánticamente** – el programa tiene que tener sentido.

Desafortunadamente, un programador también puede cometer errores con cada uno de los cuatro sentidos anteriores. Cada uno de ellos puede hacer que el programa se vuelva completamente inútil.

Supongamos que has escrito con éxito un programa. ¿Cómo persuadimos a la computadora para que lo ejecute? Tienes que convertir tu programa en lenguaje de máquina. Afortunadamente, la traducción la puede hacer una computadora, lo que hace que todo el proceso sea rápido y eficiente.

Hay dos formas diferentes de **transformar un programa de un lenguaje de programación de alto nivel a un lenguaje de máquina**:

-  **Compilación** - el programa fuente se traduce una vez (sin embargo, este acto debe repetirse cada vez que se modifique el código fuente) al obtener un archivo (por ejemplo, un .exe si el código está destinado a ejecutarse en MS Windows) que contiene el código máquina. Ahora se puede distribuir el archivo en todo el mundo; el programa que realiza esta traducción se llama **compilador** o **traductor**.
-  **Interpretación** - tú o cualquier usuario del código puede traducir el programa fuente cada vez que se debe ejecutar. El programa que realiza este tipo de transformación se denomina **intérprete**, ya que interpreta el código cada vez que se pretende ejecutar. También significa que no puedes simplemente distribuir el código fuente tal cual, porque el usuario final también necesita el intérprete para ejecutarlo.

Debido a algunas razones muy fundamentales, un lenguaje de programación de alto nivel en particular está diseñado para caer en una de estas dos categorías.

1.1.6 ¿Qué hace el intérprete?

Supongamos una vez más que has escrito un programa. Ahora, existe como un **archivo de computadora**: un programa de computadora es en realidad un fragmento de texto, por lo que el código fuente generalmente se coloca en **archivos de texto**.

Nota: tiene que ser **texto puro**, sin decoraciones como diferentes fuentes, colores, imágenes incrustadas u otros medios. Ahora debes invocar al intérprete y dejar que lea tu archivo fuente.

El intérprete lee el código fuente de la forma habitual en la cultura occidental: de arriba a abajo y de izquierda a derecha. Hay algunas excepciones: se cubrirán más adelante en el curso.

En primer lugar, el intérprete verifica si todas las líneas posteriores son correctas (utilizando los cuatro aspectos cubiertos anteriormente).

Si el compilador encuentra un error, finaliza el trabajo inmediatamente. El único resultado en este caso es un **mensaje de error**.

El intérprete te informará dónde se encuentra el error y qué lo causó. Sin embargo, estos mensajes pueden ser engañosos, ya que el intérprete no puede seguir tus intenciones exactas y puede detectar errores a cierta distancia de sus causas reales.

Por ejemplo, si intentas utilizar una entidad de un nombre desconocido, causará un error, pero el error se descubrirá en el lugar donde intenta usar la entidad, no donde se introdujo el nombre de la nueva entidad.

En otras palabras, el motivo suele estar ubicado un poco antes en el código, por ejemplo, en el lugar donde tenías que informar al intérprete que ibas a utilizar la entidad del nombre.

Si la línea se ve bien, el intérprete intenta ejecutarlo (nota: cada línea generalmente se ejecuta por separado, por lo que el trío "leer-verificar-ejecutar" se puede repetir muchas veces, más veces mas que el número real de líneas en el archivo fuente, ya que algunas partes del código puede ejecutarse más de una vez).

También es posible que una parte significativa del código pueda ejecutarse con éxito antes de que el intérprete encuentra un error. Este es un comportamiento normal en este modelo de ejecución.

Puedes preguntarte ahora: ¿cuál es mejor? ¿El modelo de "compilación" o el modelo de "interpretación"? No hay una respuesta obvia. De haberlo existido, uno de estos modelos habría dejado de existir hace mucho tiempo. Ambos tienen sus ventajas y sus desventajas.

1.1.7 Compilación vs. Interpretación – Ventajas y Desventajas

	Compilación	Interpretación
Ventajas	✓ la ejecución del código traducido suele ser más rápida; ✓ solo el usuario debe tener el compilador; el usuario final puede usar el código sin él; ✓ el código traducido se almacena usando lenguaje máquina; como es muy difícil de entender, es probable que tus propios inventos y trucos de programación sigan siendo tu secreto.	✓ puedes ejecutar el código tan pronto como lo completes; no hay fases adicionales de traducción; ✓ el código se almacena usando un lenguaje de programación, no un lenguaje máquina; esto significa que se puede ejecutar en computadoras que usan diferentes lenguajes máquina; no compila tu código por separado para cada arquitectura diferente.

Desventajas	× la compilación en sí puede ser un proceso que consume mucho tiempo; es posible que no puedas ejecutar su código inmediatamente después de realizar una modificación; × debes tener tantos compiladores como plataformas de hardware donde desees que se ejecute tu código.	× no esperes que la interpretación acelere tu código a alta velocidad: tu código compartirá el poder de la computadora con el intérprete, por lo que no puede ser realmente rápido; × tanto tu como el usuario final deben tener el intérprete para ejecutar tu código.
-------------	--	---

¿Qué significa todo esto para ti?

- Python es un **lenguaje interpretado**. Esto significa que hereda todas las ventajas y desventajas descritas. Por supuesto, agrega algunas de sus características únicas a ambos conjuntos.
- Si deseas programar en Python, necesitarás el **intérprete de Python**. No podrás ejecutar tu código sin él. Afortunadamente, **Python es gratuito**. Esta es una de sus ventajas más importantes.

Por razones históricas, los lenguajes diseñados para ser utilizados en la interpretación a menudo se denominan **lenguajes de scripting**, mientras que la fuente los programas codificados con ellos se denominan **scripts**. Bien, conozcamos a Python.

1.2 INTRODUCCIÓN A PYTHON

1.2.1 ¿Qué es Python?

Python es un lenguaje de programación de alto nivel, interpretado, orientado a objetos y de uso generalizado con semántica dinámica, que se utiliza para la programación de propósito general.

Aunque puede que conozcas a la pitón como una gran serpiente, el nombre del lenguaje de programación Python proviene de una vieja serie de comedia de la BBC llamada **Monty Python's Flying Circus**.

En el apogeo de su éxito, el equipo de Monty Python estaba realizando sus escenas en vivo para audiencias en todo el mundo, incluso en el Hollywood Bowl.

Dado que Monty Python es considerado uno de los dos nutrientes fundamentales para un programador (el otro es la pizza), el creador de Python nombró el lenguaje en honor al programa de televisión.

1.2.2 ¿Quién creó Python?

Una de las características sorprendentes de Python es el hecho de que en realidad es el trabajo de una persona. Por lo general, los grandes lenguajes de programación son desarrollados y publicados por grandes compañías que emplean a muchos profesionales, y debido a las normas de derechos de autor, es muy difícil nombrar a cualquiera de las personas involucradas en el proyecto. Python es una excepción.

No existen muchos lenguajes de programación cuyos autores sean conocidos por su nombre. Python fue creado por Guido van Rossum, nacido en 1956 en Haarlem, Países Bajos. Por supuesto, Guido van Rossum no desarrolló y evolucionó todos los componentes de Python.



Guido van Rossum

La velocidad con la que Python se ha extendido por todo el mundo es el resultado del trabajo continuo de miles de (muy a menudo anónimos) programadores, testers, usuarios (muchos de ellos no son especialistas en TI) y entusiastas, pero hay que decir que la primera idea (la semilla de la que brotó Python) llegó a una cabeza: la de Guido.

1.2.3 Un proyecto de programación por pasatiempo

Las circunstancias en las que se creó Python son un poco desconcertantes. Según Guido van Rossum:

En diciembre de 1989, estaba buscando un proyecto de programación de "pasatiempo" que me mantendría ocupado durante la semana de Navidad. Mi oficina (...) estaría cerrada, pero tenía una computadora en casa y no mucho más en mis manos. Decidí escribir un intérprete para el nuevo lenguaje de scripting en el que había estado pensando últimamente: un descendiente de ABC que atraería a los hackers de Unix/C. Elegí Python como el título de trabajo para el proyecto, estando en un estado de ánimo ligeramente irreverente (y un gran fanático de Monty Python's Flying Circus). Guido van Rossum

1.2.3.1 Los objetivos de Python

En 1999, Guido van Rossum definió sus objetivos para Python:

- Un lenguaje **fácil e intuitivo** tan poderoso como los de los principales competidores.
- De **código abierto**, para que cualquiera pueda contribuir a su desarrollo.
- El código que es tan **comprensible** como el inglés simple.
- **Adecuado para tareas cotidianas**, permitiendo tiempos de desarrollo cortos.

Unos 20 años después, está claro que todas estas intenciones se han cumplido. Algunas fuentes dicen que Python es el lenguaje de programación más popular del mundo, mientras que otros afirman que es el tercero o el quinto.

De cualquier manera, todavía ocupa un alto rango en el top ten de [PYPL Popularity of Programming Language](#) y el [TIOBE Programming Community Index](#).

Python no es una lengua joven. Es **maduro y digno de confianza**. No es una maravilla de un solo golpe. Es una estrella brillante en el firmamento de programación, y el tiempo dedicado a aprender Python es una muy buena inversión.

1.2.4 ¿Qué hace que Python sea tan especial?

¿Por qué los programadores, jóvenes y viejos, experimentados y novatos, quieren usarlo? ¿Cómo fue que las grandes empresas adoptaron Python e implementaron sus productos al usarlo?

Existen muchas razones. Ya hemos enumerado algunas de ellas, pero vamos a enumerarlas de una manera más práctica:

- es **fácil de aprender** - el tiempo necesario para aprender Python es más corto que en muchos otros lenguajes; esto significa que es posible comenzar la programación real más rápido.
- es **fácil de enseñar** - la carga de trabajo de enseñanza es menor que la que necesitan otros lenguajes; esto significa que el profesor puede poner más énfasis en las técnicas de programación generales (independientes del lenguaje), no gastando energía en trucos exóticos, extrañas excepciones y reglas incomprensibles.
- es **fácil de utilizar** para escribir software nuevo - a menudo es posible escribir código más rápido cuando se emplea Python.
- es **fácil de entender** - a menudo, también es más fácil entender el código de otra persona más rápido si está escrito en Python.
- es **fácil de obtener, instalar y desplegar** - Python es gratuito, abierto y multiplataforma; no todos los lenguajes pueden presumir de eso.

1.2.5 ¿Rivales de Python?

Python tiene dos competidores directos, con propiedades y predisposiciones comparables. Estos son:

- **Perl** - un lenguaje de scripting originalmente escrito por Larry Wall.
- **Ruby** - un lenguaje de scripting originalmente escrito por Yukihiro Matsumoto.

El primero es más tradicional, más conservador que Python, y se parece a algunos de los buenos lenguajes antiguos derivados del lenguaje de programación C clásico.

En contraste, este último es más innovador y está más lleno de ideas nuevas. Python se encuentra en algún lugar entre estas dos creaciones.

Internet está lleno de foros con discusiones infinitas sobre la superioridad de uno de estos tres sobre los otros, por si deseas obtener más información sobre cada uno de ellos.

1.2.6 ¿Dónde podemos ver a Python en acción?

Lo vemos todos los días y en casi todas partes. Se utiliza ampliamente para implementar complejos **Servicios de Internet** como motores de búsqueda, almacenamiento en la nube y herramientas, redes sociales, etc. Cuando utilizas cualquiera de estos servicios, en realidad estás muy cerca de Python.

Muchas **herramientas de desarrollo** se implementan en Python. Cada vez se escriben más **aplicaciones de uso diario** en Python. Muchos **científicos** han abandonado las costosas herramientas patentadas y se han cambiado a Python. Muchos **testers** de proyectos de TI han comenzado a usar Python para llevar a cabo procedimientos de prueba repetibles. La lista es larga.

1.2.7 ¿Por qué no Python?

A pesar de la creciente popularidad de Python, todavía existen algunos nichos en los que Python está ausente o rara vez se ve:

- **Programación de bajo nivel** (a veces llamada programación "cercana al metal"): si deseas implementar un controlador o motor gráfico extremadamente efectivo, no se usaría Python.
- **Aplicaciones para dispositivos móviles:** este territorio aún está a la espera de ser conquistado por Python, lo más probable es que suceda algún día.

1.2.8 Python 2 vs. Python 3

Existen dos tipos principales de Python, llamados Python 2 y Python 3.

Python 2 es una versión anterior del Python original. Su desarrollo se ha estancado intencionalmente, aunque eso no significa que no haya actualizaciones. Por el contrario, las actualizaciones se emiten de forma regular, pero no pretenden modificar el idioma de manera significativa. Prefieren arreglar cualquier error recién descubierto y agujeros de seguridad. La ruta de desarrollo de Python 2 ya ha llegado a un callejón sin salida, pero Python 2 en sí todavía está muy vivo.

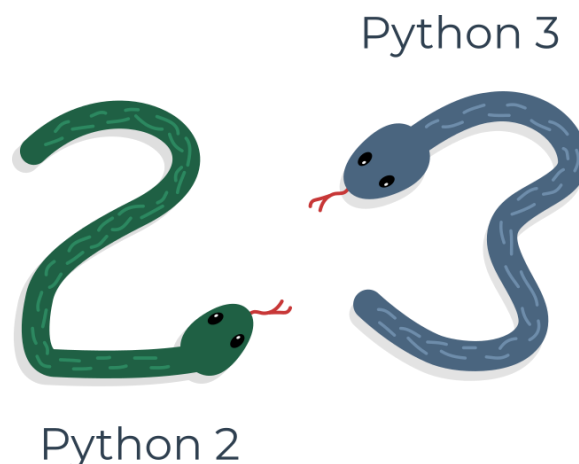
Python 3 es la versión más nueva (para ser precisos, la actual) del lenguaje. Está atravesando su propio camino de evolución, creando sus propios estándares y hábitos.

Estas dos versiones de Python no son compatibles entre sí. Las secuencias de comandos de Python 2 no se ejecutarán en un entorno de Python 3 y viceversa, por lo que si deseas que un intérprete de Python 3 ejecute el código Python 2 anterior, la única solución posible es volver a escribirlo, no desde cero, por supuesto. Grandes partes del código pueden permanecer intactas, pero tienes que revisar todo el código para encontrar todas las incompatibilidades posibles. Desafortunadamente, este proceso no puede ser completamente automatizado.

Es demasiado difícil, consume mucho tiempo, es demasiado caro y es demasiado arriesgado migrar una aplicación Python 2 antigua a una nueva plataforma. Es posible que reescribir el código le introduzca nuevos errores. Es más fácil y más sensato dejar estos sistemas solos y mejorar el intérprete existente, en lugar de intentar trabajar dentro del código fuente que ya funciona.

Python 3 no es solo una versión mejorada de Python 2, es un lenguaje completamente diferente, aunque es muy similar a su predecesor. Cuando se miran a distancia, parecen ser el mismo, pero cuando se observan de cerca, se notan muchas diferencias.

Si estás modificando una solución de Python existente, entonces es muy probable que esté codificada en Python 2. Esta es la razón por la que Python 2 todavía está en uso. Hay demasiadas aplicaciones de Python 2 existentes para descartarlo por completo.



1.2.9 Implementaciones de Python

Además de Python 2 y Python 3, hay más de una versión de cada uno.

Siguiendo la [Página wiki de Python](#), una *implementación* de Python se refiere a "un programa o entorno que brinda soporte para la ejecución de programas escritos en el lenguaje Python, representado por la Implementación de Referencia de CPython."

La implementación *tradicional* de Python, llamada **CPython**, es la versión de referencia del lenguaje informático Python de Guido van Rossum, y se suele llamar simplemente "Python". Cuando escuches el nombre *CPython*, lo más probable es que se use para distinguirlo de otras implementaciones alternativas no tradicionales.

Pero, lo primero es lo primero. Están los Pythons que son mantenidos por la gente reunida alrededor de la PSF ([Python Software Foundation](#)), una comunidad que tiene como objetivo desarrollar, mejorar, expandir y popularizar Python y su entorno. El presidente del PSF es el mismo Guido von Rossum, y por eso, estos pythons se llaman **canónicos**. También se consideran **Pythons de referencia**, ya que cualquier otra implementación del lenguaje debe seguir todos los estándares establecidos por la PSF.

Guido van Rossum usó el lenguaje de programación "C" para implementar la primera versión de su lenguaje y esta decisión aún está vigente. Todos los Pythons que provienen del PSF están escritos en el lenguaje "C". Hay muchas razones para este enfoque. Uno de ellos (probablemente el más importante) es que gracias a él, Python puede ser portado y migrado fácilmente a todas las plataformas con la capacidad de compilar y ejecutar programas en lenguaje "C" (prácticamente todas las plataformas tienen esta función, lo que abre muchas posibilidades de expansión). oportunidades para Python).

Esta es la razón por la cual la implementación de PSF a menudo se denomina **CPython**. Este es el Python más influyente entre todos los Pythons del mundo.

Algunas de las implementaciones alternativas de Python más populares:

-  **Cython**
-  **Jython**
-  **PyPy**
-  **MicroPython**

Para comenzar tu trabajo, necesitas las siguientes herramientas:

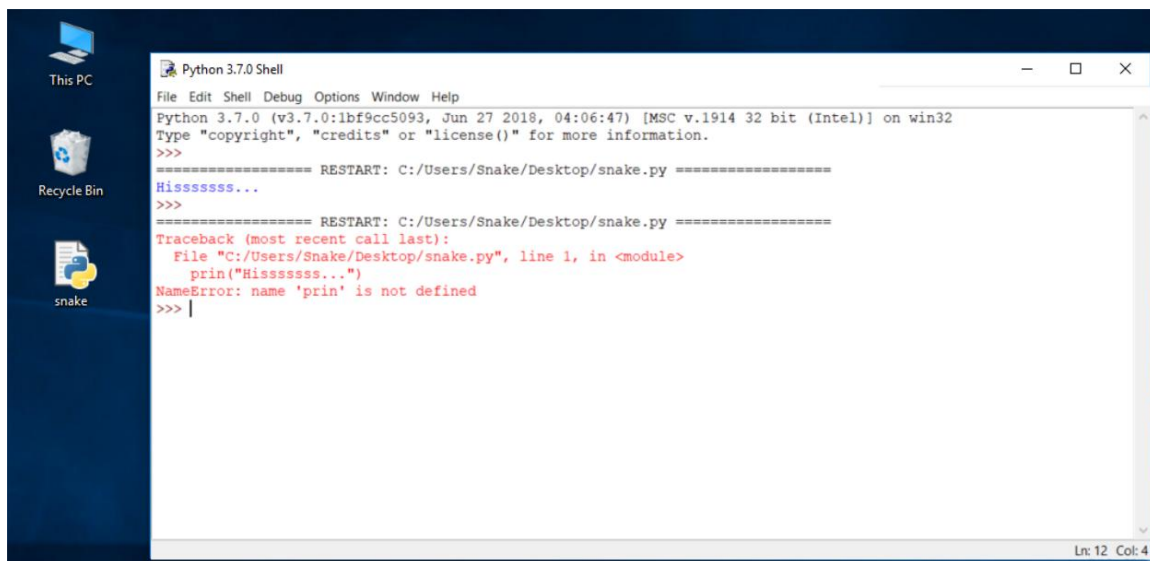
- Un editor que te ayudará a escribir el código (debe tener algunas características especiales, no disponibles en herramientas simples); este editor dedicado te dará más que el equipo estándar del sistema operativo.
- Una consola en la que puedas ejecutar tu código recién escrito y detenerlo por la fuerza cuando se sale de control.
- Una herramienta llamada depurador, capaz de ejecutar tu código paso a paso y te permite inspeccionarlo en cada momento de su ejecución.

Errores

La ventana del editor no proporciona ninguna información útil sobre el error, pero las ventanas de la consola podrían.

El mensaje (en rojo) muestra (en las líneas siguientes):

- el traceback (que es la ruta que recorre el código a través de diferentes partes del programa; puedes ignorarlo por ahora, ya que está vacío en un código tan simple);
- la ubicación del error (el nombre del archivo que contiene el error, el número de línea y el nombre del módulo); nota: el número puede ser engañoso, ya que Python suele mostrar el lugar donde notó por primera vez los efectos del error, no necesariamente el error en sí;
- el contenido de la línea errónea; nota: la ventana del editor de IDLE no muestra números de línea, pero muestra la ubicación actual del cursor en la esquina inferior derecha; utilízalo para localizar la línea errónea en un código fuente largo;
- el nombre del error y una breve explicación.



2 TIPOS DE DATOS, VARIABLES, OPERACIONES BÁSICAS DE ENTRADA Y SALIDA, OPERADORES BÁSICOS

2.1 EL PROGRAMA “HOLA MUNDO”

2.1.1 La función print()

Observa la línea de código de abajo:

```
1 | print("¡Hola, Mundo!")
2 |
```

La palabra *print* que puedes ver aquí es el **nombre de una función**. Eso no significa que dondequiera que aparezca esta palabra, será siempre el nombre de una función. El significado de la palabra proviene del contexto en el cual se haya utilizado la palabra.

Probablemente hayas encontrado el término función muchas veces antes, durante las clases de matemáticas. Probablemente también puedes recordar varios nombres de funciones matemáticas, como seno o logaritmo.

Las funciones de Python, sin embargo, son más flexibles y pueden contener más que sus parientes matemáticos.

Una función (en este contexto) es una parte separada del código de computadora el cual es capaz de:

- **causar algún efecto** (por ejemplo, enviar texto a la terminal, crear un archivo, dibujar una imagen, reproducir un sonido, etc.); esto es algo completamente inaudito en el mundo de las matemáticas.
- **evaluar un valor** (por ejemplo, la raíz cuadrada de un valor o la longitud de un texto dado) y **devolverlo como el resultado de la función**; esto es lo que hace que las funciones de Python sean parientes de los conceptos matemáticos.

Además, muchas de las funciones de Python pueden hacer las dos cosas anteriores juntas.

2.1.1.1 *¿De dónde provienen las funciones?*

- Pueden venir **de Python mismo**. La función `print` es una de este tipo; dicha función es un valor agregado de Python junto con su entorno (está **integrada**); no tienes que hacer nada especial (por ejemplo, pedirle a alguien algo) si quieres usarla.
- Pueden provenir de uno o varios de los **módulos**; de Python llamados complementos algunos de los módulos vienen con Python, otros pueden requerir una instalación por separado - cual sea el caso, todos deben estar conectados explícitamente con el código (te mostraremos cómo hacer esto pronto).
- Puedes **escribirlas tú mismo**, colocando tantas funciones como desees y necesites dentro de su programa para hacerlo más simple, claro y elegante.

El nombre de la función debe ser **significativo** (el nombre de la función `print` es evidente) imprime en la terminal.

Si vas a utilizar alguna función ya existente, no podrás modificar su nombre, pero cuando comiences a escribir tus propias funciones, debes considerar cuidadosamente la elección de nombres.

2.1.2 **Argumentos de funciones**

Como se dijo anteriormente, una función puede tener:

- Un **efecto**;
- Un **resultado**.

También existe un tercer componente de la función, muy importante - el o los **argumento(s)**.

Las funciones matemáticas usualmente toman un argumento. por ejemplo, $\sin(x)$ toma una x , que es la medida de un ángulo.

Las funciones de Python, por otro lado, son más versátiles. Dependiendo de las necesidades individuales, pueden aceptar cualquier cantidad de argumentos - tantos como sea necesario para realizar sus tareas. Nota: Cuando dijimos *cualquier número*, eso incluye el cero - algunas funciones de Python no necesitan ningún argumento.

```
print("¡Hola, Mundo!")
```


A pesar del número de argumentos necesarios o proporcionados, las funciones de Python demandan fuertemente la presencia de **un par de paréntesis** - el de apertura y de cierre, respectivamente.

Si deseas entregar uno o más argumentos a una función, colócalos **dentro de los paréntesis**. Si vas a utilizar una función que no tiene ningún argumento, aún tiene que tener los paréntesis.

Nota: para distinguir las palabras comunes de los nombres de funciones, coloca un **par de paréntesis vacíos** después de sus nombres, incluso si la función correspondiente requiere uno o más argumentos. Esta es una medida estándar.

La función de la que estamos hablando aquí es `print()`.

¿La función `print()` en nuestro ejemplo tiene algún argumento?

Por supuesto que sí, pero ¿qué son los argumentos?

2.1.2.1 Una cadena como el argumento de la función `print()`

El único argumento entregado a la función `print()` en este ejemplo es una **cadena**:

```
print("¡Hola, Mundo!")
```

Como puedes ver, la **cadena está delimitada por comillas** - de hecho, las comillas forman la cadena, recortan una parte del código y le asignan un significado diferente.

Podemos imaginar que las comillas significan algo así: el texto entre nosotros no es un código. No está diseñado para ser ejecutado, y se debe tomar tal como está.

Casi cualquier cosa que ponga dentro de las comillas se tomará de manera literal, no como código, sino como **datos**. Intenta jugar con esta cadena en particular - puedes modificarla, Ingresa contenido nuevo, o borra parte del contenido existente.

Existe más de una forma de como especificar una cadena dentro del código de Python, pero por ahora, esta será suficiente.

Hasta ahora, has aprendido acerca de dos partes importantes del código: la función y la cadena. Hemos hablado de ellos en términos de sintaxis, pero ahora es el momento de discutirlos en términos de semántica.

2.1.3 Invocación de funciones

El nombre de la función (***print*** en este caso) junto con los paréntesis y los argumento(s), forman la **invocación de la función**.

```
print("¡Hola, Mundo!")
```

Discutiremos esto con más profundidad pronto, pero arrojemos un poco de luz sobre esto ahora.

¿Qué sucede cuando Python encuentra una invocación como la que está a continuación?

```
function_name(argument)
```

Veamos:

- Primero, Python comprueba si el nombre especificado es **legal** (explora sus datos internos para encontrar una función existente del nombre; si esta búsqueda falla, Python aborta el código)
- En segundo lugar, Python comprueba si los requisitos de la función para el número de argumentos **le permiten invocar** la función de esta manera (por ejemplo, si una función específica exige exactamente dos argumentos, cualquier invocación que entregue solo un argumento se considerará errónea y abortará la ejecución del código)
- Tercero, Python **deja el código por un momento** y salta dentro de la función que se desea invocar; por lo tanto, también toma los argumento(s) y los pasa a la función;
- Cuarto, la función **ejecuta el código**, provoca el efecto deseado (si lo hubiera), evalúa el (los) resultado(s) deseado(s) y termina la tarea;
- Finalmente, Python **regresa al código** (al lugar inmediatamente después de la invocación) y reanuda su ejecución.

2.1.4 La función print() y su efecto, argumentos, y valores retornados

Hay que responder a tres preguntas importantes lo antes posible:

1. ¿Qué efecto tiene la función print()?

El efecto es muy útil y muy espectacular. La función:

- toma sus argumentos (puede aceptar más de un argumento y también puede aceptar menos de un argumento)
- los convierte a un formato legible si es necesario (como puedes sospechar, las cadenas no requieren esta acción, ya que la cadena ya es legible)
- y **envía los datos resultantes al dispositivo de salida** (normalmente la consola); en otras palabras, todo lo que pongas en la función print() se aparecerá en tu pantalla.

No es de extrañar entonces que, de ahora en adelante, utilices print() muy intensamente para ver los resultados de sus operaciones y evaluaciones.

2. ¿Qué argumentos espera print()?

Cualquiera. Pronto te mostraremos que print() puede operar con prácticamente todos los tipos de datos que ofrece Python. Cadenas, números, caracteres, valores lógicos, objetos - cualquiera de estos se puede pasar con éxito a print().

3. ¿Qué valor devuelve la función print()?

Ninguno. Su efecto es suficiente.

2.1.5 Instrucciones

Ya has visto un programa de computadora que contiene una invocación de función. La invocación de una función es uno de los muchos tipos posibles de **instrucciones** de Python.

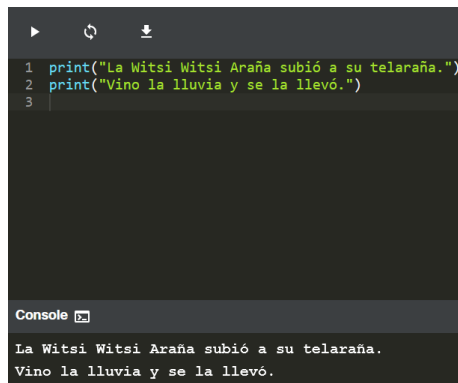
Por supuesto, cualquier programa complejo generalmente contiene muchas más instrucciones que una. La pregunta es: ¿Cómo se acopla más de una instrucción en el código de Python?

La sintaxis de Python es bastante específica en esta área. A diferencia de la mayoría de los lenguajes de programación, Python requiere que **no haya más de una instrucción por línea**.

Una línea puede estar vacía (por ejemplo, puede no contener ninguna instrucción) pero no debe contener dos, tres o más instrucciones. Esto está estrictamente prohibido.

Nota: Python hace una excepción a esta regla - permite que una instrucción se extienda por más de una línea (lo que puede ser útil cuando el código contiene construcciones complejas).

Vamos a expandir el código un poco, puedes verlo en el editor.



```
1 print("La Witsi Witsi Araña subió a su telaraña.")
2 print("Vino la lluvia y se la llevó.")
3
```

Console

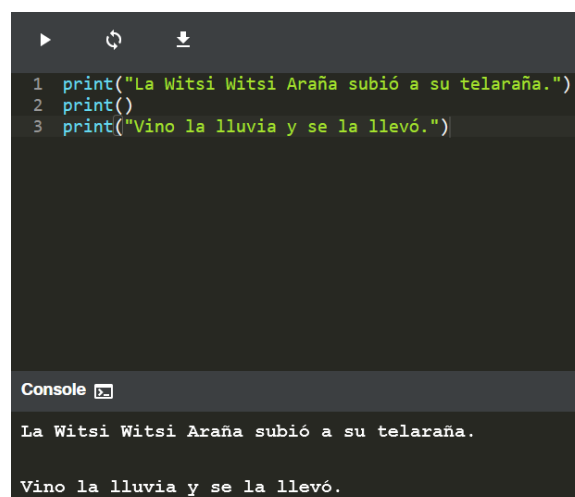
La Witsi Witsi Araña subió a su telaraña.
Vino la lluvia y se la llevó.

Esta es una buena oportunidad para hacer algunas observaciones:

- El programa **invoca a la función print() dos veces**, y puedes ver dos líneas separadas en la consola - esto significa que print() comienza su salida desde una nueva línea cada vez que comienza su ejecución; puedes cambiar este comportamiento, pero también puedes usarlo a tu favor;
- Cada invocación de print() contiene una cadena diferente, como su argumento, y el contenido de la consola lo refleja - esto significa que **las instrucciones en el código se ejecutan en el mismo orden** en el que se han colocado en el archivo fuente; no se ejecuta ninguna instrucción posterior hasta que se completa la anterior (hay algunas excepciones a esta regla, pero puede ignorarlas por ahora.)

Hemos cambiado un poco el ejemplo - hemos agregado una invocación **vacía** de la función print(). La llamamos vacía porque no hemos entregado ningún argumento a la función.

Puedes verlo en la ventana del editor.



```
1 print("La Witsi Witsi Araña subió a su telaraña.")
2 print()
3 print("Vino la lluvia y se la llevó.")
```

Console

La Witsi Witsi Araña subió a su telaraña.

Vino la lluvia y se la llevó.

Como puedes ver, la invocación vacía de `print()` no está tan vacía como podrías haber esperado - genera una línea vacía o (esta interpretación también es correcta) genera una nueva línea.

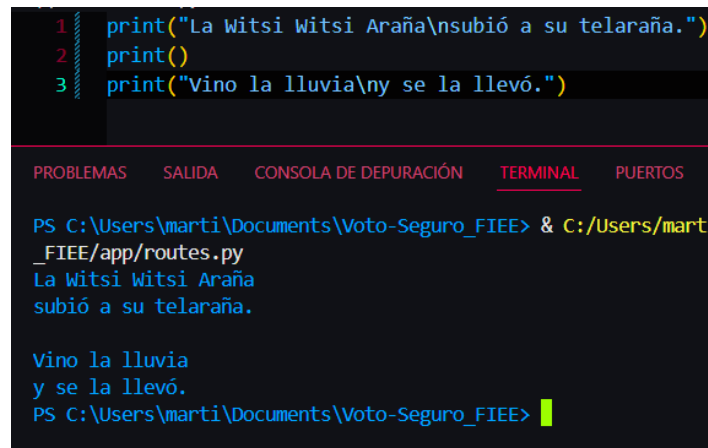
Esta no es la única forma de producir una **nueva línea** en la consola de salida. Ahora le mostraremos otra manera.

2.1.6 Caracteres de escape y nueva línea en Python

Hemos modificado el código de nuevo. Obsérvalo con cuidado.

Hay dos cambios muy sutiles - hemos insertado un par extraño de caracteres dentro del texto. Se ven así: `\n`.

```
1 print("La Witsi Witsi Araña\nsubió a su telaraña.")
2 print()
3 print("Vino la lluvia\ny se la llevó.")
```



```
PS C:\Users\marti\Documents\Voto-Seguro_FIEE> & C:/Users/marti/_FIEE/app/routes.py
La Witsi Witsi Araña
subió a su telaraña.

Vino la lluvia
y se la llevó.
PS C:\Users\marti\Documents\Voto-Seguro_FIEE>
```

Curiosamente, mientras **puedes ver dos caracteres, Python ve uno.**

La barra invertida (`\`) tiene un significado muy especial cuando se usa dentro de cadenas - se llama **carácter de escape**.

La palabra *escape* debe entenderse específicamente - significa que la serie de caracteres en la cadena se escapa por un momento (un momento muy breve) para introducir una inclusión especial.

En otras palabras, la barra invertida no significa nada en sí misma, sino que es solo una especie de anuncio de que el siguiente carácter después de la barra invertida también tiene un significado diferente.

La letra `n` colocada después de la barra invertida proviene de la palabra *newline*.

Tanto la barra invertida como `n` forman un símbolo especial llamado **un carácter de nueva línea**, que insta a la consola a iniciar una **nueva línea de salida**.

Como se puede observar, aparecen dos nuevas líneas en la canción infantil, en los lugares donde se ha utilizado `\n`.

Esta convención tiene dos consecuencias importantes:

1. Si deseas colocar solo una barra invertida dentro de una cadena, no olvides su naturaleza de escape - tienes que duplicarla. Por ejemplo, una invocación como esta provocará un error:

```
print("\n")
```

mientras que esta no lo hará:

```
print("\\n")
```

2. No todos los pares de escape (la barra invertida junto con otro carácter) significan algo.

2.1.7 Usando múltiples argumentos

Hasta ahora hemos probado el comportamiento de la función `print()` sin argumentos y con un argumento. También vale la pena intentar alimentar a la función `print()` con más de un argumento.

Mira la ventana del editor. Esto es lo que vamos a probar ahora:



```
1 print("La Witsi Witsi Araña", "subió", "a su telaraña.")
2
```

Console

La Witsi Witsi Araña subió a su telaraña.

Hay una invocación de la función `print()`, pero contiene **tres argumentos**. Todos ellos son cadenas.

Los argumentos están **separados por comas**. Los hemos rodeado de espacios para hacerlos más visibles, pero no es realmente necesario, y no lo haremos más.

En este caso, las comas al separar los argumentos juegan un papel completamente diferente al de la coma dentro de la cadena. El primero es parte de la sintaxis de Python, mientras que el segundo está diseñado para mostrarse en la consola.

Si observas el código nuevamente, verás que hay no hay espacios dentro de las cadenas.

Dos conclusiones emergen de este ejemplo:

- La función `print()` invocada con más de un argumento **los muestra todos en una sola línea**.
- La función `print()` **pone un espacio entre los argumentos de salida** por iniciativa propia.

2.1.8 Argumentos posicionales

La forma en que estamos pasando los argumentos a la función `print()` es la más común en Python, y se llama **la forma posicional**. Este nombre proviene del hecho de que el significado del argumento está dictado por su posición (por ejemplo, el segundo argumento se mostrará después del primero, no al revés).

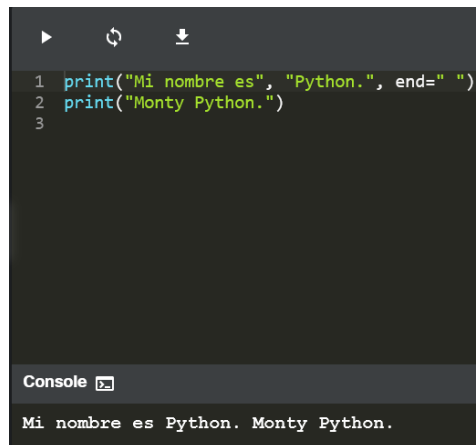
2.1.9 Argumentos de palabra clave

Python ofrece otro mecanismo para el paso de argumentos, que puede ser útil cuando deseas convencer a la función `print()` para que cambie un poco su comportamiento.

No vamos a explicarlo en profundidad ahora. Planeamos hacerlo cuando hablemos de funciones. Por ahora, simplemente queremos mostrarte cómo funciona. Siéntete libre de usarlo en tus propios programas.

El mecanismo se llama **argumentos de palabras clave**. El nombre proviene del hecho de que el significado de estos argumentos se toma no de su ubicación (posición) sino de la palabra especial (palabra clave) utilizada para identificarlos.

La función `print()` tiene dos argumentos de palabra clave que puedes usar para tus propósitos. El primero se llama `end`.



```
1 print("Mi nombre es", "Python.", end=" ")
2 print("Monty Python.")
3
```

Console

Mi nombre es Python. Monty Python.

Para usarlo, es necesario conocer algunas reglas:

- Un argumento de palabra clave consta de tres elementos: una **palabra clave** se identifica el argumento (`end` aquí); un **signo de igual** (`=`); y un **valor** asignado a ese argumento;
- cualquier argumento de palabra clave debe colocarse **después del último argumento posicional** (esto es muy importante)

En nuestro ejemplo, hemos utilizado el argumento de palabra clave `end`, y lo hemos configurado como cadena que contiene un espacio.

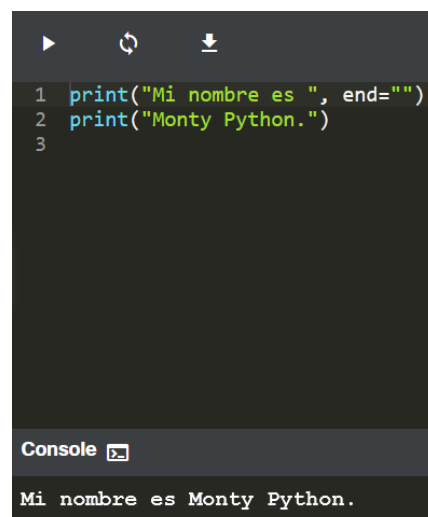
Como puedes ver, el argumento de palabra clave `end` determina los caracteres que la función `print()` envía a la salida una vez que llega al final de sus argumentos posicionales.

El comportamiento predeterminado refleja la situación en la que el argumento de palabra clave `end` se usa **implícitamente** de la siguiente manera: `end="\n"`.

Y ahora es el momento de intentar algo más difícil.

Si miras con atención, verás que hemos usado el argumento `end`, pero la cadena que se le asignó está vacía (no contiene ningún carácter).

¿Qué sucederá ahora? Ejecuta el programa en el editor para averiguarlo.



```
1 print("Mi nombre es ", end="")
2 print("Monty Python.")
3
```

Console

Mi nombre es Monty Python.

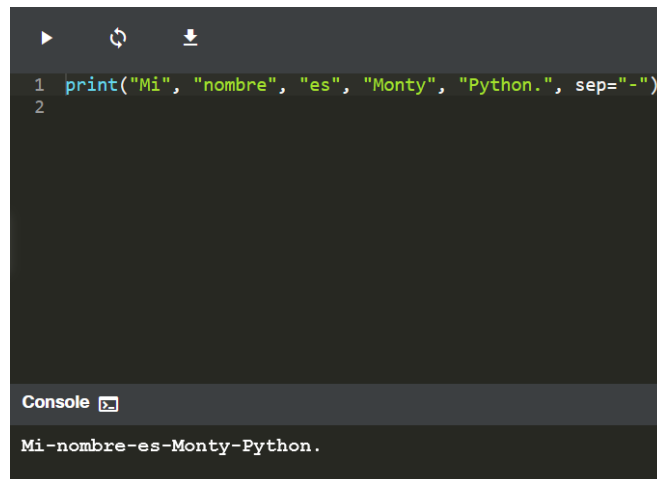
Como el argumento `end` se ha establecido a nada, la función `print()` tampoco genera nada, una vez que se han agotado sus argumentos posicionales.

Nota: **no se han enviado líneas nuevas a la salida.**

La cadena asignada al argumento de palabra clave `end` puede ser de cualquier longitud.

Dijimos anteriormente que la función `print()` separa sus argumentos de salida con espacios. Este comportamiento también se puede cambiar.

El **argumento de palabra clave** que puede hacer esto se denomina `sep` (como en *separador*).



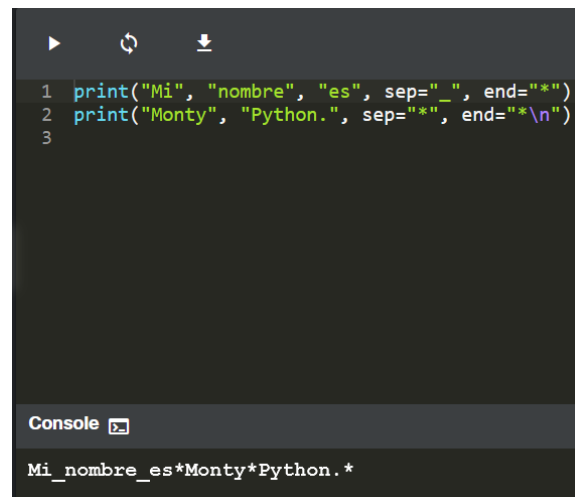
```
1 print("Mi", "nombre", "es", "Monty", "Python.", sep="-")
2
```

Console

Mi-nombre-es-Monty-Python.

La función `print()` ahora usa un guión, en lugar de un espacio, para separar los argumentos de salida.

Nota: el valor del argumento `sep` también puede ser una cadena vacía.



```
1 print("Mi", "nombre", "es", sep="_", end="*")
2 print("Monty", "Python.", sep="*", end="\n")
3
```

Console

Mi_nombre_es*Monty*Python.*

Ahora que comprendes la función `print()`, estás listo para considerar cómo almacenar y procesar datos en Python.

Sin `print()`, no podrías ver ningún resultado.

2.1.10 Resumen de Sección

1. La función `print()` es una función **integrada** imprime/envía un mensaje específico a la pantalla/ventana de consola.
2. Las funciones integradas, al contrario de las funciones definidas por el usuario, están siempre disponibles y no tienen que ser importadas. Python 3.7.1 viene con 69 funciones incorporadas. Puedes encontrar su lista completa en orden alfabético en [Python Standard Library](#).
3. Para llamar a una función (**invocación de función**), debe utilizarse el nombre de la función seguido de un paréntesis. Puedes pasar argumentos a una función colocándolos dentro de los paréntesis. Se deben separar los argumentos con una coma, por ejemplo, `print("¡Hola,", "Mundo!")`. Una función `print()` "vacía" imprime una línea vacía a la pantalla.
4. Las cadenas de Python están delimitadas por **comillas**, por ejemplo, "Soy una cadena" (comillas dobles), o 'Yo soy una cadena, también' (comillas simples).
5. Los programas de computadora son colecciones de **instrucciones**. Una instrucción es un comando para realizar una tarea específica cuando se ejecuta, por ejemplo, para imprimir un determinado mensaje en la pantalla.
6. En las cadenas de Python la **barra diagonal inversa** (`\`) es un carácter especial que anuncia que el siguiente carácter tiene un significado diferente, por ejemplo, `\n` (el **carácter de nueva línea**) comienza una nueva línea de salida.
7. Los **argumentos posicionales** son aquellos cuyo significado viene dictado por su posición, por ejemplo, el segundo argumento se emite después del primero, el tercero se emite después del segundo, etc.
8. Los **argumentos de palabra clave** son aquellos cuyo significado no está dictado por su ubicación, sino por una palabra especial (palabra clave) que se utiliza para identificarlos.
9. Los parámetros `end` y `sep` se pueden usar para dar formato la salida de la función `print()`. El parámetro `sep` especifica el separador entre los argumentos emitidos. Por ejemplo, `print("H", "E", "L", "L", "O", sep="-")`, mientras que el parámetro `end` especifica que imprimir al final de la declaración de impresión.

2.2 LITERALES DE PYTHON

2.2.1 Literales - los datos en sí mismos

Ahora que tienes un poco de conocimiento acerca de algunas de las poderosas características que ofrece la función `print()`, es tiempo de aprender sobre cuestiones nuevas, y un nuevo término - el **literal**.

Un literal se refiere a datos cuyos valores están determinados por el literal mismo.

Debido a que es un concepto un poco difícil de entender, un buen ejemplo puede ser muy útil.

Observa los siguientes dígitos:

123

¿Puedes adivinar qué valor representa? Claro que puedes - es *ciento veintitrés*.

Que tal este:

c

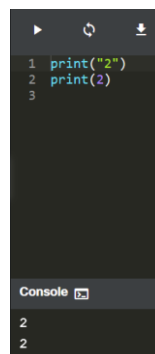
¿Representa algún valor? Tal vez. Puede ser el símbolo de la velocidad de la luz, por ejemplo. También puede representar la constante de integración. Incluso la longitud de una hipotenusa en el Teorema de Pitágoras. Existen muchas posibilidades.

No se puede elegir el valor correcto sin algo de conocimiento adicional.

Y esta es la pista: 123 es un literal, y c no lo es.

Se utilizan literales **para codificar datos y ponerlos dentro del código**. Ahora mostraremos algunas convenciones que se deben seguir al utilizar Python.

Comencemos con un sencillo experimento - observa el fragmento de código en el editor.



```
1 print("2")
2 print(2)
3
```

Console

```
2
2
```

La primera línea luce familiar. La segunda parece ser errónea debido a la falta visible de comillas.

A través de este ejemplo, encuentras dos tipos diferentes de literales:

- Una **cadena**, la cual ya conoces,
- Y un número **entero**, algo completamente nuevo.

La función `print()` los muestra exactamente de la misma manera - Sin embargo, internamente, la memoria de la computadora los almacena de dos maneras completamente diferentes - La cadena existe como eso solo una cadena - una serie de letras.

El número es convertido a una representación máquina (una serie de bits). La función `print()` es capaz de mostrar ambos en una forma legible para humanos.

Vamos a tomar algo de tiempo para discutir literales numéricas y su vida interna.

2.2.2 Enteros

Quizá ya sepas un poco acerca de cómo las computadoras hacen cálculos con números. Tal vez has escuchado del **sistema binario**, y como es que ese es el sistema que las computadoras utilizan para almacenar números y como es que pueden realizar cualquier tipo de operaciones con ellos.

No exploraremos las complejidades de los sistemas numéricos posicionales, pero se puede afirmar que todos los números manejados por las computadoras modernas son de dos tipos:

- **enteros**, es decir, aquellos que no tienen una parte fraccionaria,

- y números **punto-flotantes** (o simplemente **flotantes**), los cuales contienen (o son capaces de contener) una parte fraccionaria.

Esta definición no es tan precisa, pero es suficiente por ahora. La distinción es muy importante, y la frontera entre estos dos tipos de números es muy estricta. Ambos tipos difieren significativamente en cómo son almacenados en una computadora y en el rango de valores que aceptan.

La característica del valor numérico que determina el tipo, rango y aplicación se denomina el **tipo**.

Si se codifica un literal y se coloca dentro del código de Python, la forma del literal determina la representación (tipo) que Python utilizará para **almacenarlo en la memoria**.

Por ahora, dejemos los números flotantes a un lado (regresaremos a ellos pronto) y analicemos como es que Python reconoce un número entero.

El proceso es casi como usar lápiz y papel - es simplemente una cadena de dígitos que conforman el número, pero hay una condición - no se deben insertar caracteres que no sean dígitos dentro del número.

Tomemos, por ejemplo, el número *once millones ciento once mil ciento once*. Si tomaras ahorita un lápiz en tu mano, escribirías el siguiente número: 11,111,111, o así: 11.111.111, incluso de esta manera: 11 111 111.

Es claro que la separación hace que sea más fácil de leer, especialmente cuando el número tiene demasiados dígitos. Sin embargo, Python no acepta estas cosas, está **prohibido**. ¿Qué es lo que Python permite? El uso de **guion bajo** en los literales numéricos. *

Por lo tanto, el número se puede escribir ya sea así: 11111111, o como sigue: 11_111_111.

Nota *Python 3.6 ha introducido el guion bajo en los literales numéricos, permitiendo colocar un guion bajo entre dígitos y después de especificadores de base para mejorar la legibilidad. Esta característica no está disponible en versiones anteriores de Python.

¿Cómo se codifican los números negativos en Python? Como normalmente se hace, agregando un signo de **menos**. Se puede escribir: -11111111, o -11_111_111.

Los números positivos no requieren un signo positivo antepuesto, pero es permitido, si se desea hacer. Las siguientes líneas describen el mismo número: +11111111 y 11111111.

2.2.2.1 Números octales y hexadecimales

Existen dos convenciones adicionales en Python que no son conocidas en el mundo de las matemáticas. El primero nos permite utilizar un número en su representación **octal**.

Si un número entero esta precedido por un código 0O o 0o (cero-o), el número será tratado como un valor octal. Esto significa que el número debe contener dígitos en el rango del [0..7] únicamente.

0o123 es un número **octal** con un valor (decimal) igual a 83.

La función print() realiza la conversión automáticamente. Intenta esto:

```
print(0o123) => salida de consola = 83
```

La segunda convención nos permite utilizar números en **hexadecimal**. Dichos números deben ser precedidos por el prefijo 0x o 0X (cero-x).

0x123 es un número **hexadecimal** con un valor (decimal) igual a 291. La función `print()` puede manejar estos valores también. Intenta esto:

```
print(0x123) => 291
```

2.2.3 Flotantes

Ahora es tiempo de hablar acerca de otro tipo, el cual está designado para representar y almacenar los números que (como lo diría un matemático) tienen una **parte decimal no vacía**.

Son números que tienen (o pueden tener) una parte fraccionaria después del punto decimal, y aunque esta definición es muy pobre, es suficiente para lo que se desea discutir.

Cuando se usan términos como *dos y medio* o *menos cero punto cuatro*, pensamos en números que la computadora considera como números **punto-flotante**:

2.5

-0.4

Nota: *dos punto cinco* se ve normal cuando se escribe en un programa, sin embargo, si tu idioma nativo prefiere el uso de una coma en lugar de un punto, se debe asegurar que **el número no contenga comas**.

Python no lo aceptará, o (en casos poco probables) puede malinterpretar el número, debido a que la coma tiene su propio significado en Python.

Si se quiere utilizar solo el valor de dos punto cinco, se debe escribir como se mostró anteriormente. Nota que: hay un punto entre el 2 y el 5, no una coma.

Como puedes imaginar, el valor de **cero punto cuatro** puede ser escrito en Python como:

0.4

Pero no hay que olvidar esta sencilla regla - se puede omitir el cero cuando es el único dígito antes del punto decimal.

En esencia, el valor 0.4 se puede escribir como:

.4

Por ejemplo: el valor de 4.0 puede ser escrito como:

4.

Esto no cambiará su tipo ni su valor.

2.2.3.1 Enteros vs Flotantes

El punto decimal es esencialmente importante para reconocer números punto-flotantes en Python.

Observa estos dos números:

4

4.0

2.2.5 Cadenas

Las cadenas se emplean cuando se requiere procesar texto (como nombres de cualquier tipo, direcciones, novelas, etc.), no números.

Ya conoces un poco acerca de ellos, por ejemplo, que **las cadenas requieren comillas** así como los flotantes necesitan punto decimal.

Este es un ejemplo de una cadena: "Yo soy una cadena."

Sin embargo, hay una cuestión. Cómo se puede codificar una comilla dentro de una cadena que ya está delimitada por comillas.

Supongamos que se desea mostrar un muy sencillo mensaje:

Me gusta "Monty Python"

¿Cómo se puede hacer esto sin generar un error? Existen dos posibles soluciones.

La primera se basa en el concepto ya conocido del **carácter de escape**, el cual recordarás se utiliza empleando la **diagonal invertida**. La diagonal invertida puede también escapar de la comilla. Una comilla precedida por una diagonal invertida cambia su significado - no es un limitador, simplemente es una comilla. Lo siguiente funcionará como se desea:

```
print("Me gusta \"Monty Python\"")
```

Nota: ¿Existen dos comillas con escape en la cadena - puedes observar ambas?

La segunda solución puede ser un poco sorprendente. Python puede utilizar **una apóstrofe en lugar de una comilla**. Cualquiera de estos dos caracteres puede delimitar una cadena, pero para ello se debe ser **consistente**.

Si se delimita una cadena con una comilla, se debe cerrar con una comilla.

Si se inicia una cadena con un apóstrofe, se debe terminar con un apóstrofe.

Este ejemplo funcionará también:

```
print('Me gusta "Monty Python"')
```

Nota: en este ejemplo no se requiere nada de escapes.

Como se puede observar, la diagonal invertida es una herramienta muy poderosa - puede escapar no solo comillas, sino también apóstrofes.

Ya se ha mostrado, pero se desea hacer énfasis en este fenómeno una vez mas: **una cadena puede estar vacía** - puede no contener carácter alguno.

Una cadena vacía sigue siendo una cadena:

```
"  
""
```

2.2.6 Valores Booleanos

Para concluir con los literales de Python, existen dos más.

No son tan obvios como los anteriores y se emplean para representar un valor muy abstracto - **la veracidad**.

Cada vez que se le pregunta a Python si un número es más grande que otro, el resultado es la creación de un tipo de dato muy específico - un valor **booleano**.

El nombre proviene de George Boole (1815-1864), el autor de *Las Leyes del Pensamiento*, las cuales definen el **Álgebra Booleana** - una parte del álgebra que hace uso de dos valores: True y False, denotados como 1 y 0.

Un programador escribe un programa, y el programa hace preguntas. Python ejecuta el programa, y provee las respuestas. El programa debe ser capaz de reaccionar acorde a las respuestas recibidas.

Afortunadamente, las computadoras solo conocen dos tipos de respuestas:

- Si, esto es verdad.
- No, esto es falso.

Nunca habrá una respuesta como: *No lo sé o probablemente sí, pero no estoy seguro*.

Python, es entonces, un reptil **binario**.

Estos dos valores booleanos tienen denotaciones estrictas en Python:

True

False

No se pueden cambiar - se deben tomar estos símbolos como son, incluso respetando las **mayúsculas y minúsculas**.

2.2.7 Resumen de Sección

1. Los **literales** son notaciones para representar valores fijos en el código. Python tiene varios tipos de literales - es decir, un literal puede ser un número por ejemplo, 123, o una cadena (por ejemplo, "Yo soy un literal.").

2. El **sistema binario** es un sistema numérico que emplea 2 como su base. Por lo tanto, un número binario está compuesto por 0s y 1s únicamente, por ejemplo, 1010 es 10 en decimal.

Los sistemas de numeración Octales y Hexadecimales son similares pues emplean 8 y 16 como sus bases respectivamente. El sistema hexadecimal utiliza los números decimales más seis letras adicionales.

3. Los **enteros** (o simplemente **int**) son uno de los tipos numéricos que soporta Python. Son números que no tienen una parte fraccionaria, por ejemplo, 256, o -1 (enteros negativos).

4. Los números **punto-flotante** (o simplemente **flotantes**) son otro tipo numérico que soporta Python. Son números que contienen (o son capaces de contener) una parte fraccionaria, por ejemplo, 1.27.

5. Para codificar un apóstrofe o una comilla dentro de una cadena se puede utilizar el carácter de escape, por ejemplo, 'I\'m happy.', o abrir y cerrar la cadena utilizando un conjunto de símbolos distintos al símbolo que se desea codificar, por ejemplo, "I'm happy." para codificar un apóstrofe, y 'El dijo "Python", no "typhoon"' para codificar comillas.

6. Los **valores booleanos** son dos objetos constantes True y False empleados para representar valores de verdad (en contextos numéricos 1 es True, mientras que 0 es False).

Extra

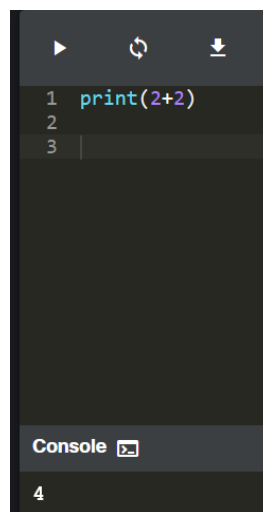
Existe un literal especial más utilizado en Python: el literal None. Este literal es llamado un objeto de NoneType, y puede ser utilizado para representar **la ausencia de un valor**. Pronto se hablará más acerca de ello.

2.3 OPERADORES - HERRAMIENTAS DE MANIPULACIÓN DE DATOS

2.3.1 Python como una calculadora

Ahora, se va a mostrar un nuevo lado de la función print(). Ya se sabe que la función es capaz de mostrar los valores de los literales que le son pasados por los argumentos.

De hecho, puede hacer algo más. Observa el siguiente fragmento de código:

A screenshot of a Python interpreter's interactive shell. The top bar contains three icons: a play button, a refresh button, and a download button. Below the icons, there are three numbered lines of code: line 1 is 'print(2+2)', line 2 is empty, and line 3 is empty. At the bottom of the window, there is a 'Console' tab with a small icon to its right. Below the console tab, the number '4' is displayed, representing the output of the previous command.

Deberías de ver el número cuatro.

Sin tomar esto con mucha seriedad, has descubierto que Python puede ser utilizado como una calculadora. No una muy útil, y definitivamente no una de bolsillo, pero una calculadora sin duda alguna.

Tomando esto más seriamente, nos estamos adentrado en el terreno de los **operadores y expresiones**.

2.3.2 Operadores Básicos

Un **operador** es un símbolo del lenguaje de programación, el cual es capaz de realizar operaciones con los valores.

Por ejemplo, como en la aritmética, el signo de + (más) es un operador el cual es capaz de **sumar** dos números, dando el resultado de la suma.

Sin embargo, no todos los operadores de Python son tan simples como el signo de más, veamos algunos de los operadores disponibles en Python, las reglas que se deben seguir para emplearlos, y como interpretar las reglas que realizan.

Se comenzará con los operadores que están asociados con las operaciones aritméticas más conocidas:

+

-

*

/

//

%

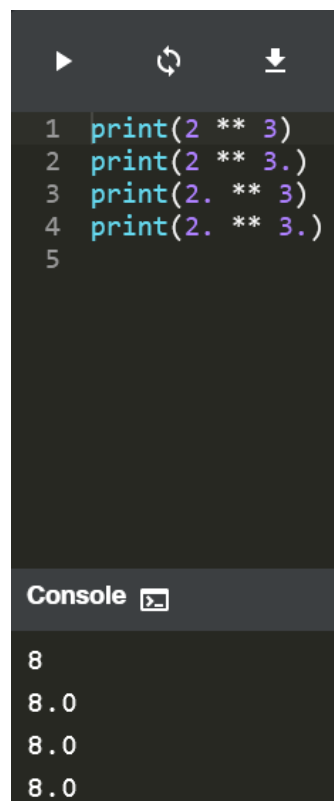
**

El orden en el que aparecen no es por casualidad. Hablaremos más de ello cuando se hayan visto todos.

Recuerda: Cuando los datos y operadores se unen, forman juntos **expresiones**. La expresión más sencilla es el literal.

2.3.2.1 Exponenciación

Observa los ejemplos en la ventana del editor:



```
1 print(2 ** 3)
2 print(2 ** 3.)
3 print(2. ** 3)
4 print(2. ** 3.)
5
```

Console

```
8
8.0
8.0
8.0
```


Nota: En los ejemplos, los dobles asteriscos están rodeados de espacios, no es obligatorio hacerlo pero hace que el código sea mas **legible**.

Los ejemplos muestran una característica importante de los **operadores numéricos** de Python.

Ejecuta el código y observa cuidadosamente los resultados que arroja. ¿Puedes observar algo?

Recuerda: Es posible formular las siguientes reglas con base en los resultados:

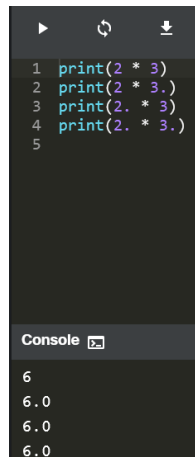
- Cuando **ambos** `**` argumentos son enteros, el resultado es entero, también;
- Cuando **al menos un** `**` argumento es flotante, el resultado también es flotante.

Esta es una distinción importante que se debe recordar.

2.3.2.2 Multiplicación

Un símbolo de `*` (asterisco) es un operador de multiplicación.

Revisa si la regla de entero vs flotante aún funciona.



```
1 print(2 * 3)
2 print(2 * 3.)
3 print(2. * 3)
4 print(2. * 3.)
5
```

Console

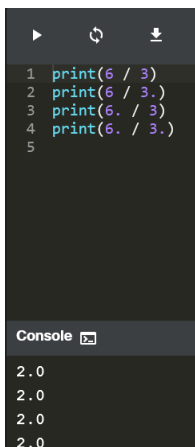
```
6
6.0
6.0
6.0
```

2.3.2.3 División

Un símbolo de `/` (diagonal) es un operador de división.

El valor después de la diagonal es el dividendo, el valor antes de la diagonal es el divisor.

Ejecuta el código y analiza los resultados.



```
1 print(6 / 3)
2 print(6 / 3.)
3 print(6. / 3)
4 print(6. / 3.)
5
```

Console

```
2.0
2.0
2.0
2.0
```

Deberías de poder observar que hay una excepción a la regla.

El resultado producido por el operador de división siempre es flotante, sin importar si a primera vista el resultado es flotante: $1 / 2$, o si parece ser completamente entero: $2 / 1$.

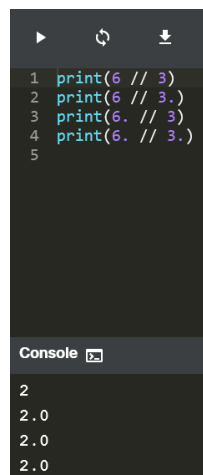
¿Esto ocasiona un problema? Sí, en ocasiones se podrá necesitar que el resultado de una división sea entero, no flotante.

Afortunadamente, Python puede ayudar con eso.

2.3.2.4 División entera

Un símbolo de `//` (doble diagonal) es un operador de **división entera**. Difiere del operador estándar `/` en dos detalles:

- El resultado carece de la parte fraccionaria, está ausente (para los enteros), o siempre es igual a cero (para los flotantes); esto significa que **los resultados siempre son redondeados**;
- Se ajusta a la regla *entero vs flotante*.



```
1 print(6 // 3)
2 print(6 // 3.)
3 print(6. // 3)
4 print(6. // 3.)
5
```

Console

```
2
2.0
2.0
2.0
```

Como se puede observar, *una división de entero entre entero* da un **resultado entero**. Todos los demás casos producen flotantes.

2.3.2.5 Residuo (módulo)

El siguiente operador es uno muy peculiar, porque no tiene un equivalente dentro de los operadores aritméticos tradicionales.

Su representación gráfica en Python es el símbolo de `%` (porcentaje), lo cual puede ser un poco confuso.

Piensa en el como una diagonal (operador de división) acompañado por dos pequeños círculos.

El resultado de la operación es el residuo que queda de la división entera.

En otras palabras, es el valor que sobra después de dividir un valor entre otro para producir un resultado entero.

Nota: el operador en ocasiones también es denominado módulo en otros lenguajes de programación.

Observa el fragmento de código:

```
1 print(14 % 4)
2
Console
2
```

Como puedes observar, el resultado es dos. Esta es la razón:

- $14 // 4$ da como resultado un 3 → esta es la parte entera, es decir el **cociente**;
- $3 * 4$ da como resultado 12 → como resultado de **la multiplicación entre el cociente y el divisor**;
- $14 - 12$ da como resultado 2 → este es el **residuo**.

2.3.2.6 Como no dividir

Como probablemente sabes, la **división entre cero no funciona**.

No intentes:

- Dividir entre cero;
- Realizar una división entera entre cero;
- Encontrar el residuo de una división entre cero.

2.3.2.7 El operador de resta, operadores unarios y binarios

El símbolo del operador de **resta** es obviamente - (el signo de menos), sin embargo debes notar que este operador tiene otra función - **puede cambiar el signo de un número**.

Esta es una gran oportunidad para mencionar una distinción muy importante entre operadores **unarios** y **binarios**.

En aplicaciones de resta, el **operador de resta espera dos argumentos**: el izquierdo (un **minuendo** en términos aritméticos) y el derecho (un **sustraendo**).

Por esta razón, el operador de resta es considerado uno de los operadores binarios, así como los demás operadores de suma, multiplicación y división.

Pero el operador negativo puede ser utilizado de una forma diferente - observa la última línea de código del siguiente fragmento:

```
1 print(-4 - 4)
2 print(4. - 8)
3 print(-1.1)
4
```

Console

```
-8
-4.0
-1.1
```

Por cierto: también hay un operador `+` unario. Aunque dicha construcción es sintácticamente correcta, utilizarla no tiene mucho sentido, y sería difícil encontrar una buena razón para hacerlo.

2.3.3 Operadores y sus prioridades

Hasta ahora, se ha tratado cada operador como si no tuviera relación con los otros. Obviamente, dicha situación tan simple e ideal es muy rara en la programación real.

También, muy seguido encontrarás más de un operador en una expresión, y entonces esta presunción ya no es tan obvia.

Considera la siguiente expresión:

`2 + 3 * 5`

Probablemente recordarás de la escuela que las multiplicaciones preceden a las sumas.

Seguramente recordarás que primero se debe multiplicar 3 por 5, mantener el 15 en tu memoria y después sumar el 2, dando como resultado el 17.

El fenómeno que causa que algunos operadores actúen antes que otros es conocido como la jerarquía de prioridades.

Python define la jerarquía de todos los operadores, y asume que los operadores de mayor jerarquía deben realizar sus operaciones antes que los de menor jerarquía.

Entonces, si se sabe que la `*` tiene una mayor prioridad que la `+`, el resultado final debe de ser obvio.

2.3.3.1 Operadores y sus enlaces

El enlace de un operador determina el orden en que se computan las operaciones de los operadores con la misma prioridad, los cuales se encuentran dentro de una misma expresión.

La mayoría de los operadores de Python tienen un enlazado hacia la izquierda, lo que significa que el cálculo de la expresión es realizado de izquierda a derecha.

Este simple ejemplo te mostrará cómo funciona. Observa:



```
1 print(9 % 6 % 2)
2
```

Console

```
1
```

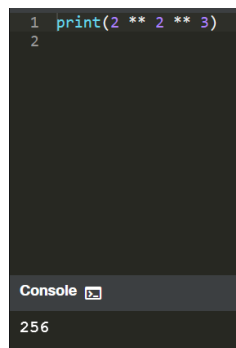
Existen dos posibles maneras de evaluar la expresión:

- De izquierda a derecha: primero $9 \% 6$ da como resultado 3, y entonces $3 \% 2$ da como resultado 1;
- De derecha a izquierda: primero $6 \% 2$ da como resultado 0, y entonces $9 \% 0$ causa un error fatal.

El resultado debe ser 1. El operador tiene un **enlazado del lado izquierdo**. Pero hay una excepción interesante.

Repite el experimento, pero ahora con exponentes.

Utiliza este fragmento de código:



```
1 print(2 ** 2 ** 3)
2
```

Console

```
256
```

Los dos posibles resultados son:

- $2 ** 2 \rightarrow 4$; $4 ** 3 \rightarrow 64$
- $2 ** 3 \rightarrow 8$; $2 ** 8 \rightarrow 256$

Ejecuta el código, ¿Qué es lo que observas?

El resultado muestra claramente que el operador de exponenciación utiliza enlazado del lado derecho.

2.3.3.2 Lista de prioridades

Como eres nuevo a los operadores de Python, no se presenta por ahora una lista completa de las prioridades de los operadores.

En lugar de ello, se mostrarán solo algunos, y se irán expandiendo conforme se vayan introduciendo operadores nuevos.

Observa la siguiente tabla:

Prioridad	Operador	
1	**	
2	+, - (Nota: Los operadores unarios a la derecha del operador exponencial enlazan con mayor fuerza.)	unario
3	*, /, //, %	
4	+, -	binario

Nota: se han enumerado los operadores en orden **de la más alta (1) a la más baja (4) prioridad**.

2.3.3.3 Operadores y paréntesis

Por supuesto, se permite hacer uso de paréntesis, lo cual cambiará el orden natural del cálculo de la operación.

De acuerdo con las reglas aritméticas, las sub-expresiones dentro de los paréntesis siempre se calculan primero.

Se pueden emplear tantos paréntesis como se necesiten, y seguido son utilizados para mejorar la legibilidad de una expresión, aun si no cambian el orden de las operaciones.

2.3.3.4 Resumen de Sección

1. Una expresión es una combinación de valores (o variables, operadores, llamadas a funciones, aprenderás de ello pronto) las cuales son evaluadas y dan como resultado un valor, por ejemplo, $1 + 2$.

2. Los operadores son símbolos especiales o palabras clave que son capaces de operar en los valores y realizar operaciones matemáticas, por ejemplo, el $*$ multiplica dos valores: $x * y$.

3. Los operadores aritméticos en Python: $+$ (suma), $-$ (resta), $*$ (multiplicación), $/$ (división clásica: regresa un flotante siempre), $\%$ (módulo: divide el operando izquierdo entre el operando derecho y regresa el residuo de la operación, por ejemplo, $5 \% 2 = 1$), $**$ (exponenciación: el operando izquierdo se eleva a la potencia del operando derecho, por ejemplo, $2 ** 3 = 2 * 2 * 2 = 8$), $//$ (división entera: retorna el número resultado de la división, pero redondeado al número entero inferior más cercano, por ejemplo, $3 // 2.0 = 1.0$)

4. Un operador unario es un operador con solo un operando, por ejemplo, -1 , o $+3$.

5. Un operador binario es un operador con dos operandos, por ejemplo, $4 + 5$, o $12 \% 5$.

6. Algunos operadores actúan antes que otros, a esto se le llama - jerarquía de prioridades:

- El operador $**$ (exponencial) tiene la prioridad más alta;
- Posteriormente los operadores unarios $+$ y $-$ (nota: los operadores unarios a la derecha del operador exponencial enlazan con mayor fuerza, por ejemplo $4 ** -1$ es igual a 0.25)
- Después $*$, $/$, $//$, y $\%$,
- Finalmente, la prioridad más baja: los operadores binarios $+$ y $-$.

7. Las sub-expresiones dentro de paréntesis siempre se calculan primero, por ejemplo:

$$15 - 1 * (5 * (1 + 2)) = 0.$$

8. Los operadores de exponenciación utilizan enlazado del lado derecho, por ejemplo:

$$2 ** 2 ** 3 = 256.$$

2.4 VARIABLES

2.4.1 Variables – cajas con forma de datos

Es justo que Python nos permita codificar literales las cuales contengan valores numéricos y cadenas.

Ya hemos visto que se pueden hacer operaciones aritméticas con estos números: sumar, restar, etc. Esto se hará una infinidad de veces en un programa.



Pero es normal preguntar cómo es que se pueden almacenar los resultados de estas operaciones, para poder emplearlos en otras operaciones, y así sucesivamente.

¿Cómo almacenar los resultados intermedios, y después utilizarlos de nuevo para producir resultados subsecuentes?

Python ayudará con ello. Python ofrece "cajas" (o "contenedores") especiales para este propósito, estas cajas son llamadas variables – el nombre mismo sugiere que el contenido de estos contenedores puede variar en casi cualquier forma.

¿Cuáles son los componentes o elementos de una variable en Python?

- Un nombre
- Un valor (el contenido del contenedor)

Comencemos con lo relacionado al nombre de la variable.

Las variables no aparecen en un programa automáticamente. Como desarrollador, tu debes decidir cuantas variables desees utilizar en tu programa.

También las debes de nombrar.

2.4.2 Nombres de Variables

Si se desea nombrar una variable, se deben seguir las siguientes reglas:

- El nombre de la variable debe de estar compuesto por MAYÚSCULAS, minúsculas, dígitos, y el carácter _ (guion bajo)
- El nombre de la variable debe comenzar con una letra;
- El carácter guion bajo es considerado una letra;
- Las mayúsculas y minúsculas se tratan de forma distinta (un poco diferente que en el mundo real - Alicia y ALICIA son el mismo nombre, pero en Python son dos nombres de variable distintos, subsecuentemente, son dos variables diferentes);
- El nombre de las variables no puede ser igual a alguna de las palabras reservadas de Python (las palabras clave - explicará más de esto pronto).

Nota que la misma restricción aplica a los nombres de funciones.

Python no impone restricciones en la longitud de los nombres de las variables, pero eso no significa que un nombre de variable largo sea mejor que uno corto.

El PEP 8 -- Style Guide for Python Code recomienda la siguiente convención de nomenclatura para variables y funciones en Python:

- Los nombres de las variables deben estar en minúsculas, con palabras separadas por guiones bajos para mejorar la legibilidad (por ejemplo, var, my_variable)
- Los nombres de las funciones siguen la misma convención que los nombres de las variables (por ejemplo, fun, my_function)
- También es posible usar letras mixtas (por ejemplo, myVariable), pero solo en contextos donde ese ya es el estilo predominante, para mantener la compatibilidad retroactiva con la convención adoptada.

2.4.2.1 Palabras Clave

Son llamadas palabras clave o (mejor dicho) palabras clave reservadas. Son reservadas porque no se deben utilizar como nombres: ni para variables, ni para funciones, ni para cualquier otra cosa que se desee crear.

El significado de la palabra reservada está predefinido, y no debe cambiar.

Afortunadamente, debido al hecho de que Python es sensible a mayúsculas y minúsculas, cualquiera de estas palabras se puede modificar cambiando una o varias letras de mayúsculas a minúsculas o viceversa, creando una nueva palabra, la cual no está reservada.

Por ejemplo - no se puede nombrar a la variable así:

import

No se puede tener una variable con ese nombre - está prohibido. pero se puede hacer lo siguiente:

Import

Estas palabras podrían parecer un misterio ahorita, pero pronto se aprenderá acerca de su significado.

2.4.3 Cómo crear una variable

¿Qué se puede poner dentro de una variable?

Cualquier cosa.

Se puede utilizar una variable para almacenar cualquier tipo de los valores que ya se han mencionado, y muchos más de los cuales aún no se han explicado.

El valor de la variable en lo que se ha puesto dentro de ella. Puede variar tanto como se necesite o requiera. El valor puede ser entero, después flotante, y eventualmente ser una cadena.

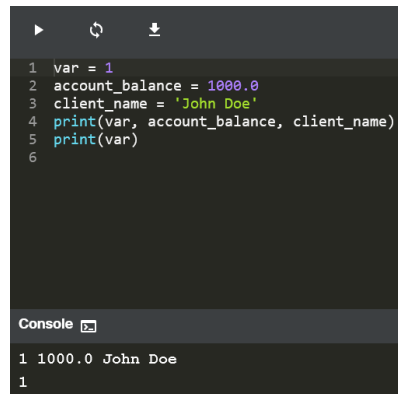
Hablemos de dos cosas importantes - como son creadas las variables, y como poner valores dentro de ellas (o, mejor dicho, como dar o pasarles valores).

RECUERDA:

Una variable se crea cuando se le asigna un valor. A diferencia de otros lenguajes de programación, no es necesario declararla.

Si se le asigna cualquier valor a una variable no existente, la variable será automáticamente creada. No se necesita hacer algo más.

La creación (o su sintaxis) es muy simple: solo utiliza el nombre de la variable deseada, después el signo de igual (=) y el valor que se desea colocar dentro de la variable.



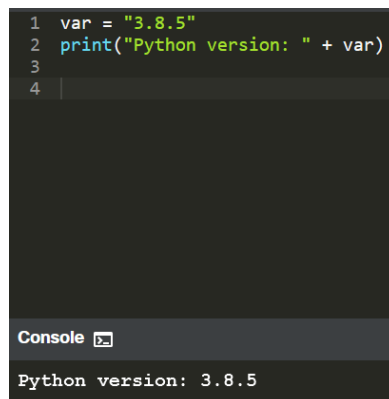
```
1 var = 1
2 account_balance = 1000.0
3 client_name = 'John Doe'
4 print(var, account_balance, client_name)
5 print(var)
6
```

Console

```
1 1000.0 John Doe
1
```

RECUERDA:

Se puede utilizar `print()` para combinar texto con variables utilizando el operador `+` para mostrar cadenas con variables, por ejemplo:



```
1 var = "3.8.5"
2 print("Python version: " + var)
3
4
```

Console

```
Python version: 3.8.5
```

2.4.4 Cómo asignar un nuevo valor a una variable ya existente

¿Cómo se le asigna un valor nuevo a una variable que ya ha sido creada? De la misma manera. Solo se necesita el signo de igual.

El signo de igual es de hecho un operador de asignación. Aunque esto suene un poco extraño, el operador tiene una sintaxis simple y una interpretación clara y precisa.

Asigna el valor del argumento de la derecha al de la izquierda, aún cuando el argumento de la derecha sea una expresión arbitraria compleja que involucre literales, operadores y variables definidas anteriormente.

Observa el siguiente código:

```
1 var = 1
2 print(var)
3 var = var + 1
4 print(var)
5
6
```

Console

```
1
2
```

La primera línea del código crea una nueva variable llamada var y le asigna el valor de 1.

La sentencia se lee de la siguiente manera: asigna el valor de 1 a una variable llamada var.

De manera más corta: asigna 1 a var.

Algunos prefieren leer el código así: var se convierte en 1.

La tercera línea le asigna a la misma variable un nuevo valor tomado de la variable misma, sumándole 1. Al ver algo así, un matemático probablemente protestaría - ningún valor puede ser igualado a si mismo más uno. Esto es una contradicción. Pero Python trata el signo = no como igual a, sino como asigna un valor.

Entonces, ¿Cómo se lee esto en un programa?

Toma el valor actual de la variable var, sumale 1 y guárdalo en la variable var.

En efecto, el valor de la variable var ha sido incrementado por uno, lo cual no está relacionado con comparar la variable con otro valor.

2.4.5 Operadores Abreviados

Es tiempo de explicar el siguiente conjunto de operadores que harán la vida del programador/desarrollador más fácil. Muy seguido, se desea utilizar la misma variable al lado derecho y al lado izquierdo del operador = operator.

Por ejemplo, si se necesita calcular una serie de valores sucesivos de la potencia de 2, se puede usar el siguiente código:

$$x = x * 2$$

También, puedes utilizar una expresión como la siguiente si no puedes dormir y estas tratando de resolverlo con alguno de los métodos tradicionales:

$$\text{sheep} = \text{sheep} + 1$$

Python ofrece una manera más corta de escribir operaciones como estas, lo cual se puede codificar de la siguiente manera:

$$x *= 2$$
$$\text{sheep} += 1$$

A continuación, se intenta presentar una descripción general para este tipo de operaciones. Si `op` es un operador de dos argumentos (esta es una condición muy importante) y el operador es utilizado en el siguiente contexto...:

`variable = variable op expresión`

...entonces se puede simplificar y mostrar de la siguiente manera:

`variable op= expresión`

2.4.6 Resumen de Sección

- Una variable es una ubicación nombrada reservada para almacenar valores en la memoria. Una variable es creada o inicializada automáticamente cuando se le asigna un valor por primera vez. (2.1.4.1)
- Cada variable debe de tener un nombre único - un identificador. Un nombre válido debe ser aquel que no contiene espacios, debe comenzar con un guion bajo(`_`), o una letra, y no puede ser una palabra reservada de Python. El primer carácter puede estar seguido de guiones bajos, letras, y dígitos. Las variables en Python son sensibles a mayúsculas y minúsculas.
- Python es un lenguaje de tipo dinámico, lo que significa que no se necesita declarar variables en él. Para asignar valores a las variables, se utiliza simplemente el operador de asignación, es decir el signo de igual (`=`), por ejemplo, `var = 1`.
- También es posible utilizar operadores de asignación compuesta (operadores abreviados) para modificar los valores asignados a las variables, por ejemplo: `var += 1`, o `var /= 5 * 2`.
- Se les puede asignar valores nuevos a variables ya existentes utilizando el operador de asignación o un operador abreviado, por ejemplo:

```
var = 2
print(var)
```

```
var = 3
print(var)
```

```
var += 1
print(var)
```

Puedes combinar texto y variables usando el operador `+` y emplear la función `print()` para generar cadenas y variables, por ejemplo:

```
var = "007"
print("Agent " + var)
```

2.5 COMENTARIOS

2.5.1 Comentarios – ¿por qué, cuándo, y dónde?

Quizá en algún momento será necesario poner algunas palabras en el código dirigidas no a Python, sino a las personas quienes estén leyendo el código con el fin de explicarles como es que funciona,

o tal vez especificar el significado de las variables, también para documentar quien es el autor del programa y en qué fecha fue escrito.

Un texto insertado en el programa el cual es, omitido en la ejecución, es denominado un comentario.

¿Cómo se colocan este tipo de comentarios en el código fuente? Tiene que ser hecho de cierta manera para que Python no intente interpretarlo como parte del código.

Cuando Python se encuentra con un comentario en el programa, el comentario es completamente transparente - desde el punto de vista de Python, el comentario es solo un espacio vacío, sin importar que tan largo sea.

En Python, un comentario es un texto que comienza con el símbolo # y se extiende hasta el final de la línea.

Si se desea colocar un comentario que abarca varias líneas, se debe colocar este símbolo en cada línea. Justo como en el siguiente código:

```
1  # Este programa evalúa la hipotenusa c.
2  # a y b son las longitudes de los catetos.
3  a = 3.0
4  b = 4.0
5  c = (a ** 2 + b ** 2) ** 0.5 # Se emplea ** en lugar de una raíz cuadrada.
6  print("c =", c)
7
```

Los desarrolladores buenos y responsables **describen cada pieza importante de código**, por ejemplo, el explicar el rol de una variable. Aunque la mejor manera de comentar una variable es dándole un nombre que no sea ambiguo.

Por ejemplo, si una variable determinada está diseñada para almacenar el área de un cuadrado, el nombre square_area será muchísimo mejor que aunt_jane.

El nombre dado a la variable se puede definir como **auto-comentable**.

2.5.2 Marcar fragmentos de código

Los comentarios pueden ser útiles en otro aspecto - se pueden utilizar para **marcar un fragmento de código que actualmente no se necesita**, cual sea la razón. Observa el siguiente ejemplo, si se **descomenta** la línea resaltada, esto afectara la salida o resultado del código:

```
1  # Este es un programa de prueba.
2  x = 1
3  y = 2
4  # y = y + x
5  print(x + y)
6
```

Esto es frecuentemente realizado cuando se está probando un programa, con el fin de aislar un fragmento de código donde posiblemente se encuentra un error.

2.5.3 Resumen de Sección

1. Los comentarios pueden ser utilizados para colocar información adicional en el código. Son omitidos al momento de la ejecución. Dicha información es para los lectores que están

manipulando el código. En Python, un comentario es un fragmento de texto que comienza con un #. El comentario se extiende hasta el final de la línea.

2. Si deseas colocar un comentario que abarque varias líneas, es necesario colocar un # al inicio de cada línea. Además, se puede utilizar un comentario para marcar un fragmento de código que no es necesario en el momento y no se desea ejecutar. (observa la última línea de código del siguiente fragmento), por ejemplo:

```
1 | # Este programa imprime
2 | # una introducción en la pantalla.
3 | print("!Hola!") #Invocando a la función print()
4 | # print("Soy Python.")
5 |
```

3. Cuando sea posible, se deben **auto comentar los nombres** de las variables, por ejemplo, si se están utilizando dos variables para almacenar la altura y longitud de algo, los nombres length y width son una mejor elección que myvar1 y myvar2.
4. Es importante utilizar los comentarios para que los programas sean más fáciles de entender, además de emplear variables legibles y significativas en el código. Sin embargo, es igualmente importante **no utilizar** nombres de variables que sean confusos, o dejar comentarios que contengan información incorrecta.
5. Los comentarios pueden ser muy útiles cuando *tú* estás leyendo tu propio código después de un tiempo (es común que los desarrolladores olviden lo que su propio código hace), y cuando *otros* están leyendo tu código (les puede ayudar a comprender que es lo que hacen tus programas y como es que lo hacen).

2.6 INTERACCIÓN CON EL USUARIO

En esta sección aprenderás a hablarle a una computadora: te familiarizarás con la función input(), realizarás conversiones de tipos de datos y aprenderás a usar operadores de cadenas.

2.6.1 La función input()

Ahora se introducirá una nueva función, la cual pareciese ser un reflejo de la función print().

¿Por qué? Bueno, print() envía datos a la consola.

Esta nueva función obtiene datos de ella.

print() no tiene un resultado utilizable. La importancia de esta nueva función es que regresa un valor muy utilizable.

La función se llama input(). El nombre de la función lo dice todo.

La función input() es capaz de leer datos que fueron introducidos por el usuario y pasar esos datos al programa en ejecución.

El programa entonces puede manipular los datos, haciendo que el código sea verdaderamente interactivo.

Todos los programas leen y procesan datos. Un programa que no obtiene datos de entrada del usuario es un programa sordo.

Observa el ejemplo:

```

1 print("Dime lo que sea...")
2 anything = input()
3 print("Hmm...", anything, "... ¿en serio?")
4

```

Se muestra un ejemplo muy sencillo de como utilizar la función `input()`.

Nota:

- El programa solicita al usuario que inserte algún dato desde la consola (seguramente utilizando el teclado, aunque también es posible introducir datos utilizando la voz o alguna imagen);
- La función `input()` es invocada sin argumentos (es la manera más sencilla de utilizar la función); la función pondrá la consola en modo de entrada; aparecerá un cursor que parpadea, y podrás introducir datos con el teclado, al terminar presiona la tecla Enter; todos los datos introducidos serán enviados al programa a través del resultado de la función;
- Nota: el resultado debe ser asignado a una variable; esto es crucial, si no se hace los datos introducidos se perderán;
- Después se utiliza la función `print()` para mostrar los datos que se obtuvieron, con algunas observaciones adicionales.

```

1 print("Dime lo que sea...")
2 anything = input()
3 print("Hmm...", anything, "... ¿en serio?")
4
5

```

Console

Dime lo que sea...

Dudu Phudu

Hmm... Dudu Phudu ... ¿en serio?

2.6.2 La función `input()` con un argumento

La función `input()` puede hacer otra cosa: puede avisar al usuario sin ninguna ayuda de `print()`.

Hemos modificado un poco nuestro ejemplo, mira el código:

```

1 anything = input("Dime lo que sea...")
2 print("Hmm...", anything, "...¿en serio?")
3
4

```

Console

Dime lo que sea...Hola

Hmm... Hola ...¿en serio?

Nota:

- la función `input()` se invoca con un argumento: es una cadena que contiene un mensaje;

- el mensaje se mostrará en la consola antes de que el usuario tenga la oportunidad de ingresar algo;
- input() entonces hará su trabajo.

Esta variante de la invocación de input() simplifica el código y lo hace más claro.

2.6.3 El resultado de la función input()

Se ha dicho antes, pero hay que decirlo sin ambigüedades una vez más: el **resultado de la función input() es una cadena**.

Una cadena que contiene todos los caracteres que el usuario introduce desde el teclado. No es un entero ni un flotante.

Esto significa que **no se debe utilizar como un argumento para operaciones matemáticas**, por ejemplo, no se pueden utilizar estos datos para elevarlos al cuadrado, para dividirlos entre algo o por algo.

2.6.4 Conversión de tipos (conversiones de tipos)

Python ofrece dos simples funciones para especificar un tipo de dato y resolver este problema, aquí están: int() y float().

Sus nombres indican cual es su función:

- La función int() toma un argumento (por ejemplo, una cadena: int(string)) e intenta convertirlo a un valor entero; si llegase a fallar, el programa entero fallará también (existe una manera de solucionar esto, se explicará mas adelante);
- La función float() toma un argumento (por ejemplo, una cadena: float(string)) e intenta convertirlo a flotante (el resto es lo mismo).

Esto es muy simple y efectivo. Sin embargo, estas funciones se pueden invocar directamente pasando el resultado de la función input() directamente. No hay necesidad de emplear variables como almacenamiento intermedio.

Se ha implementado esta idea en el editor - observa el código.

¿Puedes imaginar como la cadena introducida por el usuario fluye desde la función input() hacia la función print()?

Intenta ejecutar el código modificado. No olvides introducir un número valido.

Prueba con diferentes valores, pequeños, grandes, negativos y positivos. El cero también es un buen valor para introducir.

```

1 anything = float(input("Ingresa un número: "))
2 something = anything ** 2.0
3 print(anything, "a la potencia de 2 es", something)
4

```

Console 

```

Ingresa un número: 4
4.0 a la potencia de 2 es 16.0

```

2.6.5 Más sobre input() y conversión de tipo

El tener un equipo compuesto por input()-int()-float() abre muchas nuevas posibilidades.

Eventualmente serás capaz de escribir programas completos, los cuales acepten datos en forma de números, los cuales serán procesados y se mostrarán los resultados.

Por supuesto, estos programas serán muy primitivos y no muy utilizables, debido a que no pueden tomar decisiones, y consecuentemente no son capaces de reaccionar acorde a cada situación.

Sin embargo, esto no es un problema; se explicará como solucionarlo pronto.

El siguiente ejemplo hace referencia al programa anterior que calcula la longitud de la hipotenusa. Vamos a reescribirlo, para que pueda leer las longitudes de los catetos desde la consola.

Revisa la ventana del editor - así es como se ve ahora:

```
1 leg_a = float(input("Ingresa la longitud del primer cateto: "))
2 leg_b = float(input("Ingresa la longitud del segundo cateto: "))
3 hypo = (leg_a**2 + leg_b**2) ** .5
4 print("La longitud de la hipotenusa es:", hypo)
5
```

Console

```
Ingresa la longitud del primer cateto: 5
Ingresa la longitud del segundo cateto: 4
La longitud de la hipotenusa es: 6.4031242374328485
```

Este programa le preguntó al usuario los dos catetos, calcula la hipotenusa e imprime el resultado. Ejecútalo de nuevo e intenta introducir valores negativos.

El programa desafortunadamente no reacciona correctamente a este error. Vamos a ignorar esto por ahora. Regresaremos a ello pronto.

Toma en cuenta que en el programa que puede ver en el editor, la variable hypo se usa con un solo propósito - guardar el valor calculado entre la ejecución de la línea de código contigua.

Debido a que la función print() acepta una expresión como argumento, se puede **quitar la variable** del código.

Como se muestra en el siguiente código:

```
1 leg_a = float(input("Ingresa la longitud del primer cateto: "))
2 leg_b = float(input("Ingresa la longitud del segundo cateto: "))
3 print("La longitud de la hipotenusa es:", (leg_a**2 + leg_b**2) ** .5)
```

Console

```
Ingresa la longitud del primer cateto: 3
Ingresa la longitud del segundo cateto: 4
La longitud de la hipotenusa es: 5.0
```


2.6.6 Operadores cadena

Es tiempo de regresar a estos dos operadores aritméticos: + y *.

Ambos tienen una función secundaria. Son capaces de hacer algo más que **sumar** y **multiplicar**.

Los hemos visto en acción cuando sus argumentos son (flotantes o enteros, no hay diferencia).

Ahora veremos que son capaces también de manejar o manipular cadenas, aunque, en una manera muy específica.

El signo de + (más), al ser aplicado a dos cadenas, se convierte en **un operador de concatenación**:

string + string

Simplemente **concatena** (junta) dos cadenas en una. Por supuesto, puede ser utilizado más de una vez en una misma expresión, y en tal contexto se comporta con enlazado del lado izquierdo.

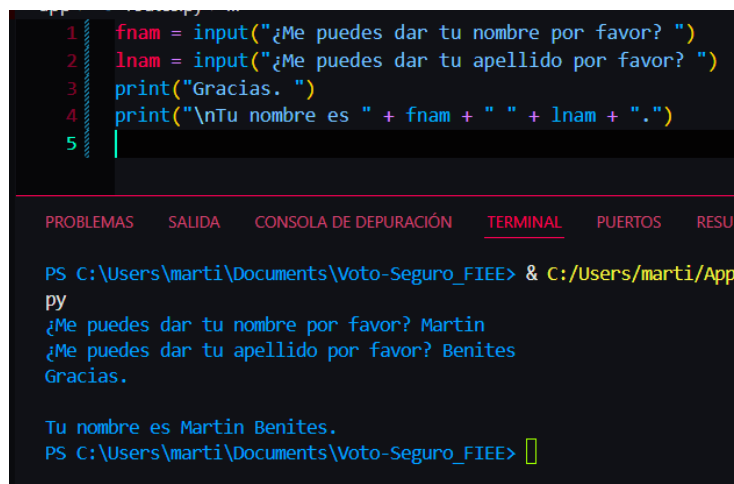
En contraste con el operador aritmético, el operador de concatenación **no es conmutativo**, por ejemplo, "ab" + "ba" no es lo mismo que "ba" + "ab".

No olvides, si se desea que el signo + sea un **concatenador**, no un sumador, solo se debe asegurar que **ambos argumentos sean cadenas**.

No se pueden mezclar los tipos de datos aquí.

Este sencillo programa muestra el signo + en su segundo uso:

```
1  fnam = input("¿Me puedes dar tu nombre por favor? ")
2  lname = input("¿Me puedes dar tu apellido por favor? ")
3  print("Gracias. ")
4  print("\nTu nombre es " + fnam + " " + lname + ".")
5  |
```



```
PS C:\Users\marti\Documents\Voto-Seguro_FIEE> & C:/Users/marti/AppData/Local/Programs/Python/Python39-64/Python.exe C:/Users/marti/AppData/Local/Programs/Python/Python39-64/Scripts/python.py
¿Me puedes dar tu nombre por favor? Martin
¿Me puedes dar tu apellido por favor? Benites
Gracias.

Tu nombre es Martin Benites.
PS C:\Users\marti\Documents\Voto-Seguro_FIEE>
```

Nota: el usar + para concatenar cadenas te permite construir la salida de una manera más precisa que con una función print() pura, incluso si está enriquecida con end= y sep= argumentos de palabras clave.

2.6.6.1 Replicación

El signo de * (asterisco), cuando es aplicado a una cadena y a un número (o a un número y cadena) se convierte en un operador de replicación:

string * number

number * string

Replica la cadena el número de veces indicado por el número.

Un número menor o igual a cero produce una **cadena vacía**.

Este sencillo programa "dibuja" un rectángulo, haciendo uso del antiguo operador (+) en un nuevo rol:

```
1 print("+ " + 10 * "-" + "+")
2 print(("|" + " " * 10 + "|\\n") * 5, end="")
3 print("+ " + 10 * "-" + "+")
4
5
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN **TERMINAL** PUERTOS

```
PS C:\Users\marti\Documents\Voto-Seguro_FIEE> & C:/Users/
py
+-----+
|       |
|       |
|       |
|       |
|       |
+-----+
PS C:\Users\marti\Documents\Voto-Seguro_FIEE>
```

Toma en cuenta la forma en que hemos usado los paréntesis en la segunda línea del código.

2.6.7 Conversiones de tipos una vez más

2.6.7.1 *str()*

A estas alturas ya sabes como emplear las funciones `int()` y `float()` para convertir una cadena a un número.

Este tipo de conversión no es en un solo sentido. También se puede convertir un número a una cadena, lo cual es más fácil y seguro - esta operación es posible hacerla siempre.

Una función capaz de hacer esto se llama `str()`:

`str(number)`

Sinceramente, puede hacer mucho más que transformar números en cadenas, eso lo veremos después.

3 VALORES BOOLEANOS, EJECUCIÓN CONDICIONAL, BUCLES, LISTAS Y SU PROCESAMIENTO, OPERACIONES LÓGICAS Y DE BIT A BIT

3.1 COMO TOMAR DECISIONES EN PYTHON

¡Bienvenido al Módulo 3! En la primera sección, aprenderemos sobre las sentencias condicionales y cómo usarlas para tomar decisiones en Python.

3.1.1 Preguntas y Respuestas

Un programador escribe un programa y el programa hace preguntas.

Una computadora ejecuta el programa y proporciona las respuestas. El programa debe ser capaz de reaccionar de acuerdo con las respuestas recibidas.

Afortunadamente, las computadoras solo conocen dos tipos de respuestas:

- Si, es cierto.

- No, esto es falso.

Nunca obtendrás una respuesta como Déjame pensar...., no lo sé, o probablemente sí, pero no lo sé con seguridad.

Para hacer preguntas, Python utiliza un conjunto de operadores muy especiales. Revisemos uno tras otro, ilustrando sus efectos en algunos ejemplos simples.

3.1.2 Comparación: operador de igualdad

Pregunta: ¿son dos valores iguales?

Para hacer esta pregunta, se utiliza el == (igual igual) operador.

No olvides esta importante distinción:

- = es un operador de asignación, por ejemplo, a = b asigna a la variable a el valor de b;
- == es una pregunta ¿Son estos valores iguales? así que a == b compara a y b.

Es un operador binario con enlazado del lado izquierdo. Necesita dos argumentos y verifica si son iguales.

3.1.3 Operadores

3.1.3.1 Igualdad: El operador igual a (==)

El operador == (igual a) compara los valores de dos operandos. Si son iguales, el resultado de la comparación es True. Si no son iguales, el resultado de la comparación es False.

Observa la comparación de igualdad a continuación - ¿Cuál es el resultado de esta operación?

```
var == 0
```

Toma en cuenta que no podemos encontrar la respuesta si no sabemos qué valor está almacenado actualmente en la variable var.

Si la variable se ha cambiado muchas veces durante la ejecución del programa, o si se ingresa su valor inicial desde la consola, Python solo puede responder a esta pregunta en el tiempo de ejecución del programa.

Ahora imagina a un programador que sufre de insomnio, y tiene que contar las ovejas negras y blancas por separado siempre y cuando haya exactamente el doble de ovejas negras que de las blancas.

La pregunta será la siguiente:

```
black_sheep == 2 * white_sheep
```

Debido a la baja prioridad del operador ==, la pregunta será tratada como la siguiente:

```
black_sheep == (2 * white_sheep)
```

Entonces, vamos a practicar la comprensión del operador ==

```
1 var = 0 # Asignando 0 a var
2 print(var == 0)
3
4 var = 1 # Asignando 1 a var
5 print(var == 0)
6
```

Console

True
False

3.1.3.2 Desigualdad: el operador no es igual a (!=)

El operador != (no es igual a) también compara los valores de dos operandos. Aquí está la diferencia: si son iguales, el resultado de la comparación es False. Si no son iguales, el resultado de la comparación es True.

3.1.3.3 Operadores de comparación: mayor que

También se puede hacer una pregunta de comparación usando el operador > (mayor que).

Si deseas saber si hay más ovejas negras que blancas, puedes escribirlo de la siguiente manera:

```
black_sheep > white_sheep # Mayor que
```

True lo confirma; False lo niega.

3.1.3.4 Operadores de comparación: mayor o igual que

El operador mayor que tiene otra variante especial, una variante no estricta, pero se denota de manera diferente que la notación aritmética clásica: >= (mayor o igual que).

Hay dos signos subsecuentes, no uno.

Ambos operadores (estrictos y no estrictos), así como los otros dos que se analizan en la siguiente sección, son operadores binarios con enlazado del lado izquierdo, y su prioridad es mayor que la mostrada por == y !=.

Si queremos saber si tenemos que usar un gorro o no, nos hacemos la siguiente pregunta:

```
centigrade_outside >= 0.0 # Mayor o igual que
```

3.1.3.5 Operadores de comparación: menor o igual que

Como probablemente ya hayas adivinado, los operadores utilizados en este caso son: El operador < (menor que) y su hermano no estricto: <= (menor o igual que).

Observa este ejemplo simple:

```
current_velocity_mph < 85 # Menor que
current_velocity_mph <= 85 # Menor o igual que
```

Vamos a comprobar si existe un riesgo de ser multados por la ley (la primera pregunta es estricta, la segunda no).

3.1.4 Haciendo uso de las respuestas

¿Qué puedes hacer con la respuesta (es decir, el resultado de una operación de comparación) que se obtiene de la computadora?

Existen al menos dos posibilidades: primero, puedes memorizarlo (almacenarlo en una variable) y utilizarlo más tarde. ¿Cómo haces eso? Bueno, utilizarías una variable arbitraria como esta:

```
answer = number_of_lions >= number_of_lionesses
```

El contenido de la variable te dirá la respuesta a la pregunta.

La segunda posibilidad es más conveniente y mucho más común: puedes utilizar la respuesta que obtengas para **tomar una decisión sobre el futuro del programa**.

Necesitas una instrucción especial para este propósito, y la discutiremos muy pronto.

Ahora necesitamos actualizar nuestra **tabla de prioridades**, y poner todos los nuevos operadores en ella. Ahora se ve como a continuación:

Prioridad	Operador	
1	**	
2	+, - (Nota: Los operadores unarios a la derecha del operador exponencial enlazan con mayor fuerza.)	unario
3	*, /, //, %	
4	+, -	binario
5	<, <=, >=, >	
6	==, !=	

3.1.5 Condiciones y ejecución condicional

Ya sabes cómo hacer preguntas a Python, pero aún no sabes cómo hacer un uso razonable de las respuestas. Se debe tener un mecanismo que le permita hacer algo si se cumple una condición, y no hacerlo si no se cumple.

Es como en la vida real: haces ciertas cosas o no cuando se cumple una condición específica, por ejemplo, sales a caminar si el clima es bueno, o te quedas en casa si está húmedo y frío.

Para tomar tales decisiones, Python ofrece una instrucción especial. Debido a su naturaleza y su aplicación, se denomina instrucción condicional (o sentencia condicional).

Existen varias variantes de la misma. Comenzaremos con la más simple, aumentando la dificultad lentamente.

La primera forma de una sentencia condicional, que puede ver a continuación, está escrita de manera muy informal pero figurada:

```
if true_or_not:
    do_this_if_true
```

Esta sentencia condicional consta de los siguientes elementos, estrictamente necesarios en este orden:

- La palabra clave reservada `if`;
- Uno o más espacios en blanco;
- Una expresión (una pregunta o una respuesta) cuyo valor se interpretará únicamente en términos de `True` (cuando su valor no sea cero) y `False` (cuando sea igual a cero);
- Unos dos puntos seguidos de una nueva línea;

- Una instrucción con sangría o un conjunto de instrucciones (se requiere absolutamente al menos una instrucción); la sangría se puede lograr de dos maneras: insertando un número particular de espacios (la recomendación es usar cuatro espacios de sangría), o usando el tabulador; nota: si hay mas de una instrucción en la parte con sangría, la sangría debe ser la misma en todas las líneas; aunque puede parecer lo mismo si se mezclan tabuladores con espacios, es importante que todas las sangrías sean exactamente iguales Python 3 no permite mezclar espacios y tabuladores para la sangría.

¿Cómo funciona esta sentencia?

- Si la expresión `true_or_not` representa la verdad (es decir, su valor no es igual a cero), las sentencias con sangría se ejecutarán;
- Si la expresión `true_or_not` no representa la verdad (es decir, su valor es igual a cero), las sentencias con sangría se omitirán (ignorado), y la siguiente instrucción ejecutada será la siguiente al nivel de la sangría original.

En la vida real, a menudo expresamos un deseo:

si el clima es bueno, saldremos a caminar

después, almorzaremos

Como puedes ver, almorzar no es una actividad condicional y no depende del clima.

Sabiendo que condiciones influyen en nuestro comportamiento y asumiendo que tenemos las funciones sin parámetros `go_for_a_walk()` y `have_lunch()`, podemos escribir el siguiente fragmento de código:

```
1  if the_weather_is_good:
2      go_for_a_walk()
3      have_lunch()
4
```

Puedes leerlo como sigue: si `sheep_counter` es mayor o igual que 120, entonces duerme y sueña (es decir, ejecuta la función `sleep_and_dream()`).

Hemos dicho que las **sentencias condicionales deben tener sangría**. Esto crea una estructura muy legible, demostrando claramente todas las rutas de ejecución posibles en el código.

Analiza el siguiente código:

```
1  if sheep_counter >= 120:
2      make_a_bed()
3      take_a_shower()
4      sleep_and_dream()
5      feed_the_sheepdogs()
6
```

Como puedes ver, tender la cama, tomar una ducha y dormir y soñar se ejecutan **condicionalmente** - cuando `sheep_counter` alcanza el límite deseado.

Alimentar a los perros, sin embargo, **siempre se hace** (es decir, la función `feed_the_sheepdogs()` no tiene sangría y no pertenece al bloque `if`, lo que significa que siempre se ejecuta).

Ahora vamos a discutir otra variante de la sentencia condicional, que también permite realizar una acción adicional cuando no se cumple la condición.

3.1.5.1 Ejecución condicional: la sentencia if-else

Comenzamos con una frase simple que decía: Si el clima es bueno, saldremos a caminar.

Nota: no hay una palabra sobre lo que sucederá si el clima es malo. Solo sabemos que no saldremos al aire libre, pero no sabemos que podríamos hacer. Es posible que también queramos planificar algo en caso de mal tiempo.

Podemos decir, por ejemplo: Si el clima es bueno, saldremos a caminar, de lo contrario, iremos al cine.

Ahora sabemos lo que haremos si se cumplen las condiciones, y sabemos lo que haremos si no todo sale como queremos. En otras palabras, tenemos un "Plan B".

Python nos permite expresar dichos planes alternativos. Esto se hace con una segunda forma, ligeramente más compleja, de la sentencia condicional, la sentencia if-else:

```
1  if true_or_false_condition:
2      perform_if_condition_true
3  else:
4      perform_if_condition_false
5
```

Por lo tanto, hay una nueva palabra: else - esta es una **palabra clave reservada**.

La parte del código que comienza con else dice que hacer si no se cumple la condición especificada por el if (observa los **dos puntos** después de la palabra).

La ejecución de *if-else* es la siguiente:

- Si la condición se evalúa como **True** (su valor no es igual a cero), la instrucción perform_if_condition_true se ejecuta, y la sentencia condicional llega a su fin;
- Si la condición se evalúa como **False** (es igual a cero), la instrucción perform_if_condition_false se ejecuta, y la sentencia condicional llega a su fin.

3.1.5.2 La sentencia if-else: más sobre ejecución condicional

Al utilizar esta forma de sentencia condicional, podemos describir nuestros planes de la siguiente manera:

```
1  if the_weather_is_good:
2      go_for_a_walk()
3  else:
4      go_to_a_theater()
5      have_lunch()
6
```

Si el clima es bueno, saldremos a caminar. De lo contrario, iremos al cine. No importa si el clima es bueno o malo, almorzaremos después (después de la caminata o después de ir al cine).

Todo lo que hemos dicho sobre la sangría funciona de la misma manera dentro de **la rama else**:

```

1  if the_weather_is_good:
2      go_for_a_walk()
3      have_fun()
4  else:
5      go_to_a_theater()
6      enjoy_the_movie()
7      have_lunch()
8

```

3.1.5.3 Sentencias if-else anidadas

Ahora, analicemos dos casos especiales de la sentencia condicional.

Primero, considera el caso donde la instrucción colocada después del if es otro if.

Lee lo que hemos planeado para este Domingo. Si hay buen clima, saldremos a caminar. Si encontramos un buen restaurante, almorzaremos allí. De lo contrario, vamos a comer un sandwich. Si hay mal clima, iremos al cine. Si no hay boletos, iremos de compras al centro comercial más cercano.

Escribamos lo mismo en Python. Considera cuidadosamente el código siguiente:

```

1  if the_weather_is_good:
2      if nice_restaurant_is_found:
3          have_lunch()
4      else:
5          eat_a_sandwich()
6  else:
7      if tickets_are_available:
8          go_to_the_theater()
9      else:
10         go_shopping()

```

Aquí hay dos puntos importantes:

- Este uso de la sentencia if se conoce como anidamiento; recuerda que cada else se refiere al if que se encuentra en el mismo nivel de sangría; se necesita saber esto para determinar cómo se relacionan los ifs y los else;
- Considera como la sangría mejora la legibilidad y hace que el código sea más fácil de entender y rastrear.

3.1.5.4 La sentencia elif

El segundo caso especial presenta otra nueva palabra clave de Python: elif. Como probablemente sospechas, es una forma más corta de else if.

elif se usa para verificar más de una condición, y para detener cuando se encuentra la primera sentencia verdadera.

Nuestro siguiente ejemplo se parece a la anidación, pero las similitudes son muy leves. Nuevamente, cambiaremos nuestros planes y los expresaremos de la siguiente manera: si hay buen clima, saldremos a caminar, de lo contrario, si obtenemos entradas, iremos al cine, de lo contrario, si hay mesas libres en el restaurante, vamos a almorzar; si todo falla, regresaremos a casa y jugaremos ajedrez.

¿Has notado cuantas veces hemos usado las palabras de lo contrario? Esta es la etapa en la que la palabra clave reservada elif desempeña su función.

Escribamos el mismo escenario empleando Python:


```

1  if the_weather_is_good:
2      go_for_a_walk()
3  elif tickets_are_available:
4      go_to_the_theater()
5  elif table_is_available:
6      go_for_lunch()
7  else:
8      play_chess_at_home()
9

```

La forma de ensamblar las siguientes sentencias if-elif-else a veces se denomina cascada.

Observa de nuevo como la sangría mejora la legibilidad del código.

Se debe prestar atención adicional a este caso:

- No debes usar else sin un if precedente;
- else siempre es la última rama de la cascada, independientemente de si has usado elif o no;
- else es una parte opcional de la cascada, y puede omitirse;
- Si hay una rama else en la cascada, solo se ejecuta una de todas las ramas;
- Si no hay una rama else, es posible que no se ejecute ninguna de las opciones disponibles.

Esto puede sonar un poco desconcertante, pero ojalá que algunos ejemplos simples ayuden a comprenderlo mejor.

Ahora vamos a mostrarte un hecho intrigante. Python tiene una característica interesante - mira el código a continuación:

```

1  # Se leen dos números
2  number1 = int(input("Ingresa el primer número: "))
3  number2 = int(input("Ingresa el segundo número: "))
4
5  # Elige el número más grande
6  if number1 > number2: larger_number = number1
7  else: larger_number = number2
8
9  # Imprime el resultado
10 print("El número más grande es:", larger_number)
11

```

Nota: si alguna de las ramas de if-elif-else contiene una sola instrucción, puedes codificarla de forma más completa (no es necesario que aparezca una línea con sangría después de la palabra clave), pero solo continúa la línea después de los dos puntos).

Sin embargo, este estilo puede ser engañoso, y no lo vamos a usar en nuestros programas futuros, pero definitivamente vale la pena saber si quieres leer y entender los programas de otra persona.

No hay otras diferencias en el código.

3.1.6 Pseudocódigo e introducción a los bucles

Ahora deberías poder escribir un programa que encuentre el mayor de cuatro, cinco, seis o incluso diez números.

Ya conoces el esquema, por lo que ampliar el tamaño del problema no será particularmente complejo.

¿Pero qué sucede si te pedimos que escribas un programa que encuentre el mayor de doscientos números? ¿Te imaginas el código?

Necesitarás doscientas variables. Si doscientas variables no son lo suficientemente complicadas, intenta imaginar la búsqueda del número más grande de un millón.

Imagina un código que contiene 199 sentencias condicionales y doscientas invocaciones de la función `input()`. Por suerte, no necesitas lidiar con eso. Hay un enfoque más simple.

Por ahora ignoraremos los requisitos de la sintaxis de Python e intentaremos analizar el problema sin pensar en la programación real. En otras palabras, intentaremos escribir el **algoritmo**, y cuando estemos contentos con él, lo implementaremos.

En este caso, utilizaremos un tipo de notación que no es un lenguaje de programación real (no se puede compilar ni ejecutar), pero está formalizado, es conciso y se puede leer. Se llama **pseudocódigo**.

Veamos nuestro pseudocódigo a continuación:

```
1 largest_number = -999999999
2 number = int(input())
3 if number == -1:
4     print(largest_number)
5     exit()
6 if number > largest_number:
7     largest_number = number
8 # Ir a la línea 02
9
```

¿Qué está pasando en él?

En primer lugar, podemos simplificar el programa si, al principio del código, le asignamos a la variable `largest_number` un valor que será más pequeño que cualquiera de los números ingresados. Usaremos `-999999999` para ese propósito.

En segundo lugar, asumimos que nuestro algoritmo no sabrá por adelantado cuántos números se entregarán al programa. Esperamos que el usuario ingrese todos los números que desee - el algoritmo funcionará bien con cien y con mil números. ¿Cómo hacemos eso?

Hacemos un trato con el usuario: cuando se ingresa el valor `-1`, será una señal de que no hay más datos y que el programa debe finalizar su trabajo.

De lo contrario, si el valor ingresado no es igual a `-1`, el programa leerá otro número, y así sucesivamente.

El truco se basa en la suposición de que cualquier parte del código se puede realizar más de una vez - precisamente, tantas veces como sea necesario.

La ejecución de una determinada parte del código más de una vez se denomina **bucle**. El significado de este término es probablemente obvio para ti.

Las líneas 02 al 08 forman un bucle. Los **pasaremos tantas veces como sea necesario** para revisar todos los valores ingresados.

¿Puedes usar una estructura similar en un programa escrito en Python? Si, si puedes.

3.1.6.1 Información Adicional

Python a menudo viene con muchas funciones integradas que harán el trabajo por ti. Por ejemplo, para encontrar el número más grande de todos, puede usar una función incorporada de Python llamada `max()`. Puedes usarlo con múltiples argumentos. Analiza el código de abajo:

```
1 # Se leen tres números.
2 number1 = int(input("Ingresa el primer número: "))
3 number2 = int(input("Ingresa el segundo número: "))
4 number3 = int(input("Ingresa el tercer número: "))
5
6 # Verifica cuál de los números es el mayor
7 # y pásalo a la variable largest_number.
8
9 largest_number = max(number1, number2, number3)
10
11 # Imprime el resultado.
12 print("El número más grande es:", largest_number)
```

Console

```
Ingresa el primer número: 1
Ingresa el segundo número: 2
Ingresa el tercer número: 3
El número más grande es: 3
```

De la misma manera, puedes usar la función `min()` para devolver el número más pequeño. Puedes reconstruir el código anterior y experimentar con él en Sandbox.

Vamos a hablar sobre estas (y muchas otras) funciones pronto. Por el momento, nuestro enfoque se centrará en la ejecución condicional y los bucles para permitirte ganar más confianza en la programación y enseñarte las habilidades que te permitirán comprender y aplicar los dos conceptos en tu código. Entonces, por ahora, no estamos tomando atajos.

3.2 BUCLES EN PYTHON

En la segunda sección, aprenderás acerca de los bucles en Python y específicamente - los bucles *while* y *for*. Aprenderás cómo crear (y evitar caer en) bucles infinitos, cómo salir de bucles, y omitir ciertas iteraciones en los bucles.

Bucles en tu código con `while`

¿Estás de acuerdo con la sentencia presentada a continuación?

mientras haya algo que hacer
hazlo

Toma en cuenta que este registro también declara que, si no hay nada que hacer, nada ocurrirá.

En general, en Python, un bucle se puede representar de la siguiente manera:

```
while
    instruction
```

Si observas algunas similitudes con la instrucción *if*, está bien. De hecho, la diferencia sintáctica es solo una: usa la palabra `while` en lugar de la palabra `if`.

La diferencia semántica es más importante: cuando se cumple la condición, *if* realiza sus sentencias **sólo una vez**; *while* repite la ejecución siempre que la condición se evalúe como `True`.

Nota: todas las reglas relacionadas con **sangría** también se aplican aquí. Te mostraremos esto pronto.

Observa el algoritmo a continuación:

```
while conditional_expression:
    instruction_one
    instruction_two
    instruction_three
    :
    :
    instruction_n
```

Ahora, es importante recordar que:

- si deseas ejecutar más de una sentencia dentro de un while, debes (como con if) poner sangría a todas las instrucciones de la misma manera.
- una instrucción o conjunto de instrucciones ejecutadas dentro del while se llama el cuerpo del bucle.
- si la condición es False (igual a cero) tan pronto como se compruebe por primera vez, el cuerpo no se ejecuta ni una sola vez (ten en cuenta la analogía de no tener que hacer nada si no hay nada que hacer).
- el cuerpo debe poder cambiar el valor de la condición, porque si la condición es True al principio, el cuerpo podría funcionar continuamente hasta el infinito. Observa que hacer una cosa generalmente disminuye la cantidad de cosas por hacer.

3.2.1 Un bucle infinito

Un bucle infinito, también denominado bucle sin fin, es una secuencia de instrucciones en un programa que se repite indefinidamente (bucle sin fin).

Este es un ejemplo de un bucle que no puede finalizar su ejecución:

```
1 | while True:
2 |     print("Estoy atrapado dentro de un bucle.")
3 |
```

Este bucle imprimirá infinitamente "Estoy atrapado dentro de un bucle." en la pantalla.

Nota:

Si deseas obtener la mejor experiencia de aprendizaje al ver cómo se comporta un bucle infinito, inicia IDLE, crea un nuevo archivo, copia y pega el código anterior, guarda tu archivo y ejecuta el programa. Lo que verás es la secuencia interminable de cadenas impresas de "Estoy atrapado dentro de un bucle." en la ventana de la consola de Python. Para finalizar tu programa, simplemente presiona *Ctrl-C* (o *Ctrl-Break* en algunas computadoras). Esto provocará la excepción `KeyboardInterrupt` y permitirá que tu programa salga del bucle. Hablaremos de ello más adelante en el curso.

3.2.2 Bucles en tu código con for

Otro tipo de bucle disponible en Python proviene de la observación de que a veces es más importante contar los "giros o vueltas" del bucle que verificar las condiciones.

Imagina que el cuerpo de un bucle debe ejecutarse exactamente cien veces. Si deseas utilizar el bucle while para hacerlo, puede tener este aspecto:

```

1 i = 0
2 while i < 100:
3     # do_something()
4     i += 1
5

```

Sería bueno si alguien pudiera hacer esta cuenta aburrida por ti. ¿Es eso posible?

Por supuesto que lo es - hay un bucle especial para este tipo de tareas, y se llama for.

En realidad, el bucle for está diseñado para realizar tareas más complicadas, **puede "explorar" grandes colecciones de datos elemento por elemento**. Te mostraremos como hacerlo pronto, pero ahora presentaremos una variante más sencilla de su aplicación.

Observa el fragmento de código:

```

1 for i in range(100):
2     # do_something()
3     pass
4

```

Existen algunos elementos nuevos. Déjanos contarte sobre ellos:

- la palabra reservada *for* abre el bucle for; nota - No hay condición después de eso; no tienes que pensar en las condiciones, ya que se verifican internamente, sin ninguna intervención.
- cualquier variable después de la palabra reservada *for* es la **variable de control** del bucle; cuenta los giros del bucle y lo hace automáticamente.
- la palabra reservada *in* introduce un elemento de sintaxis que describe el rango de valores posibles que se asignan a la variable de control.
- la función `range()` (esta es una función muy especial) es responsable de generar todos los valores deseados de la variable de control; en nuestro ejemplo, la función creará (incluso podemos decir que **alimentará** el bucle con) valores subsiguientes del siguiente conjunto: 0, 1, 2 .. 97, 98, 99; nota: en este caso, la función `range()` comienza su trabajo desde 0 y lo finaliza un paso (un número entero) antes del valor de su argumento.
- nota la palabra clave *pass* dentro del cuerpo del bucle - no hace nada en absoluto; es una **instrucción vacía** - la colocamos aquí porque la sintaxis del bucle for exige al menos una instrucción dentro del cuerpo (por cierto - if, elif, else y while expresan lo mismo).

Nota: la función `range()` **solo acepta enteros como argumentos** y genera secuencias de enteros.

3.2.3 Más acerca del bucle for y la función `range()` con tres argumentos

La función `range()` también puede aceptar tres argumentos: Echa un vistazo al código del editor.

```

1 for i in range(2, 8, 3):
2     print("El valor de i es", i)
3
4

```

Console

```

El valor de i es 2
El valor de i es 5

```

¿Sabes por qué? El primer argumento pasado a la función `range()` nos dice cual es el número **de inicio** de la secuencia (por lo tanto 2 en la output). El segundo argumento le dice a la función dónde **detener** la secuencia (la función genera números hasta el número indicado por el segundo argumento, pero no lo incluye). Finalmente, el tercer argumento indica el **paso**, que en realidad significa la diferencia entre cada número en la secuencia de números generados por la función.

2 (número inicial) → 5 (2 incremento por 3 es igual a 5 - el número está dentro del rango de 2 a 8) → 8 (5 incremento por 3 es igual a 8 - el número no está dentro del rango de 2 a 8, porque el parámetro de parada no está incluido en la secuencia de números generados por la función).

Nota: si el conjunto generado por la función `range()` está vacío, el bucle no ejecutará su cuerpo en absoluto.

Nota: el conjunto generado por `range()` debe ordenarse en **un orden ascendente**. No hay forma de forzar el `range()` para crear un conjunto en una forma diferente. Esto significa que el segundo argumento de `range()` debe ser mayor que el primero.

3.2.4 Las sentencias `break` y `continue`

Hasta ahora, hemos tratado el cuerpo del bucle como una secuencia indivisible e inseparable de instrucciones que se realizan completamente en cada giro del bucle. Sin embargo, como desarrollador, podrías enfrentar las siguientes opciones:

- parece que no es necesario continuar el bucle en su totalidad; se debe abstener de seguir ejecutando el cuerpo del bucle e ir más allá;
- parece que necesitas comenzar el siguiente giro del bucle sin completar la ejecución del turno actual.

Python proporciona dos instrucciones especiales para la implementación de estas dos tareas. Digamos por razones de precisión que su existencia en el lenguaje no es necesaria - un programador experimentado puede codificar cualquier algoritmo sin estas instrucciones. Tales adiciones, que no mejoran el poder expresivo del lenguaje, sino que solo simplifican el trabajo del desarrollador, a veces se denominan dulces sintácticos o azúcar sintáctica.

Estas dos instrucciones son:

- `break` - sale del bucle inmediatamente, e incondicionalmente termina la operación del bucle; el programa comienza a ejecutar la instrucción más cercana después del cuerpo del bucle.
- `continue` - se comporta como si el programa hubiera llegado repentinamente al final del cuerpo; el siguiente turno se inicia y la expresión de condición se prueba de inmediato.

Ambas palabras son palabras clave reservadas.

```
3 print("La instrucción break:")
4 for i in range(1, 6):
5     if i == 3:
6         break
7     print("Dentro del bucle.", i)
8     print("Fuera del bucle.")
9
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAR

PS C:\Users\marti\OneDrive\Desktop\Universidad\Certificaciones\main.py
La instrucción break:
Dentro del bucle. 1
Dentro del bucle. 2
Fuera del bucle.

```
4 print("\nla instrucción continue:")
5 for i in range(1, 6):
6     if i == 3:
7         continue
8     print("Dentro del bucle.", i)
9 print("Fuera del bucle.")
10
11
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADOS

```
PS C:\Users\marti\OneDrive\Desktop\Universidad\Certificaciones> & C:/
Universidad/Certificaciones/main.py

La instrucción continue:
Dentro del bucle. 1
Dentro del bucle. 2
Dentro del bucle. 4
Dentro del bucle. 5
Fuera del bucle.
PS C:\Users\marti\OneDrive\Desktop\Universidad\Certificaciones>
```

3.2.5 El bucle while y el bloque else

Ambos bucles while y for, tienen una característica interesante (y rara vez se usa).

Te mostraremos cómo funciona - intenta juzgar por ti mismo si es utilizable.

En otras palabras, trata de convencerte si la función es valiosa y útil, o solo es azúcar sintáctica.

Echa un vistazo al fragmento en el editor. Hay algo extraño al final - la palabra reservada else.

Como pudiste haber sospechado, los bucles también pueden tener la rama else, como los if.

La rama else del bucle siempre se ejecuta una vez, independientemente de si el bucle ha entrado o no en su cuerpo.

¿Puedes adivinar la output? Ejecuta el programa para comprobar si tenías razón.

```
1 i = 1
2 while i < 5:
3     print(i)
4     i += 1
5 else:
6     print("else:", i)
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADOS

```
PS C:\Users\marti\OneDrive\Desktop\Universidad\Certificaciones/main.py
1
2
3
4
else: 5
```

3.2.6 El bucle for y el bloque else

Los bucles for se comportan de manera un poco diferente - echa un vistazo al fragmento en el editor y ejecútalo.

```
1 for i in range(5):
2     print(i)
3 else:
4     print("else:", i)
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADOS

```
PS C:\Users\marti\OneDrive\Desktop\Universidad\Certificaciones/main.py
0
1
2
3
4
else: 4
```

La output puede ser un poco sorprendente.

La variable `i` conserva su último valor.

Nota: si la variable de control no existe antes de que comience el bucle, no existirá cuando la ejecución llegue a la rama `else branch`.

¿Cómo te sientes acerca de esta variante de `else`?

Ahora vamos a informarte sobre otros tipos de variables. Nuestras variables actuales solo pueden **almacenar un valor a la vez**, pero hay variables que pueden hacer mucho más - pueden **almacenar tantos valores como desees**. Pero primero hagamos algunos laboratorios.

3.3 OPERACIONES LÓGICAS Y DE BITS EN PYTHON

En esta sección, aprenderás sobre los operadores lógicos y de bit a bit en Python, conceptos como las tablas de verdad y el desplazamiento de bits.

3.3.1 Lógica de computadoras

¿Te has dado cuenta de que las condiciones que hemos usado hasta ahora han sido muy simples, por no decir, bastante primitivas? Las condiciones que utilizamos en la vida real son mucho más complejas. Veamos este enunciado:

Si tenemos tiempo libre, y el clima es bueno, saldremos a caminar.

Hemos utilizado la conjunción `and` (`y`), lo que significa que salir a caminar depende del cumplimiento simultáneo de estas dos condiciones. En el lenguaje de la lógica, tal conexión de condiciones se denomina conjunción. Y ahora otro ejemplo:

Si tú estás en el centro comercial o yo estoy en el centro comercial, uno de nosotros le comprará un regalo a mamá.

La aparición de la palabra `or` (`o`) significa que la compra depende de al menos una de estas condiciones. En lógica, este compuesto se llama una disyunción.

Está claro que Python debe tener operadores para construir conjunciones y disyunciones. Sin ellos, el poder expresivo del lenguaje se debilitaría sustancialmente. Se llaman operadores lógicos.

3.3.1.1 El operador `and`

Un operador de conjunción lógica en Python es la palabra `and`. Es un operador binario con una prioridad inferior a la expresada por los operadores de comparación. Nos permite codificar condiciones complejas sin el uso de paréntesis como este:

```
counter > 0 and value == 100
```

El resultado proporcionado por el operador `and` se puede determinar sobre la base de la tabla de verdad.

Si consideramos la conjunción de `A` and `B`, el conjunto de valores posibles de argumentos y los valores correspondientes de conjunción se ve de la siguiente manera:

Argumento A	Argumento B	A and B
False	False	False
False	True	False
True	False	False
True	True	True

3.3.1.2 El operador or

Un operador de disyunción es la palabra or. Es un operador binario con una prioridad más baja que and (al igual que + en comparación con *). Su tabla de verdad es la siguiente:

Argumento A	Argumento B	A or B
False	False	False
False	True	True
True	False	True
True	True	True

3.3.1.3 El operador not

Además, hay otro operador que se puede aplicar para condiciones de construcción. Es un operador unario que realiza una negación lógica. Su funcionamiento es simple: convierte la verdad en falso y lo falso en verdad.

Este operador se escribe como la palabra not, y su prioridad es muy alta: igual que el unario + y -. Su tabla de verdad es simple:

Argumento	not Argumento
False	True
True	False

$$\text{not (p and q)} == (\text{not p}) \text{ or } (\text{not q})$$

$$\text{not (p or q)} == (\text{not p}) \text{ and } (\text{not q})$$

3.3.2 Valores lógicos vs bits individuales

Los operadores lógicos toman sus argumentos como un todo, independientemente de cuantos bits contengan. Los operadores solo conocen el valor: cero (cuando todos los bits se restablecen) significa False; no cero (cuando se establece al menos un bit) significa True.

El resultado de sus operaciones es uno de estos valores: False o True. Esto significa que este fragmento de código asignará el valor True a la variable j si i no es cero; de lo contrario, será False.

$$i = 1$$

$$j = \text{not not } i$$

3.3.3 Operadores bit a bit

Sin embargo, hay cuatro operadores que le permiten manipular bits de datos individuales. Se denominan operadores bit a bit.

Cubren todas las operaciones que mencionamos anteriormente en el contexto lógico, y un operador adicional. Este es el operador xor (significa o exclusivo), y se denota como ^ (signo de intercalación).

Aquí están todos ellos:

- & (ampersand) – conjunción a nivel de bits;
- | (barra vertical) - disyunción a nivel de bits;
- ~ (tilde) - negación a nivel de bits;
- ^ (signo de intercalación) - o exclusivo a nivel de bits (xor).


```
bit = i & j
```

El operador & operará con cada par de bits correspondientes por separado, produciendo los valores de los bits relevantes del resultado. Por lo tanto, el resultado será el siguiente:

[illegible]

Estos bits corresponden al valor entero de seis.

Veamos ahora los operadores de negación. Primero el lógico:

logneg = not i

La variable logneg se establecerá en False - no es necesario hacer nada más.

La negación a nivel de bits es así:

$$\text{bitneg} = \sim i$$

Puede ser un poco sorprendente: el valor de la variable `bitneg` es -16. Esto puede parecer extraño, pero no lo es en absoluto. Si deseas obtener más información, debes consultar el sistema de números binarios y las reglas que rigen los números de complemento de dos.

[illegible]

Cada uno de estos operadores de dos argumentos se puede utilizar en **forma abreviada**. Estos son los ejemplos de sus notaciones equivalentes:

$x = x \ \& \ y$	$x \ \&= y$
$x = x \mid y$	$x \ \mid= y$
$x = x \wedge y$	$x \ \wedge= y$

3.3.4 ¿Cómo tratar con bits individuales?

Ahora te mostraremos para que puedas usar los operadores de bit a bit. Imagina que eres un desarrollador obligado a escribir una pieza importante de un sistema operativo. Se te ha dicho que puedes usar una variable asignada de la siguiente forma:

```
flag_register = 0x1234
```

La variable almacena la información sobre varios aspectos de la operación del sistema. **Cada bit de la variable almacena un valor de si/no.** También se te ha dicho que solo uno de estos bits es tuyo - el tercero (recuerda que los bits se numeran desde cero y el número de bits cero es el más bajo, mientras que el más alto es el número 31). Los bits restantes no pueden cambiar, porque están destinados a almacenar otros datos. Aquí está tu bit marcado con la letra x:

```
flag_register = 00000000000000000000000000000000x000
```

Es posible que tengas que hacer frente a las siguientes tareas:

1. **Comprobar el estado de tu bit** - deseas averiguar el valor de su bit; comparar la variable completa con cero no hará nada, porque los bits restantes pueden tener valores completamente impredecibles, pero puedes usar la siguiente propiedad de conjunción:

$$x \ \& \ 1 = x$$

$$\mathbf{x} \& 0 = 0$$

Si aplicas la operación & a la variable flag_register junto con la siguiente imagen de bits:

[illegible]

(observa el 1 en la posición de tu bit) como resultado, obtendrás una de las siguientes cadenas de bits:

- [illegible]

Dicha secuencia de ceros y unos, cuya tarea es tomar el valor o cambiar los bits seleccionados, se denomina **máscara de bits**.

Construyamos una máscara de bits para detectar el estado de tus bits. Debería apuntar a **el tercer bit**. Ese bit tiene el peso de $2^3=8$. Se podría crear una máscara adecuada mediante la siguiente sentencia:

$$\text{the_mask} = 8$$

También puedes hacer una secuencia de instrucciones dependiendo del estado de tu bit, aquí está:

```
1  if flag_register & the_mask:
2      # Mi bit se estableció en 1.
3  else:
4      # Mi bit se restableció a 0.
5
```

2. **Reinicia tu bit** - asigna un cero al bit, mientras que todos los otros bits deben permanecer sin cambios; usemos la misma propiedad de la conjunción que antes, pero usemos una máscara ligeramente diferente - exactamente como se muestra a continuación:

1111111111111111111111111110111

Toma en cuenta que la máscara se creó como resultado de la negación de todos los bits de la variable `the_mask`. Restablecer el bit es simple, y se ve así (elige el que más te guste):

```
1 flag_register = flag_register & ~the_mask
2 flag_register &= ~the_mask
3
```

3. Establece tu bit - asigna un 1 a tu bit, mientras que todos los bits restantes deben permanecer sin cambios; usa la siguiente propiedad de disyunción:

1	x		1 = 1
2	x		0 = x
3			

Ya estás listo para configurar su bit con una de las siguientes instrucciones:

```
1 flag_register = flag_register | the_mask
2 flag_register |= the_mask
3
```

4. Niega tu bit - reemplaza un 1 con un 0 y un 0 con un 1. Puedes utilizar una propiedad interesante del operador $\sim$ x:

1	$x \wedge 1 = \sim x$
2	$x \wedge 0 = x$
3	

Niega tu bit con las siguientes instrucciones:

```
1 flag_register = flag_register ^ the_mask
2 flag_register ^= the_mask
3
```

3.3.5 Desplazamiento binario a la izquierda y desplazamiento binario a la derecha

Python ofrece otra operación relacionada con los bits individuales: shifting. Esto se aplica solo a los valores de número entero, y no debe usar flotantes como argumentos para ello.

Ya aplicas esta operación muy a menudo y muy inconscientemente. ¿Cómo multiplicas cualquier número por diez? Echa un vistazo:

$$12345 \times 10 = 123450$$

Como puede ver, multiplicar por diez es de hecho un desplazamiento de todos los dígitos a la izquierda y llenar el vacío resultante con cero.

¿División entre diez? Echa un vistazo:

$$12340 \div 10 = 1234$$

Dividir entre diez no es más que desplazar los dígitos a la derecha.

La computadora realiza el mismo tipo de operación, pero con una diferencia: como dos es la base para los números binarios (no 10), desplazar un valor un bit a la izquierda corresponde a multiplicarlo por dos; respectivamente, desplazar un bit a la derecha es como dividir entre dos (observa que se pierde el bit más a la derecha).

Los operadores de cambio en Python son un par de dígrafos: $<<$ y $>>$, sugiriendo claramente en qué dirección actuará el cambio.

valor $<<$ bits

valor $>>$ bits

El argumento izquierdo de estos operadores es un valor entero cuyos bits se desplazan. El argumento correcto determina el tamaño del desplazamiento.

Esto demuestra que esta operación ciertamente no es conmutativa

La prioridad de estos operadores es muy alta. Los verás en la tabla de prioridades actualizada, que te mostraremos al final de esta sección.

Echa un vistazo a los turnos en la ventana del editor.

```

1 var = 17
2 var_right = var >> 1
3 var_left = var << 2
4 print(var, var_left, var_right)
5

```

Console

17 68 8

Nota:

- $17 \gg 1 \rightarrow 17 // 2$ (17 dividido entre 2 a la potencia de 1) $\rightarrow 8$ (desplazarse hacia la derecha en un bit equivale a la división entera entre dos)
- $17 \ll 2 \rightarrow 17 * 4$ (17 multiplicado por 2 a la potencia de 2) $\rightarrow 68$ (desplazarse hacia la izquierda dos bits es lo mismo que multiplicar números enteros por cuatro)

Y aquí está la **tabla de prioridades actualizada**, que contiene todos los operadores presentados hasta ahora:

Prioridad	Operador	Notas
1	~, +, -	unario
2	**	
3	*, /, //, %	
4	+, -	binario
5	<<, >>	
6	<, <=, >, >=	
7	==, !=	
8	&	
9		
10	=, +=, -=, *=, /=, %=, &=, ^=, =, >>=, <<=	

Resumen de sección

1. Python es compatible con los siguientes operadores lógicos:

- and $\rightarrow$ si ambos operandos son verdaderos, la condición es verdadera, por ejemplo, (True and True) es True.
- or $\rightarrow$ si alguno de los operandos es verdadero, la condición es verdadera, por ejemplo, (True or False) es True.

- `not` → devuelve `False` si el resultado es verdadero y devuelve `True` si es falso, por ejemplo, `not True` es `False`.

2. Puedes utilizar operadores bit a bit para manipular bits de datos individuales. Los siguientes datos de muestra:

- `x = 15`, el cual es `0000 1111` en binario,
- `y = 16`, el cual es `0001 0000` en binario.

Se utilizarán para ilustrar el significado de operadores bit a bit en Python. Analiza los ejemplos a continuación:

- `&` hace un bit a bit and (y), por ejemplo, `x & y = 0`, el cual es `0000 0000` en binario,
- `|` hace un bit a bit or (o), por ejemplo, `x | y = 31`, el cual es `0001 1111` en binario,
- `~` hace un bit a bit not (no), por ejemplo, `~ x = 240*`, el cual es `1111 0000` en binario,
- `^` hace un bit a bit xor, por ejemplo, `x ^ y = 31`, el cual es `0001 1111` en binario,
- `>>` hace un desplazamiento bit a bit a la derecha, por ejemplo, `y >> 1 = 8`, el cual es `0000 1000` en binario,
- `<<` hace un desplazamiento bit a bit a la izquierda, por ejemplo, `y << 3 = 128`, el cual es `1000 0000` en binario.

3.4 LISTAS

¡Bienvenido la sección 4! Aquí aprenderás sobre las listas de Python y cómo realizar varias operaciones en ellas. Aprenderás a cómo indexar, actualizar y eliminar elementos de lista, cómo crear rodajas y cómo usar algunas de las funciones y métodos de lista más importantes.

3.4.1 ¿Por qué necesitamos listas?

Puede suceder que tengas que leer, almacenar, procesar y, finalmente, imprimir docenas, quizás cientos, tal vez incluso miles de números. ¿Entonces qué? ¿Necesitas crear una variable separada para cada valor? ¿Tendrás que pasar largas horas escribiendo sentencias como la que se muestra a continuación?

```
var1 = int(input())
var2 = int(input())
var3 = int(input())
var4 = int(input())
var5 = int(input())
var6 = int(input())
:
:
```

Si no crees que esta sea una tarea complicada, toma un papel y escribe un programa que:

- lea cinco números.
- los imprima en orden desde el más pequeño hasta el más grande (Este tipo de procesamiento se denomina ordenamiento).

Debes percatarte que ni siquiera tienes suficiente papel para completar la tarea.

Hasta ahora, has aprendido como declarar variables que pueden almacenar exactamente un valor dado a la vez. Tales variables a veces se denominan escalares por analogía con las matemáticas. Todas las variables que has usado hasta ahora son realmente escalares.

Piensa en lo conveniente que fuera declarar una variable que podría almacenar más de un valor. Por ejemplo, cien, o mil o incluso diez mil. Todavía sería una y la misma variable, pero muy amplia

y espaciosa. ¿Suenan atractivos? Quizás, pero ¿cómo manejarías un contenedor así lleno de valores diferentes? ¿Cómo elegirías solo el que necesitas?

¿Y si solo pudieras numerarlos? Y luego di: dame el valor número 2; asigna el valor número 15; aumenta el número del valor 10000.

Te mostraremos como declarar tales variables de múltiples valores. Haremos esto con el ejemplo que acabamos de sugerir. Escribiremos un programa que ordene una secuencia de números. No seremos particularmente ambiciosos - asumiremos que hay exactamente cinco números.

Vamos a crear una variable llamada `numbers`; se le asigna no solo un número, sino que se llena con una lista que consta de cinco valores (nota: la lista comienza con un corchete abierto y termina con un corchete cerrado; el espacio entre los corchetes es llenado con cinco números separados por comas).

`numbers = [10, 5, 7, 2, 1]`

Digamos lo mismo utilizando una terminología adecuada: **`numbers` es una lista que consta de cinco valores, todos ellos números**. También podemos decir que esta sentencia crea una lista de longitud igual a cinco (ya que contiene cinco elementos).

Los elementos dentro de una lista **pueden tener diferentes tipos**. Algunos de ellos pueden ser enteros, otros son flotantes y otros pueden ser listas.

Python ha adoptado una convención que indica que los elementos de una lista están **siempre numerados desde cero**. Esto significa que el elemento almacenado al principio de la lista tendrá el número cero. Como hay cinco elementos en nuestra lista, al último de ellos se le asigna el número cuatro. No olvides esto.

Pronto te acostumbrarás, y se convertirá en algo natural.

Antes de continuar con nuestra discusión, debemos indicar lo siguiente: nuestra lista **es una colección de elementos, pero cada elemento es un escalar**.

¿Cómo cambias el valor de un elemento elegido en la lista?

Vamos a **asignar un nuevo valor de 111 al primer elemento** en la lista. Lo hacemos de esta manera:

```
1 numbers = [10, 5, 7, 2, 1]
2 print("Contenido de la lista:", numbers) # Imprimiendo contenido de la lista original.
3
4 numbers[0] = 111
5 print("Nuevo contenido de la lista: ", numbers) # Contenido actual de la lista.
6
7
```

Console

```
Contenido de la lista: [10, 5, 7, 2, 1]
Nuevo contenido de la lista: [111, 5, 7, 2, 1]
```

Y ahora queremos **copiar el valor del quinto elemento al segundo elemento** - ¿puedes adivinar cómo hacerlo?


```
1 numbers = [10, 5, 7, 2, 1]
2 print("Contenido de la lista original:", numbers) # Imprimiendo contenido de la lista original.
3
4 numbers[0] = 111
5 print("\nPrevio contenido de la lista:", numbers) # Imprimiendo contenido de la lista anterior.
6
7 numbers[1] = numbers[4] # Copiando el valor del quinto elemento al segundo elemento.
8 print("Nuevo contenido de la lista:", numbers) # Imprimiendo el contenido de la lista actual.
9
10
```

Console

```
Contenido de la lista original: [10, 5, 7, 2, 1]

Previo contenido de la lista: [111, 5, 7, 2, 1]
Nuevo contenido de la lista: [111, 1, 7, 2, 1]
```

El valor dentro de los corchetes que selecciona un elemento de la lista se llama un **índice**, mientras que la operación de seleccionar un elemento de la lista se conoce como **indexación**.

Vamos a utilizar la función `print()` para imprimir el contenido de la lista cada vez que realicemos los cambios. Esto nos ayudará a seguir cada paso con más cuidado y ver que sucede después de una modificación de la lista en particular.

Nota: todos los índices utilizados hasta ahora son literales. Sus valores se fijan en el tiempo de ejecución, pero **cualquier expresión también puede ser un índice**. Esto abre muchas posibilidades.

3.4.2 Acceso al contenido de las listas

Se puede acceder a cada uno de los elementos de la lista por separado. Por ejemplo, se puede imprimir:

```
1 print(numbers[0]) # Accediendo al primer elemento de la lista.
2 print("\nLongitud de la lista:", len(numbers)) # Imprimiendo la longitud de la lista.
3
```

Como probablemente hayas notado antes, Python decora la output de una manera que sugiere que todos los valores presentados forman una lista.

3.4.2.1 La función `len()`

La longitud de una lista puede variar durante la ejecución. Se pueden agregar nuevos elementos a la lista, mientras que otros pueden eliminarse de ella. Esto significa que la lista es una entidad muy dinámica.

Si deseas verificar la longitud actual de la lista, puedes usar una función llamada `len()` (su nombre proviene de `length` - longitud).

La función toma el nombre de la lista como un argumento y devuelve el número de elementos almacenados actualmente dentro de la lista (en otras palabras - la longitud de la lista).

3.4.3 Eliminando elementos de una lista

Cualquier elemento de la lista puede ser eliminado en cualquier momento - esto se hace con una instrucción llamada `del` (eliminar). Nota: es una instrucción, no una función.

Tienes que apuntar al elemento que quieres eliminar - desaparecerá de la lista y la longitud de la lista se reducirá en uno.

Mira el fragmento de abajo. ¿Puedes adivinar qué output producirá? Ejecuta el programa en el editor y comprueba.

```
main.py > ...
1 numbers = [10, 5, 7, 2, 1]
2 print("Contenido de la lista original:", numbers) # Imprimiendo el contenido de la lista original.
3
4 numbers[0] = 111
5 print("\nContenido de la lista con cambio:", numbers) # Imprimiendo contenido de la lista con 111.
6
7 numbers[1] = numbers[4] # Copiando el valor del quinto elemento al segundo elemento.
8 print("Contenido de la lista con intercambio:", numbers) # Imprimiendo contenido de la lista con intercambio.
9
10 print("\nLongitud de la lista:", len(numbers)) # Imprimiendo la longitud de la lista.
11
12 ###
13
14 del numbers[1] # Eliminando el segundo elemento de la lista.
15 print("Longitud de la nueva lista:", len(numbers)) # Imprimiendo nueva longitud de la lista.
16 print("\nNuevo contenido de la lista:", numbers) # Imprimiendo el contenido de la lista actual.
17
18 ###
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN **TERMINAL** PUERTOS RESULTADOS DE LA CONSULTA

```
PS C:\Users\marti\OneDrive\Desktop\Universidad\Certificaciones> & C:/Users/marti/AppData/Local/Programs/Python/Python310/python
Drive/Desktop/Universidad/Certificaciones/main.py
Contenido de la lista original: [10, 5, 7, 2, 1]

Contenido de la lista con cambio: [111, 5, 7, 2, 1]
Contenido de la lista con intercambio: [111, 1, 7, 2, 1]

Longitud de la lista: 5
Longitud de la nueva lista: 4

Nuevo contenido de la lista: [111, 7, 2, 1]
PS C:\Users\marti\OneDrive\Desktop\Universidad\Certificaciones>
```

Nota: hemos eliminado uno de los elementos de la lista - ahora solo hay cuatro elementos en la lista. Esto significa que el elemento número cuatro no existe.

3.4.4 Los índices negativos son legales

Puede parecer extraño, pero los índices negativos son válidos, y pueden ser muy útiles.

Un elemento con un índice igual a -1 es el último en la lista.

```
1 numbers = [111, 7, 2, 1]
2 print(numbers[-1])
3
```

Console

```
1
```

El código del ejemplo mostrará 1.

3.4.5 Funciones vs métodos

Un método es un tipo específico de función - se comporta como una función y se parece a una función, pero difiere en la forma en que actúa y en su estilo de invocación.

Una función no pertenece a ningún dato - obtiene datos, puede crear nuevos datos y (generalmente) produce un resultado.

Un método hace todas estas cosas, pero también puede cambiar el estado de una entidad seleccionada.

Un método es propiedad de los datos para los que trabaja, mientras que una función es propiedad de todo el código.

Esto también significa que invocar un método requiere alguna especificación de los datos a partir de los cuales se invoca el método.

Puede parecer desconcertante aquí, pero lo trataremos en profundidad cuando profundicemos en la programación orientada a objetos.

En general, una invocación de función típica puede tener este aspecto:

```
result = function(arg)
```

La función toma un argumento, hace algo, y devuelve un resultado.

Una invocación de un método típico usualmente se ve así:

```
result = data.method(arg)
```

Nota: el nombre del método está precedido por el nombre de los datos que posee el método. A continuación, se agrega un **punto**, seguido del **nombre del método** y un par de **paréntesis que encierran los argumentos**.

El método se comportará como una función, pero puede hacer algo más - puede **cambiar el estado interno de los datos** a partir de los cuales se ha invocado.

Puedes preguntar: ¿por qué estamos hablando de métodos, y no de listas?

Este es un tema esencial en este momento, ya que le mostraremos como agregar nuevos elementos a una lista existente. Esto se puede hacer con métodos propios de las listas, no por funciones.

3.4.6 Agregando elementos a una lista: `append()` y `insert()`

Un nuevo elemento puede ser *añadido* al final de la lista existente:

```
list.append(value)
```

Dicha operación se realiza mediante un método llamado `append()`. Toma el valor de su argumento y lo coloca **al final de la lista** que posee el método.

La longitud de la lista aumenta en uno.

El método `insert()` es un poco más inteligente - puede agregar un nuevo elemento **en cualquier lugar de la lista**, no solo al final.

```
list.insert(location, value)
```

Toma dos argumentos:

- el primero muestra la ubicación requerida del elemento a insertar; nota: todos los elementos existentes que ocupan ubicaciones a la derecha del nuevo elemento (incluido el

que está en la posición indicada) se desplazan a la derecha, para hacer espacio para el nuevo elemento;

- el segundo es el elemento para insertar.

Puedes **iniciar la vida de una lista creándola vacía** (esto se hace con un par de corchetes vacíos) y luego agregar nuevos elementos según sea necesario.

3.4.7 Listas en acción

Dejemos de lado las listas por un breve momento y veamos un tema intrigante.

Imagina que necesitas reorganizar los elementos de una lista, es decir, revertir el orden de los elementos: el primero y el quinto, así como el segundo y cuarto elementos serán intercambiados. El tercero permanecerá intacto.

Pregunta: ¿cómo se pueden intercambiar los valores de dos variables?

Echa un vistazo al fragmento:

```
1 variable_1 = 1
2 variable_2 = 2
3
4 variable_2 = variable_1
5 variable_1 = variable_2
6
```

Si haces algo como esto, **perderás el valor previamente almacenado** en variable_2. Cambiar el orden de las tareas no ayudará. Necesitas una tercera variable **que sirva como almacenamiento auxiliar**.

Así es como puedes hacerlo:

```
1 variable_1 = 1
2 variable_2 = 2
3
4 auxiliary = variable_1
5 variable_1 = variable_2
6 variable_2 = auxiliary
7
```

Python ofrece una forma más conveniente de hacer el intercambio - echa un vistazo:

```
1 variable_1 = 1
2 variable_2 = 2
3
4 variable_1, variable_2 = variable_2, variable_1
5
```

3.5 ORDENANDO LISTAS SIMPLES: EL ORDENAMIENTO DE BURBUJA

Bienvenido a la sección 5, donde aprenderás a cómo ordenar listas simples utilizando el algoritmo de ordenamiento burbuja.

3.5.1 Ordenamiento Burbuja

Ahora que puedes hacer malabarismos con los elementos de las listas, es hora de aprender como ordenarlos. Se han inventado muchos algoritmos de clasificación, que difieren mucho en velocidad, así como en complejidad. Vamos a mostrar un algoritmo muy simple, fácil de entender, pero

desafortunadamente, tampoco es muy eficiente. Se usa muy raramente, y ciertamente no para listas extensas.

Digamos que una lista se puede ordenar de dos maneras:

- ascendente (o más precisamente - no descendente) - si en cada par de elementos adyacentes, el primer elemento no es mayor que el segundo;
- descendente (o más precisamente - no ascendente) - si en cada par de elementos adyacentes, el primer elemento no es menor que el segundo.

En las siguientes secciones, ordenaremos la lista en orden ascendente, de modo que los números se ordenen de menor a mayor.

Aquí está la lista:

8	10	6	2	4
---	----	---	---	---

Intentaremos utilizar el siguiente enfoque: tomaremos el primer y el segundo elemento y los compararemos; si determinamos que están en el orden incorrecto (es decir, el primero es mayor que el segundo), los intercambiaremos; si su orden es válido, no haremos nada. Un vistazo a nuestra lista confirma lo último - los elementos 01 y 02 están en el orden correcto, así como $8 < 10$.

Ahora observa el segundo y el tercer elemento. Están en las posiciones equivocadas. Tenemos que intercambiarlos:

8	6	10	2	4
---	---	----	---	---

Vamos más allá y observemos los elementos tercero y cuarto. Una vez más, esto no es lo que se supone que es. Tenemos que intercambiarlos:

8	6	2	10	4
---	---	---	----	---

Ahora comprobemos los elementos cuarto y quinto. Si, ellos también están en las posiciones equivocadas. Ocurre otro intercambio:

8	6	2	4	10
---	---	---	---	----

El primer paso a través de la lista ya está terminado. Todavía estamos lejos de terminar nuestro trabajo, pero algo curioso ha sucedido mientras tanto. El elemento más grande, 10, ya ha llegado al final de la lista. Ten en cuenta que este es el **lugar deseado** para el. Todos los elementos restantes forman un lío pintoresco, pero este ya está en su lugar.

Ahora, por un momento, intenta imaginar la lista de una manera ligeramente diferente - es decir, de esta manera:

10
4
2
6
8

Observa - El 10 está en la parte superior. Podríamos decir que flotó desde el fondo hasta la superficie, al igual que las burbujas **en una copa de champán**. El método de clasificación deriva su nombre de la misma observación - se denomina **ordenamiento burbuja**.

Ahora comenzamos con el segundo paso a través de la lista. Miramos el primer y el segundo elemento - es necesario un intercambio:

6	8	2	4	10
---	---	---	---	----

Es hora del segundo y tercer elemento: también tenemos que intercambiarlos:

6	2	8	4	10
---	---	---	---	----

Ahora el tercer y cuarto elementos, y la segunda pasada, se completa, ya que 8 ya está en su lugar:

6	2	4	8	10
---	---	---	---	----

Comenzamos el siguiente pase inmediatamente. Observa atentamente el primer y el segundo elemento - se necesita otro cambio:

2	6	4	8	10
---	---	---	---	----

Ahora 6 necesita ir a su lugar. Cambiamos el segundo y el tercer elemento:

2	4	6	8	10
---	---	---	---	----

La lista ya está ordenada. No tenemos nada más que hacer. Esto es exactamente lo que queremos.

Como puedes ver, la esencia de este algoritmo es simple: **comparamos los elementos adyacentes y, al intercambiar algunos de ellos, logramos nuestro objetivo**.

Codifiquemos en Python todas las acciones realizadas durante un solo paso a través de la lista, y luego consideraremos cuántos pases necesitamos para realizarlo. No hemos explicado esto hasta ahora, pero lo haremos pronto.

3.5.2 Ordenando una lista

¿Cuántos pases necesitamos para ordenar la lista completa?

Resolvamos este problema de la siguiente manera: **introducimos otra variable**, su tarea es observar si se ha realizado algún intercambio durante el pase o no. Si no hay intercambio, entonces la lista ya está ordenada, y no hay que hacer nada más. Creamos una variable llamada `swapped`, y le asignamos un valor de `False` para indicar que no hay intercambios. De lo contrario, se le asignará `True`.

```
main.py > ...
1 my_list = [8, 10, 6, 2, 4] # lista a ordenar
2
3 for i in range(len(my_list) - 1): # necesitamos (5 - 1) comparaciones
4     if my_list[i] > my_list[i + 1]: # compara elementos adyacentes
5         my_list[i], my_list[i + 1] = my_list[i + 1], my_list[i] # si terminamos aquí, tenemos que intercambiar elementos.
```

Deberías poder leer y comprender este programa sin ningún problema:

```
▶ ↺ ⬇
1 my_list = [8, 10, 6, 2, 4] # lista a ordenar
2 swapped = True # Lo necesitamos verdadero (True) para ingresar al bucle while.
3
4 while swapped:
5     swapped = False # no hay intercambios hasta ahora
6     for i in range(len(my_list) - 1):
7         if my_list[i] > my_list[i + 1]:
8             swapped = True # ¡ocurrió el intercambio!
9             my_list[i], my_list[i + 1] = my_list[i + 1], my_list[i]
10
11 print(my_list)
12
13
```

Console

```
[2, 4, 6, 8, 10]
```

Python, sin embargo, tiene sus propios mecanismos de clasificación. Nadie necesita escribir sus propias clases, ya que hay un número suficiente de **herramientas listas-para-usar**.

Te explicamos este sistema de clasificación porque es importante aprender como procesar los contenidos de una lista y mostrarte como puede funcionar la clasificación real.

Si quieres que Python ordene tu lista, puedes hacerlo de la siguiente manera:

```
1 my_list = [8, 10, 6, 2, 4]
2 my_list.sort()
3 print(my_list)
4
```

Es tan simple como eso.

```
▶ ↺ ⬇
1 a = "A"
2 b = "B"
3 c = "C"
4 d = " "
5
6 lst = [a, b, c, d]
7 lst.reverse()
8
9 print(lst)
```

Console

```
[' ', 'C', 'B', 'A']
```

Revertimos la lista.

3.6 OPERACIONES CON LISTAS

En esta sección, aprenderás a cómo procesar listas utilizando rebanadas y los operadores *in* y *not in*. También analizarás algunos programas simples que utilizan el concepto de listas para aprender a aplicarlas en proyectos más desafiantes.

3.6.1 La vida al interior de las listas

Ahora queremos mostrarte una característica importante y muy sorprendente de las listas, que las distingue de las variables ordinarias.

Queremos que lo memorices - ya que puede afectar tus programas futuros y causar graves problemas si se olvida o se pasa por alto.

Echa un vistazo al fragmento en el editor.

```
1 list_1 = [1]
2 list_2 = list_1
3 list_1[0] = 2
4 print(list_2)
5
```

Console [x]

[2]

El programa:

- crea una lista de un elemento llamada list_1;
- la asigna a una nueva lista llamada list_2;
- cambia el único elemento de list_1;
- imprime la list_2;

La parte sorprendente es el hecho de que el programa mostrará como resultado: [2], no [1], que parece ser la solución obvia.

Las listas (y muchas otras entidades complejas de Python) se almacenan de diferentes maneras que las variables ordinarias (escalares).

Se podría decir que:

- el nombre de una variable ordinaria es el **nombre de su contenido**;
- el nombre de una lista es el nombre de una ubicación de memoria **donde se almacena la lista**.

Lee estas dos líneas una vez más - la diferencia es esencial para comprender que vamos a hablar a continuación.

La asignación: list_2 = list_1 copia el nombre del arreglo no su contenido. En efecto, los dos nombres (list_1 y list_2) identifican la misma ubicación en la memoria de la computadora. Modificar uno de ellos afecta al otro, y viceversa.

¿Cómo te las arreglas con eso?

3.6.2 Rebanadas poderosas

Afortunadamente, la solución está al alcance de tu mano - su nombre es rebanada.

Una rebanada es un elemento de la sintaxis de Python que permite hacer una copia nueva de una lista, o partes de una lista.

En realidad, copia el contenido de la lista, no el nombre de la lista.

Esto es exactamente lo que necesitas. Echa un vistazo al fragmento de código a continuación:

```
1 list_1 = [1]
2 list_2 = list_1[:]
3 list_1[0] = 2
4 print(list_2)
```

Console

[1]

Su output es [1].

Esta parte no visible del código descrito como [:] puede producir una lista completamente nueva.

Una de las formas más generales de la rebanada es la siguiente:

```
my_list[inicio:fin]
```

Como puedes ver, se asemeja a la indexación, pero los dos puntos en el interior hacen una gran diferencia.

Una rebanada de este tipo **crea una nueva lista (de destino), tomando elementos de la lista de origen - los elementos de los índices desde el start hasta el fin fin - 1.**

Nota: no hasta el fin sino hasta fin-1. Un elemento con un índice igual a fin es el primer elemento el cual **no participa en la segmentación.**

Es posible utilizar valores negativos tanto para el inicio como para el fin(al igual que en la indexación).

Echa un vistazo al fragmento:

```
1 # Copiando la lista completa.
2 list_1 = [1]
3 list_2 = list_1[:]
4 list_1[0] = 2
5 print(list_2)
6
7 # Copiando parte de la lista.
8 my_list = [10, 8, 6, 4, 2]
9 new_list = my_list[1:3]
10 print(new_list)
11
```

Console

[1]

[8, 6]

3.6.3 Rebanadas – índices negativos

Observa el fragmento de código a continuación:

```
my_list[start:end]
```

Para confirmar:

- start es el índice del primer elemento **incluido en la rebanada**.
- end es el índice del primer elemento **no incluido en la rebanada**.

Así es como **los índices negativos** funcionan en las rebanadas:

```
1 my_list = [10, 8, 6, 4, 2]
2 new_list = my_list[1:-1]
3 print(new_list)
4
```

El resultado del fragmento es:

```
[8, 6, 4]
```

Si start especifica un elemento que se encuentra más allá del descrito por end (desde el punto de vista inicial de la lista), la rebanada estará **vacía**:

```
1 my_list = [10, 8, 6, 4, 2]
2 new_list = my_list[-1:1]
3 print(new_list)
4
```

La output del fragmento es: []

Si omites el start en tu rebanada, se supone que deseas obtener un segmento que comienza en el elemento con índice 0.

En otras palabras, la rebanada sería de esta forma:

```
my_list[:end]
```

es un equivalente más compacto de:

```
my_list[0:end]
```

Observa el fragmento de código a continuación:

```
1 my_list = [10, 8, 6, 4, 2]
2 new_list = my_list[:3]
3 print(new_list)
4
```

Es por esto que su output es: [10, 8, 6].

Del mismo modo, si omites el end en tu rebanada, se supone que deseas que el segmento termine en el elemento con el índice len(my_list).

En otras palabras, la rebanada sería de esta forma:

```
my_list[start:]
```

es un equivalente más compacto de:

```
my_list[start:len(my_list)]
```

3.6.3.1 Más sobre la instrucción del

La instrucción del descrita anteriormente puede **eliminar más de un elemento de la lista a la vez** - **también puede eliminar rebanadas**:

```
1 my_list = [10, 8, 6, 4, 2]
2 del my_list[1:3]
3 print(my_list)
4
```

La lista se queda vacía y la output es: [].

Al eliminar la rebanada del código, su significado cambia dramáticamente.

Echa un vistazo:

```
1 my_list = [10, 8, 6, 4, 2]
2 del my_list
3 print(my_list)
4
5
```

Console

Traceback (most recent call last):
File "main.py", line 3, in <module>
print(my_list)
NameError: name 'my_list' is not defined

La instrucción del **eliminará la lista, no su contenido**.

La función print() de la última línea del código provocará un error de ejecución.

3.6.4 Los operadores in y not in

Python ofrece dos operadores muy poderosos, capaces de revisar la lista para verificar si un valor específico está almacenado dentro de la lista o no.

Estos operadores son:

```
1 elem in my_list
2 elem not in my_list
3
```

El primero de ellos (in) verifica si un elemento dado (el argumento izquierdo) está actualmente almacenado en algún lugar dentro de la lista (el argumento derecho) - el operador devuelve True en este caso.

El segundo (not in) comprueba si un elemento dado (el argumento izquierdo) está ausente en una lista - el operador devuelve True en este caso.

Observa el código en el editor. El fragmento muestra ambos operadores en acción. ¿Puedes adivinar su output? Ejecuta el programa para comprobar si tenías razón.

```
1 my_list = [0, 3, 12, 8, 2]
2
3 print(5 in my_list)
4 print(5 not in my_list)
5 print(12 in my_list)
6
7
```

Console

```
False
True
True
```

3.7 APLICACIONES AVANZADAS DE LISTAS

En esta sección aprenderás sobre arreglos, listas anidadas (matrices) y listas por comprensión.

3.7.1 Listas dentro de listas

Las listas pueden constar de escalares (es decir, números) y elementos de una estructura mucho más compleja (ya has visto ejemplos como cadenas, booleanos o incluso otras listas en las lecciones del Resumen de la Sección anterior). Veamos más de cerca el caso en el que los elementos de una lista son listas.

A menudo encontramos estos arreglos en nuestras vidas. Probablemente el mejor ejemplo de esto sea un tablero de ajedrez.

Un tablero de ajedrez está compuesto de filas y columnas. Hay ocho filas y ocho columnas. Cada columna está marcada con las letras de la A a la H. Cada línea está marcada con un número del uno al ocho.

La ubicación de cada campo se identifica por pares de letras y dígitos. Por lo tanto, sabemos que la esquina inferior derecha del tablero (la que tiene la torre blanca) es A1, mientras que la esquina opuesta es H8.

Supongamos que podemos usar los números seleccionados para representar cualquier pieza de ajedrez. También podemos asumir que cada fila en el tablero de ajedrez es una lista.

Observa el siguiente código:

```
1 row = []
2
3 for i in range(8):
4     row.append(WHITE_PAWN)
5
```

Crea una lista que contiene ocho elementos que representan la segunda fila del tablero de ajedrez: la que está llena de peones (supón que WHITE_PAWN es un **símbolo predefinido** que representa un peón blanco).

3.7.1.1 Comprensión de lista

El mismo efecto se puede lograr mediante una comprensión de lista, la sintaxis especial utilizada por Python para completar o llenar listas masivas.

Una comprensión de lista es en realidad una lista, pero se creó sobre la marcha durante la ejecución del programa, y no se describe de forma estática.

Echa un vistazo al fragmento:

```
1 row = [WHITE_PAWN for i in range(8)]
2
```

La parte del código colocada dentro de los paréntesis especifica:

- los datos que se utilizarán para completar la lista (WHITE_PAWN)
- la cláusula que especifica cuántas veces se producen los datos dentro de la lista (for i in range(8))

Permítenos mostrarte otros **ejemplos de comprensión de lista**:

Ejemplo #1:

```
1 squares = [x ** 2 for x in range(10)]
2
```

El fragmento de código genera una lista de diez elementos y la rellena con cuadrados de diez números enteros que comienzan desde cero (0, 1, 4, 9, 16, 25, 36, 49, 64, 81)

Ejemplo #2:

```
1 twos = [2 ** i for i in range(8)]
2
```

El fragmento crea un arreglo de ocho elementos que contiene las primeras ocho potencias del número dos (1, 2, 4, 8, 16, 32, 64, 128)

Ejemplo #3:

```
1 odds = [x for x in squares if x % 2 != 0]
2
```

El fragmento crea una lista con solo los elementos impares de la lista squares.

3.7.2 Arreglos de dos dimensiones

Supongamos también que un símbolo predefinido denominado EMPTY designa un campo vacío en el tablero de ajedrez.

Entonces, si queremos crear una lista de listas que representen todo el tablero de ajedrez, se puede hacer de la siguiente manera:

```
1 board = []
2
3 for i in range(8):
4     row = [EMPTY for i in range(8)]
5     board.append(row)
6
```

Nota:

- la parte interior del bucle crea una fila que consta de ocho elementos (cada uno de ellos es igual a `EMPTY`) y lo agrega a la lista del board;
- la parte exterior se repite ocho veces;
- en total, la lista board consta de 64 elementos (todos iguales a `EMPTY`).

Este modelo imita perfectamente el tablero de ajedrez real, que en realidad es una lista de elementos de ocho elementos, todos ellos en filas individuales. Resumamos nuestras observaciones:

- los elementos de las filas son campos, ocho de ellos por fila;
- los elementos del tablero de ajedrez son filas, ocho de ellos por tablero de ajedrez.

La variable board ahora es un **arreglo bidimensional**. También se le llama, por analogía a los términos algebraicos, una **matriz**.

Como las listas de comprensión puede ser **anidadas**, podemos acortar la creación del tablero de la siguiente manera:

```
1 board = [[EMPTY for i in range(8)] for j in range(8)]
2
```

La parte interna crea una fila, y la parte externa crea una lista de filas.

El acceso al campo seleccionado del tablero requiere dos índices - el primero selecciona la fila; el segundo - el número del campo dentro de la fila, el cual es un número de columna.

Echa un vistazo al tablero de ajedrez. Cada campo contiene un par de índices que se deben dar para acceder al contenido del campo:

	A	B	C	D	E	F	G	H	
8	[0][0]	[0][1]	[0][2]	[0][3]	[0][4]	[0][5]	[0][6]	[0][7]	8
7	[1][0]	[1][1]	[1][2]	[1][3]	[1][4]	[1][5]	[1][6]	[1][7]	7
6	[2][0]	[2][1]	[2][2]	[2][3]	[2][4]	[2][5]	[2][6]	[2][7]	6
5	[3][0]	[3][1]	[3][2]	[3][3]	[3][4]	[3][5]	[3][6]	[3][7]	5
4	[4][0]	[4][1]	[4][2]	[4][3]	[4][4]	[4][5]	[4][6]	[4][7]	4
3	[5][0]	[5][1]	[5][2]	[5][3]	[5][4]	[5][5]	[5][6]	[5][7]	3
2	[6][0]	[6][1]	[6][2]	[6][3]	[6][4]	[6][5]	[6][6]	[6][7]	2
1	[7][0]	[7][1]	[7][2]	[7][3]	[7][4]	[7][5]	[7][6]	[7][7]	1
	A	B	C	D	E	F	G	H	

Echando un vistazo a la figura que se muestra arriba, coloquemos algunas piezas de ajedrez en el tablero. Primero, agreguemos todas las torres:

```
1 board[0][0] = ROOK
2 board[0][7] = ROOK
3 board[7][0] = ROOK
4 board[7][7] = ROOK
5
```

Si deseas agregar un caballo a C4, hazlo de la siguiente manera:

```
1 board[4][2] = KNIGHT
2
```

Y ahora un peón a E5:

```
1 board[3][4] = PAWN
2
```

3.7.3 Naturaleza multidimensional de las listas: aplicaciones avanzadas

Profundicemos en la naturaleza multidimensional de las listas. Para encontrar cualquier elemento de una lista bidimensional, debes usar dos coordenadas:

- Una vertical (número de fila).
- Una horizontal (número de columna).

Imagina que desarrollas una pieza de software para una estación meteorológica automática. El dispositivo registra la temperatura del aire cada hora y lo hace durante todo el mes. Esto te da un total de $24 \times 31 = 744$ valores. Intentemos diseñar una lista capaz de almacenar todos estos resultados.

Primero, debes decidir qué tipo de datos sería adecuado para esta aplicación. En este caso, sería mejor un float, ya que este termómetro puede medir la temperatura con una precisión de 0.1 °C.

Luego tomarás la decisión arbitraria de que las filas registrarán las lecturas cada hora exactamente (por lo que la fila tendrá 24 elementos) y cada una de las filas se asignará a un día del mes (supongamos que cada mes tiene 31 días, por lo que necesita 31 filas). Aquí está el par apropiado de comprensiones (h es para las horas, d para el día):

```
1 temps = [[0.0 for h in range(24)] for d in range(31)]
2
```

Toda la matriz está llena de ceros ahora. Puede suponer que se actualiza automáticamente utilizando agentes de hardware especiales. Lo que tienes que hacer es esperar a que la matriz se llene con las mediciones.

Ahora es el momento de determinar la temperatura promedio mensual del mediodía. Suma las 31 lecturas registradas al mediodía y divide la suma por 31. Puedes suponer que la temperatura de medianoche se almacena primero. Aquí está el código:

```

1 temps = [[0.0 for h in range(24)] for d in range(31)]
2 #
3 # La matriz se actualiza aquí.
4 #
5
6 total = 0.0
7
8 for day in temps:
9     total += day[11]
10
11 average = total / 31
12
13 print("Temperatura promedio al mediodía:", average)
14

```

Console

Temperatura promedio al mediodía: 0.0

Nota: La variable day utilizada por el bucle for no es un escalar - cada paso a través de la matriz temps lo asigna a la siguiente fila de la matriz; Por lo tanto, es una lista. Se debe indexar con 11 para acceder al valor de temperatura medida al mediodía.

Ahora encuentra la temperatura más alta durante todo el mes - analiza el código:

```

1 temps = [[0.0 for h in range(24)] for d in range(31)]
2 #
3 # La matriz se actualiza aquí.
4 #
5
6 highest = -100.0
7
8 for day in temps:
9     for temp in day:
10         if temp > highest:
11             highest = temp
12
13 print("La temperatura más alta fue:", highest)
14

```

Console

La temperatura más alta fue: 0.0

Note:

- la variable day itera en todas las filas de la matriz temps;
- la variable temp itera a través de todas las mediciones tomadas en un día.

Ahora cuenta los días en que la temperatura al mediodía fue de al menos 20 °C:

```

1 temps = [[0.0 for h in range(24)] for d in range(31)]
2 #
3 # La matriz se actualiza aquí.
4 #
5
6 hot_days = 0
7
8 for day in temps:
9     if day[11] > 20.0:
10         hot_days += 1
11
12 print(hot_days, "fueron los días calurosos.")
13

```

Console

0 fueron los días calurosos.

Python no limita la profundidad de la inclusión lista en lista. Aquí puedes ver un ejemplo de un arreglo tridimensional:

```
1 rooms = [[[False for r in range(20)] for f in range(15)] for t in range(3)]
2
```

El primer índice (0 a 2) selecciona uno de los edificios; el segundo (0 a 14) selecciona el piso, el tercero (0 a 19) selecciona el número de habitación. Todas las habitaciones están inicialmente desocupadas.

Ahora ya puedes reservar una habitación para dos recién casados: en el segundo edificio, en el décimo piso, habitación 14:

```
1 rooms[1][9][13] = True
2
```

y desocupar el segundo cuarto en el quinto piso ubicado en el primer edificio:

```
1 rooms[0][4][1] = False
2
```

Verifica si hay disponibilidad en el piso 15 del tercer edificio:

```
1 vacancy = 0
2
3 for room_number in range(20):
4     if not rooms[2][14][room_number]:
5         vacancy += 1
6
```

La variable vacancy contiene 0 si todas las habitaciones están ocupadas, o en dado caso el número de habitaciones disponibles.

4 FUNCIONES, TUPLAS, DICCIONARIOS, EXCEPCIONES Y PROCESAMIENTO DE DATOS

4.1 FUNCIONES

¡Bienvenido al Módulo 4! En la sección 1 aprenderás a crear, usar, y mandar llamar a tus propias funciones. ¡Comencemos!

4.1.1 ¿Por qué necesitamos funciones?

Hasta ahorita has implementado varias veces el uso de funciones, pero solo se han visto algunas de sus ventajas. Solo se han invocado funciones para utilizarlas como herramientas, con el fin de hacer la vida más fácil, y para simplificar tareas tediosas y repetitivas.

Cuando se desea mostrar o imprimir algo en consola se utiliza print(). Cuando se desea leer el valor de una variable se emplea input(), combinados posiblemente con int() o float().

También se ha hecho uso de algunos métodos, los cuales también son funciones, pero declarados de una manera muy específica.

Ahora aprenderás a escribir tus propias funciones, y como utilizarlas. Escribiremos varias de ellas juntos, desde muy sencillas hasta algo complejas. Se requerirá de tu concentración y atención.

Muy a menudo ocurre que un cierto fragmento de código se repite muchas veces en un programa. Se repite de manera literal o, con algunas modificaciones menores, empleando algunas otras variables dentro del programa. También ocurre que un programador ha comenzado a copiar y pegar ciertas partes del código en más de una ocasión en el mismo programa.

Puede ser muy frustrante percatarse de repente que existe un error en el código copiado. El programador tendrá que escarbar bastante para encontrar todos los lugares en el código donde hay que corregir el error. Además, existe un gran riesgo de que las correcciones produzcan errores adicionales.

Definamos la primera condición por la cual es una buena idea comenzar a escribir funciones propias: si un fragmento de código comienza a aparecer en más de una ocasión, considera la posibilidad de aislarlo en la forma de una función invocando la función desde el lugar en el que originalmente se encontraba.

Puede suceder que el algoritmo que se desea implementar sea tan complejo que el código comience a crecer de manera incontrolada y, de repente, ya no se puede navegar por él tan fácilmente.

Se puede intentar solucionar este problema comentando el código, pero pronto te darás cuenta de que esto empeorará la situación - demasiados comentarios hacen que el código sea más difícil de leer y entender. Algunos dicen que una función bien escrita debe ser comprensible con tan solo una mirada.

Un buen desarrollador divide el código (o mejor dicho: el problema) en piezas aisladas, y codifica cada una de ellas en la forma de una función.

Esto simplifica considerablemente el trabajo del programa, debido a que cada pieza se codifica por separado, y consecuentemente se prueba por separado. A este proceso se le llama comúnmente **descomposición**.

Existe una segunda condición: **si un fragmento de código se hace tan extenso que leerlo o entenderlo se hace complicado, considera dividirlo en pequeños problemas, por separado, e implementa cada uno de ellos como una función independiente.**

Esta descomposición continúa hasta que se obtiene un conjunto de funciones cortas, fáciles de comprender y probar.

4.1.2 Descomposición

Es muy común que un programa sea tan largo y complejo que no puede ser asignado a un solo desarrollador, y en su lugar un equipo de desarrolladores trabajará en él. El problema, debe ser dividido entre varios desarrolladores de una manera en que se pueda asegurar su eficiencia y cooperación.

Es inconcebible que más de un programador deba escribir el mismo código al mismo tiempo, por lo tanto, el trabajo debe de ser dividido entre todos los miembros del equipo.

Este tipo de descomposición tiene diferentes propósitos, no solo se trata de compartir el trabajo, sino también de compartir la responsabilidad entre varios desarrolladores.

Cada uno debe escribir un conjunto bien definido y claro de funciones, las cuales al ser combinadas dentro de un módulo (esto se clarificará un poco más adelante) nos dará como resultado el producto final.

Esto nos lleva directamente a la tercera condición: si se va a dividir el trabajo entre varios programadores, se debe descomponer el problema para permitir que el producto sea

implementado como un conjunto de funciones escritas por separado empacadas juntas en diferentes módulos.

4.1.3 ¿De dónde provienen las funciones?

En general, las funciones provienen de al menos tres lugares:

Python: Varias funciones como `print()` son una parte integral de Python, y siempre están disponibles sin algún esfuerzo adicional del programador; se les llama a estas funciones integradas.

Módulos: Muchas de las funciones, las cuales comúnmente son menos utilizadas que las integradas, están disponibles en módulos instalados juntamente con Python; para poder utilizar estas funciones el programador debe realizar algunos pasos adicionales (se explicara acerca de eso en unos momentos).

Directamente del Código: tu puedes escribir tus propias funciones, colocarlas dentro del código y usarlas libremente.

4.1.4 Tu primera función

¿Es posible separar ese código *repetido*, darle un nombre y hacerlo reutilizable? Significaría que **el cambio hecho en un solo lugar será propagado a todos los lugares donde se utilice**.

Para que esto funcione, dicho código debe ser invocado cada vez que se requiera.

Es posible, esto es exactamente para lo que existen las funciones.

Se necesita **definirla**. Aquí, la palabra *definir* es significativa.

Así es como se ve la definición más simple de una función:

```
def function_name():  
    cuerpo de la función
```

- Siempre comienza con la palabra reservada `def` (que significa definir)
- después de `def` va el nombre de la función (las reglas para darle nombre a las funciones son las mismas que para las variables)
- después del nombre de la función, hay un espacio para un par de paréntesis (ahorita no contienen algo, pero eso cambiará pronto)
- la línea debe de terminar con dos puntos;
- la línea inmediatamente después de `def` marca el comienzo del cuerpo de la función - donde varias o (al menos una) instrucción anidada, será ejecutada cada vez que la función sea invocada; nota: la función termina donde el anidamiento termina, se debe ser cauteloso.

A continuación, se **definirá** la función. Se llamará `message` – aquí está:

```
1 | def message():  
2 |     print("Ingresar valor: ")
```

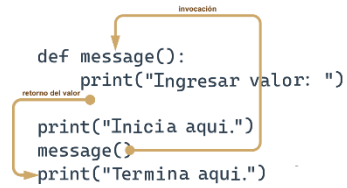
La función es muy sencilla, pero completamente **utilizable**. Se ha nombrado `message`, pero eso es opcional, tu puedes cambiarlo. Hagamos uso de ella.

Cómo funcionan las funciones

Observa la imagen.

Intenta mostrarte el proceso completo:

- cuando se invoca una función, Python recuerda el lugar donde esto ocurre y salta hacia dentro de la función invocada;
- el cuerpo de la función es entonces ejecutado;
- al llegar al final de la función, Python regresa al lugar inmediato después de donde ocurrió la invocación.



Existen dos consideraciones muy importantes, la primera de ella es:

No se debe invocar una función antes de que se haya definido.

Recuerda - Python lee el código de arriba hacia abajo. No va a adelantarse en el código para determinar si la función invocada está definida más adelante, (el lugar "correcto" para definirla es "antes de ser invocada".)

No intentes forzar a Python a encontrar funciones que no están definidas en el lugar correcto.

La segunda consideración es más sencilla:

Una función y una variable no pueden compartir el mismo nombre.

El asignar un valor al nombre message causa que Python olvide su rol anterior.

4.2 CÓMO LAS FUNCIONES SE COMUNICAN CON SU ENTORNO

En esta sección aprenderás sobre funciones con y sin parámetros, así como también a escribir funciones de uno-, dos-, y tres-parámetros y pasarles argumentos. ¡Vamos a empezar!

4.2.1 Funciones parametrizadas

El potencial completo de una función se revela cuando puede ser equipada con una interface que es capaz de aceptar datos provenientes de la invocación. Dichos datos pueden modificar el comportamiento de la función, haciéndola más flexible y adaptable a condiciones cambiantes.

Un parámetro es una variable, pero existen dos factores que hacen a un parámetro diferente:

- los parámetros solo existen dentro de las funciones en donde han sido definidos, y el único lugar donde un parámetro puede ser definido es entre los paréntesis después del nombre de la función, donde se encuentra la palabra clave reservada def;
- la asignación de un valor a un parámetro de una función se hace en el momento en que la función se manda llamar o se invoca, especificando el argumento correspondiente.

```
def function(parameter):  
    ###
```

Recuerda que:

- los parámetros solo existen dentro de las funciones (este es su entorno natural)
- los argumentos existen fuera de las funciones, y son los que pasan los valores a los parámetros correspondientes.

Existe una clara división entre estos dos mundos.

Enriquezcamos la función anterior agregándole un parámetro - se utilizará para mostrar al usuario el valor de un número que la función pide.

Se tendrá que modificar la definición def de la función - así es como se ve ahora:

```
def message(number):  
    ###
```

Esta definición especifica que nuestra función opera con un solo parámetro con el nombre de number. Se puede utilizar como una variable normal, pero **solo dentro de la función** - no es visible en otro lugar.

Un valor para el parámetro llegará del entorno de la función.

Recuerda: **especificar uno o más parámetros en la definición de la función** es un requerimiento, y se debe de cumplir durante la invocación de la misma. Se debe **proveer el mismo número de argumentos como haya parámetros definidos**.

El no hacerlo provocará un error.

Existe una circunstancia importante que se debe mencionar.

Es posible tener una variable con el mismo nombre del parámetro de la función.

El siguiente código muestra un ejemplo de esto:

```
1  def message(number):  
2      print("Ingresa un número:", number)  
3  
4      number = 1234  
5      message(1)  
6      print(number)  
7
```

Una situación como la anterior activa un mecanismo denominado **sombreado**:

- el parámetro x sombrea cualquier variable con el mismo nombre, pero...
- ... solo dentro de la función que define el parámetro.

El parámetro llamado number es una entidad completamente diferente de la variable llamada number.

Una función puede tener **tantos parámetros como se desee**, pero entre más parámetros, es más difícil memorizar su rol y propósito.

Modifiquemos la función- ahora tiene **dos parámetros**:

```
def message(what, number):  
    print("Ingresa", what, "número", number)
```

Esto significa que para invocar la función se necesitan dos argumentos.

El primer parámetro va a contener el nombre del valor deseado.

4.2.2 Paso de parámetros posicionales

La técnica que asigna cada argumento al parámetro correspondiente, es llamada paso de parámetros posicionales, los argumentos pasados de esta manera son llamados argumentos posicionales.

Ya se ha utilizado, pero Python ofrece mucho más. Se abordará este tema a continuación.

```
1 def my_function(a, b, c):
2     print(a, b, c)
3
4 my_function(1, 2, 3)
5
```

Nota: el paso de parámetros posicionales es usado de manera intuitiva por las personas en muchas situaciones. Por ejemplo, es generalmente aceptado que cuando nos presentamos mencionamos primero nuestro nombre(s) y después nuestro apellido, por ejemplo, "Me llamo Juan Pérez."

4.2.3 Paso de argumentos de palabra clave

Python ofrece otra manera de pasar argumentos, donde **el significado del argumento está definido por su nombre**, no su posición - a esto se le denomina **paso de argumentos con palabra clave**.

Observa el siguiente código:

```
1 def introduction(first_name, last_name):
2     print("Hola, mi nombre es", first_name, last_name)
3
4 introduction(first_name = "James", last_name = "Bond")
5 introduction(last_name = "Skywalker", first_name = "Luke")
6
7
```

Console

```
Hola, mi nombre es James Bond
Hola, mi nombre es Luke Skywalker
```

El concepto es claro - los valores pasados a los parámetros son precedidos por el nombre del parámetro al que se le va a pasar el valor, seguido por el signo de =.

La posición no es relevante aquí - cada argumento conoce su destino con base en el nombre utilizado.

4.2.4 Mezclando argumentos posicionales y de palabras clave

Es posible combinar ambos tipos si se desea - solo hay una regla inquebrantable: se deben colocar primero los **argumentos posicionales** y después los **de palabra clave**.

Piénsalo por un momento, entenderás el porque.

Para mostrarte cómo funciona, se utilizará la siguiente función de tres parámetros:

```

1 def adding(a, b, c):
2     print(a, "+", b, "+", c, "=", a + b + c)
3
4 adding(1, 2, 3)
5
6

```

Console

```

1 + 2 + 3 = 6

```

4.2.5 Funciones parametrizadas - más detalles

En ocasiones ocurre que algunos valores de ciertos argumentos son más utilizados que otros. Dichos argumentos tienen valores predefinidos los cuales pueden ser considerados cuando los argumentos correspondientes han sido omitidos.

Uno de los apellidos más comunes en Latinoamérica es González. Tomémoslo para el ejemplo.

El valor por determinada para el parámetro se asigna de la siguiente manera:

```

1 def introduction(first_name, last_name="Smith"):
2     print("Hola, mi nombre es", first_name, last_name)
3
4 introduction("Jorge", "Pérez")
5
6

```

Console

```

Hola, mi nombre es Jorge Pérez

```

¿Y? No parece haber cambiado algo, pero cuando se invoca la función de una manera inusual, como esta:

```

1 def introduction(first_name, last_name="Smith"):
2     print("Hola, mi nombre es", first_name, last_name)
3
4 introduction("Enrique")
5
6
7
8

```

Console

```

Hola, mi nombre es Enrique Smith

```

no habrá errores.

4.3 RETORNANDO EL RESULTADO DE UNA FUNCIÓN

¡Bienvenido a la sección 3! En esta parte del curso, aprenderás sobre los efectos y resultados de las funciones, la expresión `return` y el valor `None`. También aprenderás a pasar listas como argumentos

de funciones, cómo devolver listas como resultados de funciones y cómo asignar resultados de funciones a variables. ¡Vamos!

4.3.1 Efectos y resultados: la instrucción return

Todas las funciones presentadas anteriormente tienen algún tipo de efecto - producen un texto y lo envían a la consola.

Por supuesto, las funciones - al igual que las funciones matemáticas - pueden tener resultados.

Para lograr que las funciones devuelvan un valor (pero no solo para ese propósito) se utiliza la instrucción return (regresar o retornar).

Esta palabra nos da una idea completa de sus capacidades. Nota: es una palabra clave reservada de Python.

La instrucción return tiene dos variantes diferentes - considerémoslas por separado.

4.3.1.1 *return sin una expresión*

Consideremos la siguiente función:

```
1 def happy_new_year(wishes = True):
2     print("Tres...")
3     print("Dos...")
4     print("Uno...")
5     if not wishes:
6         return
7
8     print("¡Feliz año nuevo!")
9
10 happy_new_year()
11
12
```

Console

Tres...
Dos...
Uno...
¡Feliz año nuevo!

Al proporcionar False como argumento:

```
1 def happy_new_year(wishes = True):
2     print("Tres...")
3     print("Dos...")
4     print("Uno...")
5     if not wishes:
6         return
7
8     print("¡Feliz año nuevo!")
9
10 happy_new_year(False)
11
12
```

Console

Tres...
Dos...
Uno...

4.3.1.2 *return con una expresión*

La segunda variante de return está extendida con una expresión:

```
def function():  
    return expression
```

Existen dos consecuencias de usarla:

- provoca la terminación inmediata de la ejecución de la función (nada nuevo en comparación con la primera variante)
- además, la función evaluará el valor de la expresión y lo devolverá (de ahí el nombre una vez más) como el resultado de la función.

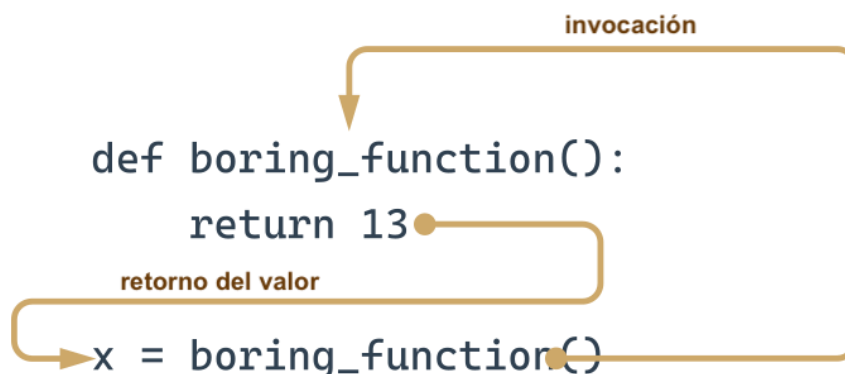
Si, este ejemplo es sencillo:

```
1 def boring_function():  
2     return 123  
3  
4 x = boring_function()  
5  
6 print("La función boring_function ha devuelto su resultado. Es:", x)  
7  
8
```

Console

La función boring_function ha devuelto su resultado. Es: 123

Analiza la siguiente figura:



La instrucción return, enriquecida con la expresión (la expresión es muy simple aquí), "transporta" el valor de la expresión al lugar donde se ha invocado la función.

El resultado se puede usar libremente aquí, por ejemplo, para ser asignado a una variable.

También puede ignorarse por completo y perderse sin dejar rastro.

No olvides:

- siempre se te permite ignorar el resultado de la función y estar satisfecho con el efecto de la función (si la función tiene alguno)
- si una función intenta devolver un resultado útil, debe contener la segunda variante de la instrucción return.

Espera un segundo - ¿significa esto que también hay resultados inútiles? Sí, en cierto sentido.

4.3.2 Unas pocas palabras sobre None

Permítenos presentarte un valor muy curioso (para ser honestos, un valor que es ninguno) llamado None.

Sus datos no representan valor razonable alguno - en realidad, no es un valor en lo absoluto; por lo tanto, no debe participar en ninguna expresión.

Nota: None es una **palabra clave reservada**.

Solo existen dos tipos de circunstancias en las que None se puede usar de manera segura:

- cuando **se le asigna a una variable** (o se devuelve como **el resultado de una función**)
- cuando **se compara con una variable** para diagnosticar su estado interno.

Al igual que aquí:

```
1 value = None
2 if value is None:
3     print("Lo siento, no contiene ningún valor")
4
```

No olvides esto: si una función no devuelve un cierto valor utilizando una cláusula de expresión return, se asume que **devuelve implícitamente None**.

Vamos a probarlo.

Echa un vistazo al código en el editor.

```
1 def strange_function(n):
2     if(n % 2 == 0):
3         return True
4 print(strange_function(2))
5 print(strange_function(1))
6
7
```

Console

True
None

No te sorprendas la próxima vez que veas None como el resultado de la función - puede ser el síntoma de un error sutil dentro de la función.

4.3.3 Efectos y resultados: listas y funciones

Existen dos preguntas adicionales que deben responderse aquí.

La primera es: **¿Se puede enviar una lista a una función como un argumento?**

¡Por supuesto que se puede! Cualquier entidad reconocible por Python puede desempeñar el papel de un argumento de función, aunque debes asegurarte de que la función sea capaz de hacer uso de él.

Entonces, si pasas una lista a una función, la función tiene que manejarla como una lista.

Una función como la siguiente:

```
1 def list_sum(lst):
2     s = 0
3
4     for elem in lst:
5         s += elem
6
7     return s
8
9 print(list_sum([5, 4, 3]))
10
11
```

Console

12

Nota: El bucle for no puede iterar un solo valor entero.

La segunda pregunta es: ¿Puede una lista ser el resultado de una función?

¡Si, por supuesto! Cualquier entidad reconocible por Python puede ser un resultado de función.

4.4 LOS ALCANCES EN PYTHON

En esta parte del curso, aprenderás sobre los alcances en Python y la palabra clave global. Al final de la sección, podrás distinguir entre variables locales y globales, y sabrás cómo utilizar el mecanismo de espacios de nombres (namespaces) en tus programas.

4.4.1 Funciones y alcances

Comencemos con una definición:

*El **alcance de un nombre** (por ejemplo, el nombre de una variable) es la parte del código donde el nombre es reconocido correctamente.*

Por ejemplo, el alcance del parámetro de una función es la función en sí. El parámetro es inaccesible fuera de la función.

Comencemos revisando si una variable creada fuera de una función es visible dentro de una función. En otras palabras, ¿El nombre de la variable se propaga dentro del cuerpo de la función?

Observa el código en el editor. Ahí está nuestro conejillo de indias.

```
1 def my_function():
2     print("¿Conozco a la variable?", var)
3
4
5 var = 1
6 my_function()
7 print(var)
8
```

Console

¿Conozco a la variable? 1
1

El resultado de la prueba es positivo.

La respuesta es: **una variable que existe fuera de una función tiene alcance dentro del cuerpo de la función.**

Esta regla tiene una excepción muy importante. Intentemos encontrarla.

Hagamos un pequeño cambio al código:

```
1 def my_function():
2     var = 2
3     print("¿Conozco a la variable?", var)
4
5
6 var = 1
7 my_function()
8 print(var)
9
10
```

Console

```
¿Conozco a la variable? 2
1
```

¿Qué es lo que ocurrió?

- la variable `var` creada dentro de la función no es la misma que cuando se define fuera de ella; parece que hay dos variables diferentes con el mismo nombre;
- además, la variable de la función es una sombra de la variable fuera de la función.

La regla anterior se puede definir de una manera más precisa y adecuada:

Una variable que existe fuera de una función tiene un alcance dentro del cuerpo de la función, excluyendo a aquellas que tienen el mismo nombre.

También significa que **el alcance de una variable existente fuera de una función solo se puede implementar dentro de una función cuando su valor es leído**. El asignar un valor hace que la función cree su propia variable.

4.4.2 Funciones y alcances: la palabra clave `global`

Al llegar a este punto, debemos hacernos la siguiente pregunta: ¿Una función es capaz de modificar una variable que fue definida fuera de ella? Esto sería muy incómodo.

Afortunadamente, la respuesta es no.

Existe un método especial en Python el cual puede extender el alcance de una variable incluyendo el cuerpo de las funciones (para poder no solo leer los valores de las variables sino también modificarlos).

Este efecto es causado por la palabra clave reservada llamada `global`:

```
1 global name
2 global name1, name2, ...
3
```

El utilizar la palabra reservada dentro de una función con el nombre o nombres de las variables separados por comas, obliga a Python a abstenerse de crear una nueva variable dentro de la función - se empleará la que se puede acceder desde el exterior.

En otras palabras, este nombre se convierte en global (tiene un **alcance global**, y no importa si se está leyendo o asignando un valor).

Observa el código en el editor.

```
1 def my_function():
2     global var
3     var = 2
4     print("¿Conozco a aquella variable?", var)
5
6
7 var = 1
8 my_function()
9 print(var)
10
```

Console

```
¿Conozco a aquella variable? 2
2
```

Se ha agregado la palabra global a la función.

Esto debe de ser suficiente evidencia para mostrar lo que la palabra clave reservada global puede hacer.

4.4.3 Cómo interactúa la función con sus argumentos

Ahora descubramos como la función interactúa con sus argumentos.

El código en el editor nos enseña algo. Como puedes observar, la función cambia el valor de su parámetro. ¿Este cambio afecta el argumento?

```
1 def my_function(n):
2     print("Yo recibí", n)
3     n += 1
4     print("Ahora tengo", n)
5
6
7 var = 1
8 my_function(var)
9 print(var)
10
```

Console

```
Yo recibí 1
Ahora tengo 2
1
```

La conclusión es obvia - **al cambiar el valor del parámetro este no se propaga fuera de la función** (más específicamente, no cuando la variable es un valor escalar, como en el ejemplo).

Esto también significa que una función recibe el **valor del argumento**, no el argumento en sí. Esto es cierto para los valores escalares.

Vale la pena revisar cómo funciona esto con las listas (¿Recuerdas las peculiaridades de asignar rebanadas de listas en lugar de asignar la lista entera?)

```
main.py > ...
1 def my_function(my_list_1):
2     print("Print #1:", my_list_1)
3     print("Print #2:", my_list_2)
4     my_list_1 = [0, 1]
5     print("Print #3:", my_list_1)
6     print("Print #4:", my_list_2)
7
8
9 my_list_2 = [2, 3]
10 my_function(my_list_2)
11 print("Print #5:", my_list_2)
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN

```
PS C:\Users\marti\OneDrive\Desktop\Universidad\Certificaciones\main.py
Print #1: [2, 3]
Print #2: [2, 3]
Print #3: [0, 1]
Print #4: [2, 3]
Print #5: [2, 3]
```

Parece ser que se sigue aplicando la misma regla.

Finalmente, la diferencia se puede observar en el siguiente ejemplo:

```
main.py > ...
1 def my_function(my_list_1):
2     print("Print #1:", my_list_1)
3     print("Print #2:", my_list_2)
4     del my_list_1[0] # Presta atención a esta línea.
5     print("Print #3:", my_list_1)
6     print("Print #4:", my_list_2)
7
8
9 my_list_2 = [2, 3]
10 my_function(my_list_2)
11 print("Print #5:", my_list_2)
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RE

```
PS C:\Users\marti\OneDrive\Desktop\Universidad\Certificaciones>
Print #1: [2, 3]
Print #2: [2, 3]
Print #3: [3]
Print #4: [3]
Print #5: [3]
```

¿Lo puedes explicar?

Intentémoslo:

- si el argumento es una lista, el cambiar el valor del parámetro correspondiente no afecta la lista (recuerda: las variables que contienen listas son almacenadas de manera diferente que las escalares)
- pero si se modifica la lista identificada por el parámetro (nota: ¡la lista, no el parámetro!), la lista reflejará el cambio.

4.4.4 Recursividad

Existe algo más que se desea mostrar: es la **recursividad**.

Este término puede describir muchos conceptos distintos, pero uno de ellos, hace referencia a la programación computacional.

Aquí, la recursividad es una **técnica donde una función se invoca a si misma**.

Tanto la factorial como la serie Fibonacci, son las mejores opciones para ilustrar este fenómeno.

La serie de Fibonacci es un claro ejemplo de recursividad. Ya te dijimos que:

$$\text{Fib}_i = \text{Fib}_{i-1} + \text{Fib}_{i-2}$$

El número i se refiere al número $i-1$, y así sucesivamente hasta llegar a los primeros dos.

¿Puede ser empleado en el código? Por supuesto que puede. Puede hacer el código más corto y claro.

La segunda versión de la función `fib()` hace uso directo de la recursividad:

```
1 def fib(n):
2     if n < 1:
3         return None
4     if n < 3:
5         return 1
6     return fib(n - 1) + fib(n - 2)
7
```

El código es mucho más claro ahora.

¿Pero es realmente seguro? ¿Implica algún riesgo?

Si, existe algo de riesgo. **Si no se considera una condición que detenga las invocaciones recursivas, el programa puede entrar en un bucle infinito.** Se debe ser cuidadoso.

El factorial también tiene un lado **recursivo**. Observa:

$$n! = 1 \times 2 \times 3 \times \dots \times n-1 \times n$$

Es obvio que:

$$1 \times 2 \times 3 \times \dots \times n-1 = (n-1)!$$

Entonces, finalmente, el resultado es:

$$n! = (n-1)! \times n$$

Esto se empleará en nuestra nueva solución.

Aquí esta:

```
1 def factorial_function(n):
2     if n < 0:
3         return None
4     if n < 2:
5         return 1
6     return n * factorial_function(n - 1)
7
```

¿Funciona? Claro que si. Pruébalo por ti mismo.

Nuestro viaje *funcional* esta por terminar. La siguiente sección abordara dos tipos de datos en Python: tuplas y diccionarios.

```
main.py > ...
1 def fib(n):
2     if n < 1:
3         return None
4     if n < 3:
5         return 1
6
7     elem_1 = elem_2 = 1
8     the_sum = 0
9     for i in range(3, n + 1):
10         the_sum = elem_1 + elem_2
11         elem_1, elem_2 = elem_2, the_sum
12     return the_sum
13
14
15 for n in range(1, 10):
16     print(n, "->", fib(n))
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL

```
PS C:\Users\marti\OneDrive\Desktop\Universidad\Cert
versidad/Certificaciones/main.py
1 -> 1
2 -> 1
3 -> 2
4 -> 3
5 -> 5
6 -> 8
7 -> 13
8 -> 21
9 -> 34
```

4.5 TUPLAS Y DICCIONARIOS

Bienvenido a la sección seis, donde aprenderás sobre los tipos de secuencias y el concepto de mutabilidad. Aprenderás sobre las tuplas y los diccionarios, y cómo puedes usarlos para almacenar y procesar valores de datos. ¡Vamos!

4.5.1 Tipos de secuencia y mutabilidad

Antes de comenzar a hablar acerca de tuplas y diccionarios, se deben introducir dos conceptos importantes: tipos de secuencia y mutabilidad.

Un tipo de secuencia es un tipo de dato en Python el cual es capaz de almacenar más de un valor (o ninguno si la secuencia esta vacía), los cuales pueden ser secuencialmente (de ahí el nombre) examinados, elemento por elemento.

Debido a que el bucle for es una herramienta especialmente diseñada para iterar a través de las secuencias, podemos definir las de la siguiente manera: una secuencia es un tipo de dato que puede ser escaneado por el bucle for.

Hasta ahora, has trabajado con una secuencia en Python - la lista. La lista es un clásico ejemplo de una secuencia de Python. Aunque existen otras secuencias dignas de mencionar, las cuales se presentaran a continuación.

La segunda noción - la mutabilidad - es una propiedad de cualquier tipo de dato en Python que describe su disponibilidad para poder cambiar libremente durante la ejecución de un programa. Existen dos tipos de datos en Python: mutables e inmutables.

Los datos mutables pueden ser actualizados libremente en cualquier momento - a esta operación se le denomina "in situ".

In situ es una expresión en Latín que se traduce literalmente como en posición, en el lugar o momento. Por ejemplo, la siguiente instrucción modifica los datos "in situ":


```
list.append(1)
```

Los datos inmutables no pueden ser modificados de esta manera.

Imagina que una lista solo puede ser asignada y leída. No podrías adjuntar ni remover un elemento de la lista. Si se agrega un elemento al final de la lista provocaría que la lista se cree desde cero.

Se tendría que crear una lista completamente nueva, la cual contenga los elementos ya existentes más el nuevo elemento.

El tipo de datos que se desea tratar ahora se llama **tupla**. Una **tupla es una secuencia inmutable**. Se puede comportar como una lista pero no puede ser modificada en el momento.

4.5.2 Tuplas

Lo primero que distingue una lista de una tupla es la sintaxis empleada para crearlas - las **tuplas utilizan paréntesis**, mientras que las listas usan corchetes, aunque también es **posible crear una tupla tan solo separando los valores por comas**.

```
1 tuple_1 = (1, 2, 4, 8)
2 tuple_2 = 1., .5, .25, .125
3
4 print(tuple_1)
5 print(tuple_2)
6
7
8
```

Console

```
(1, 2, 4, 8)
(1.0, 0.5, 0.25, 0.125)
```

Nota: **cada elemento de una tupla puede ser de distinto tipo** (punto flotante, entero, cadena, o cualquier otro tipo de dato).

4.5.2.1 ¿Cómo crear una tupla?

¿Es posible crear una tupla vacía? Si, solo se necesitan unos paréntesis:

```
1 empty_tuple = ()
```

Si se desea crear una tupla de **un solo elemento**, se debe de considerar el hecho de que, debido a la sintaxis (una tupla debe de poder distinguirse de un valor entero ordinario), se debe de colocar una coma al final:

```
one_element_tuple_1 = (1, )
one_element_tuple_2 = 1.,
```

El quitar las comas no arruinará el programa en el sentido sintáctico, pero serán dos variables, no tuplas.

```
main.py > ...
1 my_tuple = (1, 10, 100, 1000)
2
3 print(my_tuple[0])
4 print(my_tuple[-1])
5 print(my_tuple[1:])
6 print(my_tuple[:-2])
7
8 for elem in my_tuple:
9     print(elem)
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN

```
PS C:\Users\marti\OneDrive\Desktop\Unive
versidad/Certificaciones/main.py
1
1000
(10, 100, 1000)
(1, 10)
1
10
100
1000
```

Las similitudes pueden ser engañosas - **no intentes modificar el contenido de la tupla** ¡No es una lista!

Todas estas instrucciones (con excepción de primera) causarían un error de ejecución:

```
1 my_tuple = (1, 10, 100, 1000)
2
3 my_tuple.append(10000)
4 del my_tuple[0]
5 my_tuple[1] = -10
6
```

¿Qué más pueden hacer las tuplas?

- la función `len()` acepta tuplas, y regresa el número de elementos contenidos dentro;
- el operador `+` puede unir tuplas (ya se ha mostrado esto antes)
- el operador `*` puede multiplicar las tuplas, así como las listas;
- los operadores `in` y `not in` funcionan de la misma manera que en las listas.

El fragmento de código en el editor presenta todo esto.

```
main.py > ...
1 my_tuple = (1, 10, 100)
2
3 t1 = my_tuple + (1000, 10000)
4 t2 = my_tuple * 3
5
6 print(len(t2))
7 print(t1)
8 print(t2)
9 print(10 in my_tuple)
10 print(-10 not in my_tuple)
11
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN

```
PS C:\Users\marti\OneDrive\Desktop\Unive
versidad/Certificaciones/main.py
9
(1, 10, 100, 1000, 10000)
(1, 10, 100, 1, 10, 100, 1, 10, 100)
True
True
```

Una de las propiedades de las tuplas más útiles es que pueden **aparecer en el lado izquierdo del operador de asignación**. Este fenómeno ya se vio con anterioridad, cuando fue necesario encontrar una manera de intercambiar los valores entre dos variables.

```
1 var = 123
2
3 t1 = (1, )
4 t2 = (2, )
5 t3 = (3, var)
6
7 t1, t2, t3 = t2, t3, t1
8
9 print(t1, t2, t3)
10
11
```

Console

(2,) (3, 123) (1,)

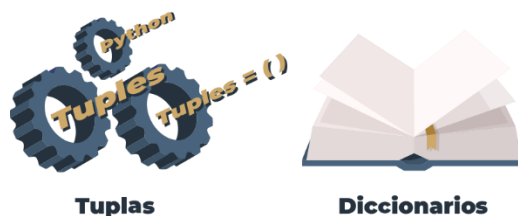
Muestra tres tuplas interactuando - en efecto, los valores almacenados en ellas "circulan" entre ellas - t1 se convierte en t2, t2 se convierte en t3, y t3 se convierte en t1.

Nota: el ejemplo presenta un importante hecho mas: los **elementos de una tupla pueden ser variables**, no solo literales. Además, pueden ser expresiones si se encuentran en el lado derecho del operador de asignación.

4.5.3 Diccionarios

El diccionario es otro tipo de estructura de datos de Python. No es una secuencia (pero puede adaptarse fácilmente a un procesamiento secuencial) y además es mutable.

Para explicar lo que es un diccionario en Python, es importante comprender de manera literal lo que es un diccionario.



4.5.3.1 ¿Cómo crear un diccionario?

Si deseas asignar algunos pares iniciales a un diccionario, utiliza la siguiente sintaxis:

```
1 dictionary = {"gato": "chat", "perro": "chien", "caballo": "cheval"}
2 phone_numbers = {'jefe': 5551234567, 'Suzy': 22657854310}
3 empty_dictionary = {}
4
5 print(dictionary)
6 print(phone_numbers)
7 print(empty_dictionary)
8
9
10
```

Console

{'gato': 'chat', 'perro': 'chien', 'caballo': 'cheval'}

{'jefe': 5551234567, 'Suzy': 22657854310}

{}

En este primer ejemplo, el diccionario emplea claves y valores las cuales ambas son cadenas. En el segundo, las claves son cadenas pero los valores son enteros. El orden inverso (claves → números, valores → cadenas) también es posible, así como la combinación número a número.

La lista de todos los pares es **encerrada con llaves**, mientras que los pares son **separados por comas**, y las **claves y valores por dos puntos**.

El primer diccionario es muy simple, es un diccionario Español-Francés. El segundo es un directorio telefónico muy pequeño.

Los diccionarios vacíos son construidos por **un par vacío de llaves** - nada inusual.

Un diccionario en Python funciona de la misma manera que **un diccionario bilingüe**. Por ejemplo, se tiene la palabra en español "gato" y se necesita su equivalente en francés. Lo que se haría es buscar en el diccionario para encontrar la palabra "gato". Eventualmente la encontrarás, y sabrás que la palabra equivalente en francés es "chat".

En el mundo de Python, la palabra que se está buscando se denomina key. La palabra que se obtiene del diccionario es denominada value.

Esto significa que un diccionario es un conjunto de pares de **key y value**. Nota:

- cada clave debe de ser **única** - no es posible tener una clave duplicada;
- una clave puede ser **un dato de cualquier tipo** - puede ser un número (entero o flotante), incluso una cadena, pero no una lista;
- un diccionario no es una lista - una lista contiene un conjunto de valores numerados, mientras que un **diccionario almacena pares de valores**;
- la función len() aplica también para los diccionarios - regresa la cantidad de pares (clave-valor) en el diccionario;
- un diccionario es **una herramienta de un solo sentido** - si fuese un diccionario español-francés, podríamos buscar en español para encontrar su contraparte en francés más no viceversa.

¿Has notado que el orden de los pares impresos es diferente a la asignación inicial? ¿Qué significa esto?

Primeramente, recordemos que los diccionarios no son listas - no guardan el orden de sus datos, el orden no tiene significado (a diferencia de los diccionarios reales). El orden en que un diccionario almacena sus datos está fuera de nuestro control. Esto es normal. (*)

Nota: En Python 3.6x los diccionarios se han convertido en colecciones ordenadas de manera predeterminada. Tu resultado puede variar dependiendo en la versión de Python que se esté utilizando.

4.5.3.2 ¿Cómo utilizar un diccionario?

Analiza el código en el editor:

```
1 dictionary = {"gato": "chat", "perro": "chien", "caballo": "cheval"}
2 phone_numbers = {'jefe': 5551234567, 'Suzy': 22657854310}
3 empty_dictionary = {}
4
5 # Imprimir valores aquí.
6 print(dictionary['gato'])
7 print(phone_numbers['Suzy'])
8
9
```

Console

chat
22657854310

El obtener el valor de un diccionario es semejante a la indexación, gracias a los corchetes alrededor del valor de la clave.

Nota:

- si una clave es una cadena, se tiene que especificar como una cadena;
- las claves son sensibles a las mayúsculas y minúsculas: 'Suzy' sería diferente a 'suzy'.

Ahora algo muy importante: **No se puede utilizar una clave que no exista.**

Nota: Cuando escribes una expresión grande o larga, puede ser una buena idea mantenerla alineada verticalmente. Así es como puede hacer que el código sea más legible y amigable para el programador, por ejemplo:

```
1 # Ejemplo 1:
2 dictionary = {
3     "gato": "chat",
4     "perro": "chien",
5     "caballo": "cheval"
6 }
7 # Ejemplo 2:
8 phone_numbers = {'jefe': 5551234567,
9                  'Suzy': 22657854310
10 }
```

Este tipo de formato se llama **sangría francesa**.

4.5.4 Métodos y funciones de los diccionarios

4.5.4.1 El método `keys()`

¿Pueden los diccionarios ser examinados utilizando el bucle for, como las listas o tuplas?

No y sí.

No, porque un diccionario no es un tipo de dato secuencial - el bucle for no es útil aquí.

Sí, porque hay herramientas simples y muy efectivas que pueden adaptar cualquier diccionario a los requerimientos del bucle for (en otras palabras, se construye un enlace intermedio entre el diccionario y una entidad secuencial temporal).

El primero de ellos es un método denominado `keys()`, el cual es parte de todo diccionario. El método retorna o regresa una lista de todas las claves dentro del diccionario. Al tener una lista de claves se puede acceder a todo el diccionario de una manera fácil y útil.

A continuación, se muestra un ejemplo:

```
1 dictionary = {"gato": "chat", "perro": "chien", "caballo": "cheval"}
2
3 for key in dictionary.keys():
4     print(key, "->", dictionary[key])
5
6
7
```

Console

```
gato -> chat
perro -> chien
caballo -> cheval
```

Otra manera de hacerlo es utilizar el método `items()`. Este método **retorna una lista de tuplas** (este es el primer ejemplo en el que las tuplas son mas que un ejemplo de si mismas) **donde cada tupla es un par de cada clave con su valor**.

Así es como funciona:

```
1 dictionary = {"gato": "chat", "perro": "chien", "caballo": "cheval"}
2
3 for spanish, french in dictionary.items():
4     print(spanish, "->", french)
5
6
```

Console

```
gato -> chat
perro -> chien
caballo -> cheval
```

4.5.4.2 *Modificar, agregar y eliminar valores*

El asignar un nuevo valor a una clave existente es sencillo - debido a que los diccionarios son completamente mutables, no existen obstáculos para modificarlos.

Se va a reemplazar el valor "chat" por "minou", lo cual no es muy adecuado, pero funcionará con nuestro ejemplo.

Observa:

```
1 dictionary = {"gato": "chat", "perro": "chien", "caballo": "cheval"}
2
3 dictionary['gato'] = 'minou'
4 print(dictionary)
5
6
7
8
```

Console

```
{'gato': 'minou', 'perro': 'chien', 'caballo': 'cheval'}
```

4.5.4.3 La función sorted()

¿Deseas que la salida esté ordenada? Solo hay que agregar al bucle for lo siguiente:

```
for key in sorted(dictionary.keys()):
```

También existe un método denominado values(), funciona de manera muy similar al de keys(), pero **regresa una lista de valores**.

Este es un ejemplo sencillo:

```
1 dictionary = {"gato": "chat", "perro": "chien", "caballo": "cheval"}
2
3 for french in dictionary.values():
4     print(french)
5
```

Console

```
chat
chien
cheval
```

Como el diccionario no es capaz de automáticamente encontrar la clave de un valor dado, el rol de este método es algo limitado.

4.5.4.4 Agregando nuevas claves

El agregar una nueva clave con su valor a un diccionario es tan simple como cambiar un valor. Solo se tiene que asignar un valor a una nueva clave que no haya existido antes.

Nota: este es un comportamiento muy diferente comparado a las listas, las cuales no permiten asignar valores a índices no existentes.

A continuación, se agrega un par nuevo al diccionario - un poco extraño pero válido:

```
1 dictionary = {"gato": "chat", "perro": "chien", "caballo": "cheval"}
2
3 dictionary['cisne'] = 'cygne'
4 print(dictionary)
```

Console

```
{'gato': 'chat', 'perro': 'chien', 'caballo': 'cheval', 'cisne': 'cygne'}
```

Nota: También es posible insertar un elemento al diccionario utilizando el método update(), por ejemplo:

```
1 dictionary = {"gato": "chat", "perro": "chien", "caballo": "cheval"}
2
3 dictionary.update({"pato": "canard"})
4 print(dictionary)
5
6
```

Console

```
{'gato': 'chat', 'perro': 'chien', 'caballo': 'cheval', 'pato': 'canard'}
```

4.5.4.5 *Eliminado una clave*

¿Puedes deducir como eliminar una clave de un diccionario?

Nota: al eliminar la clave también se removerá el valor asociado. Los valores no pueden existir sin sus claves.

Esto se logra con la instrucción del.

A continuación, un ejemplo:

```
1 dictionary = {"gato": "chat", "perro": "chien", "caballo": "cheval"}
2
3 del dictionary['perro']
4 print(dictionary)
5
6
```

Console

```
{'gato': 'chat', 'caballo': 'cheval'}
```

Nota: el eliminar una clave no existente, provocará un error.

Para eliminar el ultimo elemento de la lista, se puede emplear el método popitem():

```
1 dictionary = {"gato": "chat", "perro": "chien", "caballo": "cheval"}
2
3 dictionary.popitem()
4 print(dictionary) # salida: {'gato': 'chat', 'perro': 'chien'}
5
```

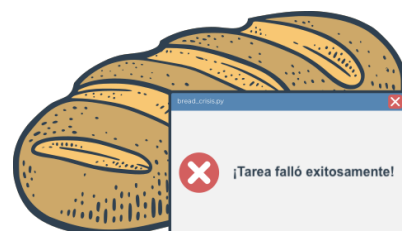
En versiones anteriores de Python, por ejemplo, antes de la 3.6.7, el método popitem() elimina un elemento al azar del diccionario.

4.6 EXCEPCIONES

Bienvenido a la sección 7, la última sección del curso, donde aprenderás sobre el mecanismo de manejo de excepciones en Python. Explorarás los temas de errores en el código y aprenderás qué puedes hacer para evitar la finalización de tus programas. También se cubrirá el tema de la pruebas y depuración de código, y aprenderás algunos consejos sobre cómo hacer que tu proceso de escritura de código sea mejor y menos propenso a errores. ¡Vamos a empezar!

4.6.1 Errores - el pan diario del desarrollador

Parece indiscutible que todos los programadores (incluso tú) quieren escribir código libre de errores y hacen todo lo posible para lograr este objetivo. Desafortunadamente, nada es perfecto en este mundo y el software no es una excepción. Presta atención a la palabra excepción, ya que la veremos de nuevo muy pronto en un significado que no tiene nada que ver con el común.



Errar es humano, es imposible no cometer errores y es imposible escribir código sin errores. No nos malinterpretes, no queremos convencerte de que escribir programas desordenados y

defectuosos es una virtud. Más bien queremos explicar que incluso el programador más cuidadoso no puede evitar defectos menores o mayores. Sólo aquellos que no hacen nada no cometen errores.

Paradójicamente, aceptar esta difícil verdad puede convertirte en un mejor programador y mejorar la calidad de tu código.

Te preguntarán: "¿Cómo podría ser esto posible?".

Intentaremos mostrártelo.

4.6.1.1 Errores en los datos frente a errores en el código

El lidiar con errores de programación tiene (al menos) dos partes. La primera es cuando te metes en problemas porque tu código, aparentemente correcto, se alimenta con datos incorrectos. Por ejemplo, esperas que se ingrese al código un valor entero, pero tu usuario descuidado ingresa algunas letras al azar.

Puede suceder que tu código termine en ese momento y el usuario se quede solo con un mensaje de error conciso y a la vez ambiguo en la pantalla. El usuario estará insatisfecho y tu también deberías estarlo.

Te mostraremos cómo proteger tu código de este tipo de fallas y cómo no provocar la ira del usuario.

La segunda parte de lidiar con errores de programación se revela cuando ocurre un comportamiento no deseado del programa debido a errores que se cometieron cuando se estaba escribiendo el código. Este tipo de error se denomina comúnmente "bug" (bicho en inglés), que es una manifestación de una creencia bien establecida de que, si un programa funciona mal, esto debe ser causado por bichos maliciosos que viven dentro del hardware de la computadora y causan cortocircuitos u otras interferencias.

Esta idea no es tan descabellada como puede parecer: incidentes de este tipo eran comunes en tiempos en que las computadoras ocupaban grandes pasillos, consumían kilovatios de electricidad y producían enormes cantidades de calor. Afortunadamente, o no, estos tiempos se han ido para siempre y los únicos errores que pueden estropear tu código son los que tú mismo sembraste en el código. Por lo tanto, intentaremos mostrarte cómo encontrar y eliminar tus errores, en otras palabras, cómo depurar tu código.

Comencemos el viaje por la tierra de los errores y las fallas o bugs.

Cuando los datos no son lo que deberían ser

4.6.2 La rama try-except

En el mundo de Python, hay una regla que dice: "Es mejor pedir perdón que pedir permiso".

Detengámonos aquí por un momento. No nos malinterpretes - no queremos que apliques la regla en tu vida diaria. No tomes el automóvil de nadie sin permiso, con la esperanza de que puedas ser tan convincente que evites la condena por lo ocurrido. La regla se trata de otra cosa.

En realidad, la regla dice: "es mejor manejar un error cuando ocurre que tratar de evitarlo".

"De acuerdo," puedes decir, "pero ¿cómo debo pedir perdón cuando el programa finaliza y no queda nada que más por hacer?" Aquí es donde algo llamado excepción entra en escena.

Observa el código en el editor:

```

1 - try:
2     # Es un lugar donde
3     # tu puedes hacer algo
4     # sin pedir permiso.
5 - except:
6     # Es un espacio dedicado
7     # exclusivamente para pedir perdón.
8

```

Puedes ver dos bloques aquí:

- el primero, comienza con la palabra clave reservada `try` este es el lugar donde se coloca el código que se sospecha que es riesgoso y puede terminar en caso de un error; nota: este tipo de error lleva por nombre **excepción**, mientras que la ocurrencia de la excepción se le denomina **generar** - podemos decir que se genera (o se generó) una excepción;
- el segundo, la parte del código que comienza con la palabra clave reservada `except` esta parte fue diseñada para manejar la excepción; depende de ti lo que quieras hacer aquí: puedes limpiar el desorden o simplemente puede barrer el problema debajo de la alfombra (aunque se prefiere la primera solución)

Entonces, podríamos decir que estos dos bloques funcionan así:

- la palabra clave reservada `try` marca el lugar donde intentas hacer algo sin permiso;
- la palabra clave reservada `except` comienza un lugar donde puedes mostrar tu talento para disculparte o pedir perdón.

Como puedes ver, este enfoque acepta errores (los trata como una parte normal de la vida del programa) en lugar de intensificar los esfuerzos para evitarlos por completo.

Resumamos lo que hemos hablado:

- cualquier fragmento de código colocado entre `try` y `except` se ejecuta de una manera muy especial: cualquier error que ocurra aquí dentro no terminará la ejecución del programa. En cambio, el control saltará inmediatamente a la primera línea situada después de la palabra clave reservada `except`, y no se ejecutará ninguna otra línea del bloque `try`;
- el código en el bloque `except` se activa solo cuando se ha encontrado una excepción dentro del bloque `try`. No hay forma de llegar por ningún otro medio;
- cuando el bloque `try` o `except` se ejecutan con éxito, el control vuelve al proceso normal de ejecución y cualquier código ubicado más allá en el archivo fuente se ejecuta como si no hubiera pasado nada.

Ahora queremos hacerte una pregunta: ¿Es `ValueError` la única forma en que el control podría caer dentro del bloque `except`?

4.6.3 Cómo lidiar con más de una excepción

La respuesta obvia es "no" - hay más de una forma posible de plantear una excepción. Por ejemplo, un usuario puede ingresar cero como entrada - ¿puedes predecir lo que sucederá a continuación?

Sí, tienes razón - la división colocada dentro de la invocación de la función `print()` generará la excepción `ZeroDivisionError`. Como es de esperarse, el comportamiento del código será el mismo que en el caso anterior - el usuario verá el mensaje "No se que hacer con...", lo que parece bastante razonable en este contexto, pero también es posible que desees manejar este tipo de problema de una manera un poco diferente.

¿Es posible? Por supuesto que lo es. Hay al menos dos enfoques que puedes implementar aquí.

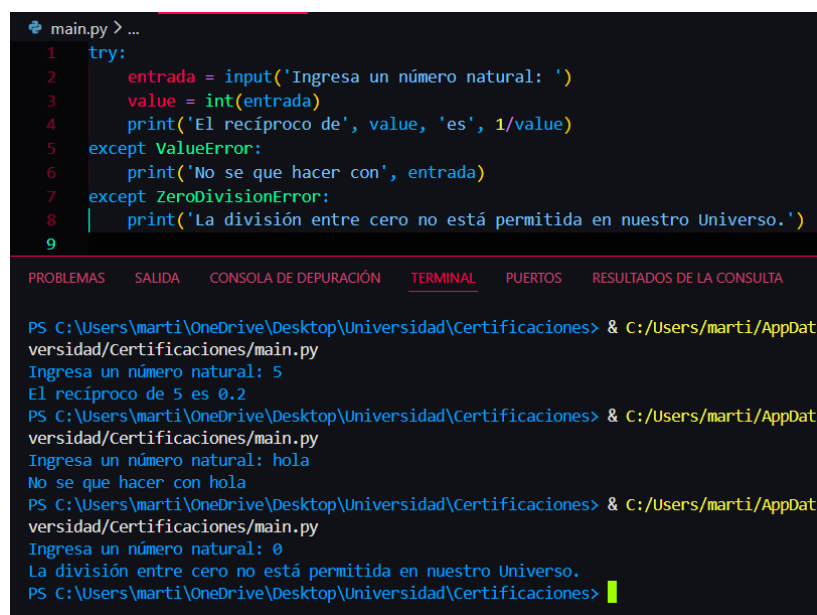
El primero de ellos es simple y complicado al mismo tiempo: puedes agregar dos bloques try por separado, uno que incluya la invocación de la función input() donde se puede generar la excepción ValueError y el segundo dedicado a manejar posibles problemas inducidos por la división. Ambos bloques try tendrían su propio except y de esa manera, tendrías un control total sobre dos errores diferentes.

Esta solución es buena, pero es un poco larga - el código se hincha innecesariamente. Además, no es el único peligro que te espera. Toma en cuenta que dejar el primer bloque try-except deja mucha incertidumbre - tendrás que agregar código adicional para asegurarte de que el valor que ingresó el usuario sea seguro para usar en la división. Así es como una solución aparentemente simple se vuelve demasiado complicada.

Afortunadamente, Python ofrece una forma más sencilla de afrontar este tipo de desafíos.

4.6.3.1 Dos excepciones después de un try

Observa el código en el editor. Como puedes ver, acabamos de agregar un segundo except. Esta no es la única diferencia - toma en cuenta que ambos except tienen nombres de excepción específicos. En esta variante, cada una de las excepciones esperadas tiene su propia forma de manejar el error, pero debe enfatizarse en que solo una de todas puede interceptar el control - si se ejecuta una, todas las demás permanecen inactivas.



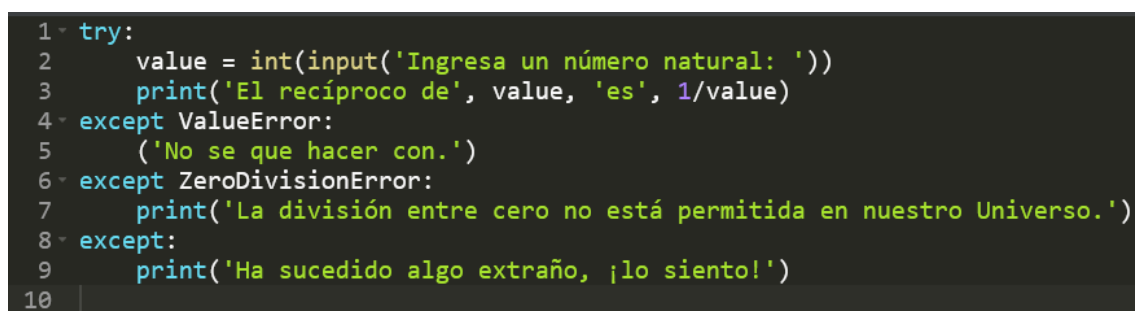
```
main.py > ...
1 try:
2     entrada = input('Ingresa un número natural: ')
3     value = int(entrada)
4     print('El recíproco de', value, 'es', 1/value)
5 except ValueError:
6     print('No se que hacer con', entrada)
7 except ZeroDivisionError:
8     print('La división entre cero no está permitida en nuestro Universo.')
9
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS RESULTADOS DE LA CONSULTA

```
PS C:\Users\marti\OneDrive\Desktop\Universidad\Certificaciones> & C:/Users/marti/AppData
versidad/Certificaciones/main.py
Ingresa un número natural: 5
El recíproco de 5 es 0.2
PS C:\Users\marti\OneDrive\Desktop\Universidad\Certificaciones> & C:/Users/marti/AppData
versidad/Certificaciones/main.py
Ingresa un número natural: hola
No se que hacer con hola
PS C:\Users\marti\OneDrive\Desktop\Universidad\Certificaciones> & C:/Users/marti/AppData
versidad/Certificaciones/main.py
Ingresa un número natural: 0
La división entre cero no está permitida en nuestro Universo.
PS C:\Users\marti\OneDrive\Desktop\Universidad\Certificaciones>
```

4.6.4 La excepción predeterminada y cómo usarla

El código ha cambiado de nuevo - ¿puedes ver la diferencia?



```
1 try:
2     value = int(input('Ingresa un número natural: '))
3     print('El recíproco de', value, 'es', 1/value)
4 except ValueError:
5     ('No se que hacer con.')
6 except ZeroDivisionError:
7     print('La división entre cero no está permitida en nuestro Universo.')
8 except:
9     print('Ha sucedido algo extraño, ¡lo siento!')
10
```

Hemos agregado un tercer except, pero esta vez no tiene un nombre de excepción específico - podemos decir que es anónimo o (lo que está más cerca de su función real) es el por defecto.

Puedes esperar que cuando se genere una excepción y no haya un except dedicado a esa excepción, esta será manejada por la excepción por defecto.

Nota: ¡el except por defecto debe ser el último except! ¡Siempre!

4.6.5 Algunas excepciones útiles

Analicemos con más detalle algunas excepciones útiles (o más bien, las más comunes) que puedes llegar a experimentar.

4.6.5.1 *ZeroDivisionError*

Esta aparece cuando intentas forzar a Python a realizar cualquier operación que provoque una división en la que el divisor es cero o no se puede distinguir de cero. Toma en cuenta que hay más de un operador de Python que puede hacer que se genere esta excepción. ¿Puedes adivinarlos todos?

Si, estos son: /, //, y %.

4.6.5.2 *ValueError*

Espera esta excepción cuando estás manejando valores que pueden usarse de manera inapropiada en algún contexto. En general, esta excepción se genera cuando una función (como int() o float()) recibe un argumento de un tipo adecuado, pero su valor es inaceptable.

4.6.5.3 *TypeError*

Esta excepción aparece cuando intentas aplicar un dato cuyo tipo no se puede aceptar en el contexto actual. Mira el ejemplo:

```
1 short_list = [1]
2 one_value = short_list[0.5]
3
```

No está permitido usar un valor flotante como índice de una lista (la misma regla también se aplica a las tuplas). TypeError es un nombre adecuado para describir el problema y una excepción adecuada a generar.

4.6.5.4 *AttributeError*

Esta excepción llega - entre otras ocasiones - cuando intentas activar un método que no existe en un elemento con el que se está tratando. Por ejemplo:

```
1 short_list = [1]
2 short_list.append(2)
3 short_list.depend(3)
4
```

La tercera línea de nuestro ejemplo intenta hacer uso de un método que no está incluido en las listas. Este es el lugar donde se genera la excepción AttributeError.

4.6.5.5 *SyntaxError*

Esta excepción se genera cuando el control llega a una línea de código que viola la gramática de Python. Puede sonar extraño, pero algunos errores de este tipo no se pueden identificar sin ejecutar primero el código. Este tipo de comportamiento es típico de los lenguajes interpretados - el intérprete siempre trabaja con prisa y no tiene tiempo para escanear todo el código fuente. Se conforma con comprobar el código que se está ejecutando en el momento. Muy pronto se te presentará un ejemplo de esta categoría.

Es una mala idea manejar este tipo de excepciones en tus programas. Deberías producir código sin errores de sintaxis, en lugar de enmascarar las fallas que has causado.

4.6.6 Por qué no puedes evitar probar tu código

Aunque vamos a resumir nuestras consideraciones excepcionales aquí, no creas que es todo lo que Python puede ofrecer para ayudarte a suplicar perdón. La maquinaria de excepciones de Python es mucho más compleja y sus capacidades te permiten desarrollar estrategias de manejo de errores expandidas. Volveremos a estos temas - lo prometemos. No dudes en realizar tus experimentos y sumergirte en las excepciones por ti mismo.

Ahora queremos contarte sobre el segundo lado de la lucha interminable contra los errores - el destino inevitable de la vida de un desarrollador. Como no puedes evitar la creación de errores en tu código, siempre debes estar listo para buscarlos y destruirlos. No entierres la cabeza en la arena - ignorar los errores no los hará desaparecer.

Un deber importante para los desarrolladores es probar el código recién creado, pero no debes olvidar que las pruebas no son una forma de demostrar que el código está libre de errores. Paradójicamente, lo único que las pruebas determinan, es que tu código contiene errores. No creas que puedes relajarte después de una prueba exitosa.

El segundo aspecto importante de las pruebas de software es estrictamente psicológico. Es una verdad conocida desde hace años que los autores - incluso aquellos que son confiables y conscientes de sí mismos - no pueden evaluar y verificar objetivamente sus trabajos.

Es por eso por lo que cada novelista necesita un editor y cada programador necesita un tester. Algunos dicen - con un poco de rencor, pero con sinceridad - que los desarrolladores prueban su código para mostrar su perfección, no para encontrar problemas que puedan frustrarlos. Los testers o probadores están libres de tales dilemas, y es por eso por lo que su trabajo es más efectivo y rentable.

Por supuesto, esto no te exime de estar atento y cauteloso. Prueba tu código lo mejor que puedas. No facilites demasiado el trabajo a los probadores.

Su deber principal es asegurarse de haber verificado todas las rutas o caminos de ejecución por las que puede pasar tu código. ¿Suena misterioso? ¡Por supuesto que no!

4.6.7 Pruebas, pruebas y más pruebas

La respuesta es más simple de lo esperado y también un poco decepcionante. Python - como seguramente sabes - es un lenguaje interpretado. Esto significa que el código fuente se analiza y ejecuta al mismo tiempo. En consecuencia, es posible que Python no tenga tiempo para analizar las líneas de código que no están sujetas a ejecución. Como dice un antiguo dicho de los desarrolladores: "es una característica, no un error" (no utilices esta frase para justificar el comportamiento extraño de tu código).

¿Entiendes ahora por qué el pasar por todos los caminos de ejecución es tan vital e inevitable?

Supongamos que terminas tu código y que las pruebas que has realizado son exitosas. Entregas tu código a los probadores y - ¡afortunadamente! - encontraron algunos errores en él. Estamos usando la palabra "afortunadamente" de manera completamente consciente. Debes aceptar que, en primer lugar, los probadores son los mejores amigos del desarrollador - no debes tratar a los errores que ellos encuentran como una ofensa o una malignidad; y, en segundo lugar, cada error que encuentran los probadores es un error que no afectará a los usuarios. Ambos factores son valiosos y merecen tu atención.

Ya sabes que tu código contiene un error o errores (lo segundo es más probable). Ahora, ¿cómo los localizas y cómo arreglas tu código?

4.6.7.1 *Error frente a depuración (Bug vs. debug)*

La medida básica que un desarrollador puede utilizar contra los errores es - como era de esperarse - un depurador, mientras que el proceso durante el cual se eliminan los errores del código se llama depuración. Según un viejo chiste, la depuración es un complicado juego de misterio en el que eres simultáneamente el asesino, el detective y - la parte más dolorosa de la intriga - la víctima. ¿Estás listo para interpretar todos estos roles? Entonces debes armarte con un depurador.

Un depurador es un software especializado que puede controlar cómo se ejecuta tu programa. Con el depurador, puedes ejecutar tu código línea por línea, inspeccionar todos los estados de las variables y cambiar sus valores en cualquier momento sin modificar el código fuente, detener la ejecución del programa cuando se cumplen o no ciertas condiciones, y hacer muchas otras tareas útiles.



Podemos decir que todo IDLE está equipado con un depurador más o menos avanzado. Incluso IDLE tiene uno, aunque puedes encontrar su manejo un poco complicado y problemático. Si deseas utilizar el depurador integrado de IDLE, debes activarlo mediante la opción "Debug" en la barra de menú de la ventana principal de IDLE. Es el punto de partida para la depuración.

4.6.8 **print debugging (depuración)**

Esta forma de depuración, que se puede aplicar a tu código mediante cualquier tipo de depurador, a veces se denomina depuración interactiva. El significado del término se explica por sí mismo - el proceso necesita su interacción (la del desarrollador) para que se lleve a cabo.

Algunas otras técnicas de depuración se pueden utilizar para cazar errores. Es posible que no puedas o no quieras usar un depurador (las razones pueden variar). ¿Estás entonces indefenso? ¡Absolutamente no!

Puedes utilizar una de las tácticas de depuración más simples y antiguas (pero aún útil) conocida como la depuración por impresión. El nombre habla por sí mismo - simplemente insertas varias invocaciones `print()` adicionales dentro de tu código para generar datos que ilustran la ruta que tu

código está negociando actualmente. Puedes imprimir los valores de las variables que pueden afectar la ejecución.

Estas impresiones pueden generar texto significativo como "Estoy aquí", "Ingresé a la función foo()", "El resultado es 0", o pueden contener secuencias de caracteres que solo tu puedes leer. Por favor, no uses palabras obscenas o indecentes para ese propósito, aunque puedas sentir una fuerte tentación - tu reputación puede arruinarse en un momento si estas payasadas se filtran al público.

Como puedes ver, este tipo de depuración no es realmente interactiva en lo absoluto, o es interactiva solo en pequeña medida, cuando decides aplicar la función `input()` para detener o retrasar la ejecución del código.

Una vez que se encuentran y eliminan los errores, las impresiones adicionales pueden comentarse o eliminarse - tu decides. No permitas que se ejecuten en el código final - pueden confundir tanto a los probadores como a los usuarios, y traer mal karma sobre ti.

4.6.9 Algunos consejos útiles

Aquí hay algunos consejos que pueden ayudarte a encontrar y eliminar errores. Ninguno de ellos es definitivo. Úsalos de manera flexible y confía en tu intuición. No te creas a ti mismo - comprueba todo dos veces.

- Intenta decirle a alguien (por ejemplo, a tu amigo o compañero de trabajo) qué es lo que se espera que haga tu código y cómo se espera que se comporte. Sé concreto y no omitas detalles. Responde todas las preguntas que te hagan. Es probable que te des cuenta de la causa del problema mientras cuentas tu historia, ya que el hablar activa esas partes de tu cerebro que permanecen inactivas mientras codificas. Si ningún humano puede ayudarte con el problema, usa un patito amarillo de goma en su lugar. No estamos bromeando - consulta el artículo de Wikipedia para obtener más información sobre esta técnica de uso común: Rubber Duck Debugging.
- Intenta aislar el problema. Puedes extraer la parte de tu código que se sospecha que es responsable de tus problemas y ejecutarla por separado. Puedes comentar partes del código para ocultar el problema. Asigna valores concretos a las variables en lugar de leerlos desde la consola. Prueba tus funciones aplicando valores de argumentos predecibles. Analiza el código cuidadosamente. Léelo en voz alta.
- Si el error apareció recientemente y no había aparecido antes, analiza todos los cambios que has introducido en tu código - uno de ellos puede ser la razón.
- Tómate un descanso, bebe una taza de café, toma a tu perro y sal a caminar, lee un buen libro, incluso dos, haz una llamada telefónica a tu mejor amigo - te sorprenderás de la frecuencia con la que esto ayuda.
- Se optimista - eventualmente encontrarás el error; te lo prometemos.

4.6.10 Pruebas unitarias - un mayor nivel de codificación

También existe una técnica de programación importante y ampliamente utilizada que tendrás que adoptar tarde o temprano durante tu carrera de desarrollador - se llama prueba unitaria. El nombre puede ser un poco confuso, ya que no se trata solo de probar el software, sino también (y, sobre todo) de cómo se escribe el código.

Para resumir la historia - las pruebas unitarias asumen que las pruebas son partes inseparables del código y la preparación de los datos de prueba es una parte inseparable de la codificación. Esto significa que cuando escribes una función o un conjunto de funciones cooperativas, también estás obligado a crear un conjunto de datos para los cuales el comportamiento de tu código es predecible y conocido.

Además, debes equipar a tu código con una interfaz que pueda ser utilizada por un entorno de pruebas automatizado. En este enfoque, cualquier enmienda realizada al código (incluso la menos significativa) debe ir seguida de la ejecución de todas las pruebas unitarias que acompañan al código fuente.

Para estandarizar este enfoque y facilitar su aplicación, Python proporciona un módulo dedicado llamado unittest. No vamos a discutirlo aquí - es un tema amplio y complejo.